Lecture Notes on

Numerical Linear Algebra

Michael C. Lehn

Ulm University

Winter Semester 2024/2025

Contents

1 Introduction **5**

2 Computer Arithmetic and Conditioning **6**
§1 Machine Numbers and Floating-Point Arithmetic 8
§2 Condition Numbers of a Problem 13

3 Direct Methods for Solving Linear Systems **14**
§1 Gaussian Elimination 14
§2 Real Symmetric Positive Definite Matrices 38
§3 Conditioning of Linear Systems 44

4 Linear Least Squares Problems **51**
§1 The Method of Least Squares 51
§2 QR Factorization: Householder Transformations 54
§3 QR Factorization with Givens Rotations 65
§4 Singular Value Decomposition 71

5 Numerical Computation of Eigenvalues and Eigenvectors **75**
§1 Estimating the Location of Eigenvalues 75
§2 The Power Method 82
§3 Wielandt's Inverse Iteration and Shift Strategies 85
§4 The QR Algorithm 86

6 Iterative Methods for Solving Linear Systems **89**
§1 Classical Iterative Methods – Splitting Methods 89
§2 The Gradient Method 95
§3 The Conjugate Gradient Method 97

A Supplements to the Chapters **105**
§1 Supplement to Chapter 3 105
§1.1 Supplement to §1 105

B Linear Algebra Miscellany **110**
§1 Principal Axis Transformation 110
§2 The Spectral Theorem 114
§3 Norms in Finite-Dimensional Vector Spaces 118
§3.1 Basic Definitions 119
§3.2 Equivalence of Norms: Motivation and Definition 120

§3.3 Equivalence of the p-Norms in $\mathbb{R}^n$ and $\mathbb{C}^n$ 122
§3.4 Equivalence of Norms in Finite-Dimensional Vector Spaces 125
§3.5 Completeness: Finite-Dimensional Normed Vector Spaces Are Always Banach Spaces 127

List of Algorithms

1 lu (row-wise) . . . 24
2 lu (column-wise) . . . 25
3 Classical Cholesky factorization . . . 43

4 QR factorization with Givens rotations . . . 68
5 Computation of $Q^T b$ from the stored Givens rotations . . . 69

6 Power method . . . 83
7 Wielandt's inverse iteration with spectral shift . . . 86
8 QR iteration . . . 86

9 Gradient method . . . 96
10 Conjugate gradient method (first version) . . . 100
11 Conjugate gradient method (final version) . . . 103

Listings

3.1 LU factorization (vectorized) 23
3.2 LU factorization row-wise 24
3.3 LU factorization column-wise 25

Chapter 1 Introduction

> "Numerical linear algebra is the continuation of linear algebra by other means."
>
> – Carl von Clausewitz, 1812[1]

Put simply, numerical mathematics is concerned with solving mathematical problems on a computer. Numerical linear algebra deals specifically with problems from linear algebra, such as solving systems of linear equations on a computer.

Numerical methods are examined not only with regard to their correctness, but also with regard to their accuracy and efficiency. At first glance, this may seem unusual: Why not simply solve a problem exactly? And why study several methods instead of directly choosing the supposedly "best" one? We shed light on the answers to these questions in Chapter 2.

From Chapter 3 onward, we then turn to the numerical methods themselves. At their core, they are based on familiar concepts from linear algebra, but they are now considered under the particular demands of numerical computation.

[1] Source: Der Postillon – Honest news – independent, fast, since 1845.

Chapter 2 Computer Arithmetic and Conditioning

This chapter is about the two keywords in its title. We first give an overview by means of small examples, which also show why the choice of algorithm matters, and then treat machine numbers (§1) and the condition of a problem (§2) in more detail.

Computer arithmetic. In numerical computation (as opposed to symbolic computation) we compute with numbers that a computer can represent. This has two consequences:

- Not every number can be represented exactly on a computer.
- Not every arithmetic operation ($+$, $-$, $\cdot$, $/$, …) can be carried out exactly.

A few examples in MATLAB:

```
1/3 * 3                                  % seems to work
1/3 * 1/5 * 1/7 - 1/3 * (1/5 * 1/7)      % result is not 0
(900/9 - 899/9) * 9 - 1                  % result is not 0
```

Conditioning. The condition of a problem is a measure of how difficult it is to solve the problem — even under ideal conditions, that is, when numbers can be represented exactly and arithmetic operations can be carried out exactly.

Example. Consider $f : \mathbb{R} \to \mathbb{R}$, $x \mapsto f(x) = 2^x$ and

$$x_1 = 100 \;\Rightarrow\; y_1 = f(x_1) = 2^{100}, \qquad x_2 = 101 \;\Rightarrow\; y_2 = f(x_2) = 2^{101}.$$

For the absolute changes we get

$$\underbrace{|x_1 - x_2| = 1}_{\text{abs. change of the input}} \quad \text{and} \quad \underbrace{|y_1 - y_2| = 2^{101} - 2^{100} = 2^{100}}_{\text{abs. change of the output}},$$

or, in relative terms,

$$\underbrace{\frac{|x_1 - x_2|}{|x_1|} = 1\,\%}_{\text{rel. change of the input}} \quad \text{and} \quad \underbrace{\frac{|y_1 - y_2|}{|y_1|} = 100\,\%}_{\text{rel. change of the output}}.$$

Stability. In practice we face the following problem:

1. Input values may be perturbed, because they are not known exactly or cannot be represented exactly (e.g., 1/3 cannot be represented exactly).
2. Arithmetic operations may introduce further errors, i.e., errors can be amplified.

The first point can, in general, not be avoided. The second point means that the "computational path", i.e., the algorithm, can be crucial:

- A *stable* algorithm does not amplify errors significantly.
- An *unstable* algorithm, in contrast, may do so.

Example. Consider the problem $f : \mathbb{R} \to \mathbb{R}$, $x \mapsto \sqrt{x+1} - \sqrt{x}$ (for $x \geq 0$).

Alg. 1: $y = \sqrt{x+1} - \sqrt{x} = f(x)$.
By an algorithm we mean that the following steps are carried out:

$$(1)\ t_1 \leftarrow x+1, \quad (2)\ t_2 \leftarrow \sqrt{t_1}, \quad (3)\ t_3 \leftarrow \sqrt{x}, \quad (4)\ y \leftarrow t_2 - t_3.$$

Alg. 2: $y = \dfrac{1}{\sqrt{x+1}+\sqrt{x}} = f(x)$, i.e.,

$$(1)\ t_1 \leftarrow x+1, \quad (2)\ t_2 \leftarrow \sqrt{t_1}, \quad (3)\ t_3 \leftarrow \sqrt{x}, \quad (4)\ t_4 \leftarrow t_2 + t_3, \quad (5)\ y \leftarrow 1/t_4.$$

Both algorithms compute the same function in exact arithmetic. A comparison in MATLAB for $x = 10^{10}$:

```
x = 1e10;
y1 = sqrt(x + 1) - sqrt(x)          % Alg. 1
y2 = 1 / (sqrt(x + 1) + sqrt(x))    % Alg. 2
```

In IEEE double precision one obtains

$$y_1 \approx 4.999994416721165 \cdot 10^{-6}, \qquad y_2 \approx 4.999999999875000 \cdot 10^{-6},$$

while the exact value is $f(10^{10}) = 4.99999999987500\ldots \cdot 10^{-6}$. Alg. 1 delivers only about six correct digits. We will see the reason in Example 2.1.6 (c).

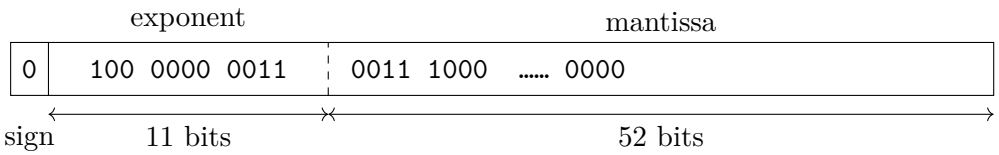
Sources and further reading. A thorough treatment of floating-point arithmetic and rounding error analysis is given by Higham [10] and Overton [16]; the binary formats are specified in the IEEE 754 standard [13].

§1 Machine Numbers and Floating-Point Arithmetic

Internally, MATLAB computes with 64-bit floating-point numbers.[1]

Example 2.1.1

The bit pattern



is interpreted as follows. The value of the exponent is

$$E = 1 \cdot 2^{10} + 0 \cdot 2^9 + \cdots + 1 \cdot 2^1 + 1 \cdot 2^0 = 1024 + 2 + 1 = 1027,$$

and the value of the mantissa is

$$M = 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} + 1 \cdot 2^{-5} = \frac{1}{8} + \frac{1}{16} + \frac{1}{32}.$$

The represented value is

$$x = (-1)^{\text{sign}} \cdot (1+M) \cdot 2^{E-1023} = (-1)^0 \cdot \left(1 + \frac{1}{8} + \frac{1}{16} + \frac{1}{32}\right) \cdot \underbrace{2^4}_{=16} = 16+2+1+\frac{1}{2} = 19.5.$$

Remark 2.1.2

For this representation the so-called *IEEE 754 format* is used. It additionally contains special cases for representing special values:

- an encoding of $+0$ and -0 (so there are two representations of zero!),
- "Infinity" and "$-$Infinity",
- "NaN" ("not a number").

Numbers "close to zero" (the case $E = 0$) are treated separately; for them

$$x = (-1)^{\text{sign}} \cdot M \cdot 2^{-1022}.$$

This *denormalized range* serves to approximate numbers close to zero better (see Figure 2.1). It consists of the numbers

$$4.94 \cdot 10^{-324} \approx 2^{-1074} \;\leq\; |x| \;<\; 2^{-1022} \approx 2.23 \cdot 10^{-308}.$$

Important for us:

1. Some numbers can be represented exactly, e.g.,

[1] There are exceptions in which MATLAB computes with integers, where this is possible.

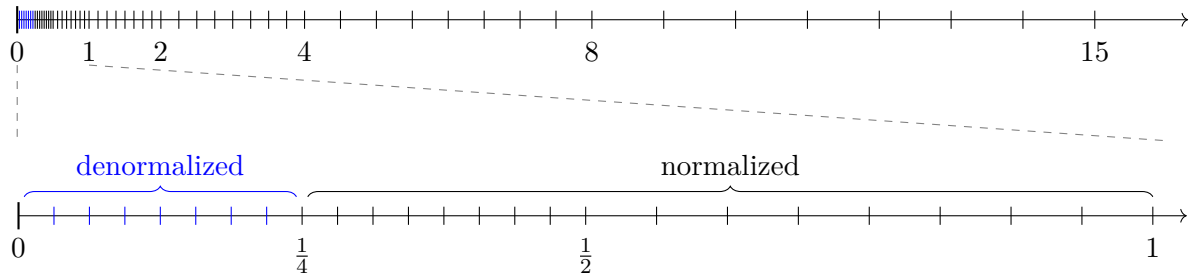


Figure 2.1: Positive numbers of a toy floating-point system with three mantissa bits and exponents $-2, \ldots, 3$ (top) and a zoom into $[0, 1]$ (bottom). The denormalized numbers (blue) fill the gap between 0 and the smallest normalized number $\frac{1}{4}$.

- integers that are not too large in absolute value,
- real numbers that can be written as a sum of powers of two — at least if the exponents are neither too large nor too small, and as long as the mantissa has enough bits. For example, $x = 2^{20} + 2^{-40}$ cannot be represented.

2. Some numbers cannot be represented exactly. For example, $x = \frac{1}{3}$ can only be written as a series (not as a finite sum) of powers of two:

$$\frac{1}{3} = \sum_{k=1}^{\infty} 2^{-2k} \qquad \text{(check this!)}$$

Definition 2.1.3

a) The set $\mathbb{M}$ *denotes the numbers that can be represented on a computer ("machine numbers"). In IEEE double precision (see Example 2.1.1) the normalized machine numbers are*

$$x = \pm(1 + M) \cdot 2^e, \quad M = \sum_{k=1}^{52} b_k \cdot 2^{-k},\ b_k \in \{0, 1\}, \quad e \in \{-1022, \ldots, 1023\}.$$

b) Real numbers can be mapped to machine numbers by rounding. Different rounding modes are common; in general, rounding can be described by a function

$$\mathrm{fl} : \mathbb{R} \to \mathbb{M} \qquad (\textit{"float"}).$$

c) The distance from 1 *to the next larger machine number is called the* machine epsilon *and is denoted by* $\varepsilon_{\text{mach}}$*. The* unit roundoff u *is the bound for the relative rounding error in d); for rounding to nearest*

$$u = \tfrac{1}{2}\,\varepsilon_{\text{mach}}.$$

d) We assume that $\mathbb{M}$ *and* fl *have the property*

$$\underbrace{\left|\frac{x - \mathrm{fl}(x)}{x}\right|}_{\substack{\textit{relative error when rounding} \\ x \textit{ to a machine number}}} \leq u$$

for all $x \in \mathbb{R}$ with $x \neq 0$. Since $u < 1$, it follows that $|x - \mathrm{fl}(x)| < |x|$. For IEEE 754 this holds as long as $\mathrm{fl}(x) \neq \pm\infty$ and x does not lie in the denormalized range, i.e., $2^{-1022} \leq |x|$.

e) For $x \in \mathbb{R}$ and an approximation $\widetilde{x} \in \mathbb{R}$ (e.g., $\widetilde{x} = \mathrm{fl}(x)$) one calls

$$e_{abs} = |x - \widetilde{x}|$$

the absolute error *and (if $x \neq 0$)*

$$e_{rel} = \frac{|x - \widetilde{x}|}{|x|}$$

the relative error*; for $\widetilde{x} = \mathrm{fl}(x)$ these are the errors made when rounding x to a machine number.*

Remark 2.1.4

a) $\mathbb{M} \not\subseteq \mathbb{R}$, since $\mathbb{M}$ also contains "Infinity", "−Infinity", "−0" and "NaN". But $\mathbb{M}' := \mathbb{M} \cap \mathbb{R} \neq \emptyset$ (the representable real numbers).

b) There are different rounding modes. IEEE 754 provides four of them:

- *rounding to nearest* (the default): $\mathrm{fl}(x)$ is the machine number closest to x. In case of a tie, the one whose last mantissa bit is 0 is taken ("round half to even"). This avoids the systematic bias one would get by always rounding ties upwards.
- *directed rounding*: towards 0 (truncation), towards $+\infty$ or towards $-\infty$.

Example: in the toy system of Figure 2.1 (three mantissa bits) the number $1 + \frac{1}{16}$ lies exactly halfway between $1 = (1.000)_2$ and $1 + \frac{1}{8} = (1.001)_2$ and is rounded to 1, whereas $1 + \frac{3}{16}$ is rounded to $1 + \frac{1}{4} = (1.010)_2$.

The machine epsilon $\varepsilon_{\text{mach}}$ depends only on the set $\mathbb{M}$, the unit roundoff u also on the rounding mode. To see this, let $2^e \leq |x| < 2^{e+1}$ (normalized range). There the machine numbers have the constant spacing $\varepsilon_{\text{mach}} \cdot 2^e$, the value of the last mantissa bit. Rounding to nearest moves x by at most half of this spacing, directed rounding by less than a full spacing:

$$|x - \mathrm{fl}(x)| \leq \tfrac{1}{2}\,\varepsilon_{\text{mach}} \cdot 2^e \leq \tfrac{1}{2}\,\varepsilon_{\text{mach}} \cdot |x| \quad \text{resp.} \quad |x - \mathrm{fl}(x)| < \varepsilon_{\text{mach}} \cdot 2^e \leq \varepsilon_{\text{mach}} \cdot |x|.$$

Hence $u = \frac{1}{2}\varepsilon_{\text{mach}}$ for rounding to nearest and $u = \varepsilon_{\text{mach}}$ for directed rounding.

c) For IEEE 754 double precision (binary64),

$$\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \cdot 10^{-16} \quad \text{and, for rounding to nearest,} \quad u = 2^{-53} \approx 1.11 \cdot 10^{-16}.$$

MATLAB's `eps` returns $\varepsilon_{\text{mach}}$.

Rounding errors can also arise from arithmetic operations.

Assumption 2.1.5 (Standard model of floating-point arithmetic)

For $x, y \in \mathbb{M} \cap \mathbb{R}$ and $\square \in \{+, -, \cdot, /\}$ (with $y \neq 0$ for $/$) we assume that

$$\mathrm{fl}(x \square y) = (x \square y)(1 + \delta) \quad \textit{with} \quad |\delta| \leq u,$$

provided that no overflow or underflow occurs.

This is a model, not a theorem: it describes what we require of the computer arithmetic. IEEE 754 arithmetic satisfies it, since every basic operation must deliver the correctly rounded exact result. The model allows us to analyze how errors propagate when an expression is evaluated, and to estimate the maximal error.

Remark: For directed rounding the model holds with $u = \varepsilon_{\text{mach}}$. In any case $|\delta| \in \mathcal{O}(\varepsilon_{\text{mach}})$.

Example 2.1.6 ("Forward analysis")

a) The expression $y = (a \cdot b) \cdot c$ is computed by the following algorithm for computing $\widetilde{y}$:

$$\begin{aligned} t_1 &= \mathrm{fl}\big(\mathrm{fl}(a) \cdot \mathrm{fl}(b)\big) = \mathrm{fl}(a) \cdot \mathrm{fl}(b) \cdot (1 + \delta_1), \\ \widetilde{y} &= \mathrm{fl}\big(t_1 \cdot \mathrm{fl}(c)\big) = t_1 \cdot \mathrm{fl}(c) \cdot (1 + \delta_2) = \mathrm{fl}(a) \cdot \mathrm{fl}(b) \cdot \mathrm{fl}(c) \cdot (1 + \delta_1)(1 + \delta_2) \end{aligned}$$

with $\mathrm{fl}(a) = a(1 + \delta_a)$, $\mathrm{fl}(b) = b(1 + \delta_b)$, $\mathrm{fl}(c) = c(1 + \delta_c)$. Hence

$$\widetilde{y} = a \cdot b \cdot c \cdot (1 + \delta_a)(1 + \delta_b)(1 + \delta_c)(1 + \delta_1)(1 + \delta_2)$$

and therefore

$$e_{\text{abs}} = |y - \widetilde{y}|, \qquad e_{\text{rel}} = \frac{|y - \widetilde{y}|}{|y|} \quad (\text{if } y \neq 0).$$

Even for this mini example, computing e_{abs} and e_{rel} exactly is not practical.

The way out: we only determine the order of magnitude by ignoring terms with factors of the form $\delta_1 \cdots \delta_r$ $(r > 1)$, in particular $\delta_1 \cdot \delta_2$ ("to first order", denoted by $\doteq$). This yields

$$\begin{aligned} \widetilde{y} &\doteq a \cdot b \cdot c + a \cdot b \cdot c \cdot (\delta_a + \delta_b + \delta_c + \delta_1 + \delta_2) \\ &= a \cdot b \cdot c \cdot (1 + \delta) \quad \text{with} \quad |\delta| \leq 5u = \mathcal{O}(u), \end{aligned}$$

so

$$e_{\text{abs}} \doteq |\delta \cdot a \cdot b \cdot c|, \qquad e_{\text{rel}} \doteq |\delta|.$$

The relative error remains of the order of magnitude of u.

b) When computing with machine numbers, the associative law does *not* hold in general. Consider two different algorithms for computing $y_1 = y_2$:

$$\begin{aligned} &(1)\ y_1 = (a \cdot b) \cdot c: \quad t_1 = \mathrm{fl}\big(\mathrm{fl}(a) \cdot \mathrm{fl}(b)\big), \quad \widetilde{y}_1 = \mathrm{fl}\big(t_1 \cdot \mathrm{fl}(c)\big), \\ &(2)\ y_2 = a \cdot (b \cdot c): \quad t_2 = \mathrm{fl}\big(\mathrm{fl}(b) \cdot \mathrm{fl}(c)\big), \quad \widetilde{y}_2 = \mathrm{fl}\big(\mathrm{fl}(a) \cdot t_2\big). \end{aligned}$$

The rounding errors in $\widetilde{y}_1$ and $\widetilde{y}_2$ can be different. A MATLAB example with $a = 0.1$, $b = 0.2$, $c = 0.3$:

```
a = 0.1; b = 0.2; c = 0.3;
(a * b) * c - a * (b * c)    % result is not 0
```

The errors in $\widetilde{y}_1$ and $\widetilde{y}_2$ are, however, both of the same order of magnitude $\mathcal{O}(u)$.

Remark: The distributive law does not hold in general either. The commutative law, however, does hold.

c) "Cancellation". For $y = a - b$ we obtain

$$\begin{aligned}\widetilde{y} &= \mathrm{fl}\big(\mathrm{fl}(a) - \mathrm{fl}(b)\big) = \mathrm{fl}\big(a(1+\delta_a) - b(1+\delta_b)\big) = \big[a(1+\delta_a) - b(1+\delta_b)\big](1+\delta) \\ &= a(1+\delta_a)(1+\delta) - b(1+\delta_b)(1+\delta) \doteq \big[a + a\cdot\underbrace{(\delta_a+\delta)}_{=:\delta_1}\big] - \big[b + b\cdot\underbrace{(\delta_b+\delta)}_{=:\delta_2}\big] \\ &= (a-b) + a\delta_1 - b\delta_2 \qquad \text{with} \quad |\delta_1|, |\delta_2| \le 2u \ \ \text{(to first order)}.\end{aligned}$$

For $y \neq 0$ it follows that

$$e_{\mathrm{rel}} = \left|\frac{y-\widetilde{y}}{y}\right| \doteq \frac{|a\delta_1 - b\delta_2|}{|a-b|} \le \frac{|a|\cdot|\delta_1| + |b|\cdot|\delta_2|}{|a-b|} \le \max\{|\delta_1|,|\delta_2|\} \cdot \underbrace{\frac{|a|+|b|}{|a-b|}}_{\text{large for } a\approx b}.$$

Equality holds in the last estimate, e.g., for $\delta_1 = u$, $\delta_2 = -u$ and $a, b > 0$. For $a = b + \varepsilon$ with $b, \varepsilon > 0$ the factor is $\frac{2b+\varepsilon}{\varepsilon} = \frac{2b}{\varepsilon} + 1$.

A numerical example: The machine numbers next to 1234 from below are $1234 - 2^{-42}$ and 1234; their midpoint is $m := 1234 - 2^{-43}$. We choose two numbers that lie extremely close to m, one on each side:

$$a = m + \underbrace{2^{-567}}_{\approx 10^{-171}}, \qquad b = m - 2^{-567}.$$

With rounding to nearest, a is rounded up and b is rounded down:

$$\widetilde{a} = \mathrm{fl}(a) = 1234, \qquad \widetilde{b} = \mathrm{fl}(b) = 1234 - 2^{-42} \approx 1234 - 2.27\cdot 10^{-13}.$$

Both relative rounding errors are tiny (below u). Hence

$$a - b = 2\cdot 2^{-567} \approx 4.14\cdot 10^{-171}, \qquad \widetilde{a} - \widetilde{b} = 2^{-42} \approx 2.27\cdot 10^{-13},$$

and therefore

$$e_{\mathrm{rel}} = \frac{2^{-42} - 2^{-566}}{2^{-566}} = 2^{524} - 1 \approx 5.5\cdot 10^{157} \quad \text{(somewhat large } \ddot{\smile}\text{)},$$

but $e_{\mathrm{abs}} \approx 2.27\cdot 10^{-13}$ (small).

Conclusion:

- As a final result, $\widetilde{y} = \widetilde{a} - \widetilde{b}$ only tells us "very small".
- Continuing to compute with it (e.g., to compute $z = \frac{1}{a-b}$) makes no sense.

This also explains the behavior of Alg. 1 in the example on page 7: for large x we have $\sqrt{x+1} \approx \sqrt{x}$, and step (4) subtracts two nearly equal numbers. Alg. 2 avoids this subtraction.

§2 Condition Numbers of a Problem

Goal: a measure of the "sensitivity of the solution to perturbations in the data".

Let $f : \mathbb{R} \to \mathbb{R}$ be twice continuously differentiable. Then for $x, \Delta x \in \mathbb{R}$

$$f(x + \Delta x) = f(x) + f'(x)\Delta x + \frac{1}{2} f''(x + \xi \Delta x)(\Delta x)^2$$

for some $\xi \in (0, 1)$. If $f(x) \neq 0$, it follows that

$$\frac{f(x + \Delta x) - f(x)}{f(x)} = \frac{f'(x) \cdot \Delta x}{f(x)} + \frac{f''(x + \xi \Delta x)}{2 \cdot f(x)} \cdot (\Delta x)^2.$$

Since f'' is continuous, $C := \max\limits_{\xi \in [0,1]} \left| \frac{f''(x+\xi\Delta x)}{2 \cdot f(x)} \right|$ exists. With the triangle inequality we obtain

$$\underbrace{\left| \frac{f(x + \Delta x) - f(x)}{f(x)} \right|}_{\text{rel. change of the result}} \leq \left| \frac{f'(x)}{f(x)} \right| \cdot \underbrace{|\Delta x|}_{\substack{\text{abs. change} \\ \text{of the argument}}} + C \cdot |\Delta x|^2 = \left| \frac{f'(x) \cdot x}{f(x)} \right| \cdot \underbrace{\frac{|\Delta x|}{|x|}}_{\substack{\text{rel. change} \\ \text{of the argument}}} + C \cdot |\Delta x|^2$$

(for $x \neq 0$). With

$$\begin{aligned} x &\mathrel{\widehat{=}} \text{exact argument}, \\ \widetilde{x} := x + \Delta x &\mathrel{\widehat{=}} \text{argument perturbed by } \Delta x, \\ y := f(x) &\mathrel{\widehat{=}} \text{exact result}, \\ \widetilde{y} := f(x + \Delta x) &\mathrel{\widehat{=}} \text{result for the perturbed argument (and exact computation)} \end{aligned}$$

we therefore have

$$\left| \frac{\widetilde{y} - y}{y} \right| \leq \kappa_{\text{rel}}(x) \cdot \left| \frac{\widetilde{x} - x}{x} \right| + \underbrace{C \cdot (\Delta x)^2}_{\in \mathcal{O}((\Delta x)^2)}.$$

Definition 2.2.1 (Relative condition number)

For f as above and $x \neq 0$ with $f(x) \neq 0$,

$$\kappa_{rel}(x) := \left| \frac{f'(x) \cdot x}{f(x)} \right|$$

is called the relative condition number of f at x. *If the function needs to be named explicitly, we also write $\kappa_f(x)$.*

For $0 < |\Delta x| \ll 1$ (i.e., $\widetilde{x} \approx x$) we have $C \cdot (\Delta x)^2 \approx 0$, and for $x \neq 0$, $y \neq 0$

$$\underbrace{\left| \frac{\widetilde{y} - y}{y} \right|}_{\text{rel. output error}} \approx \kappa_{\text{rel}}(x) \cdot \underbrace{\left| \frac{\widetilde{x} - x}{x} \right|}_{\text{rel. input error}}.$$

Chapter 3 Direct Methods for Solving Linear Systems

Goal: Solve linear systems $Ax = b$ for x.

(a) If A is a nonsingular triangular matrix, the problem can be solved relatively easily.

(b) In §1 we present a method that, for a nonsingular matrix A, determines a permutation matrix P and two nonsingular triangular matrices L and U such that $P \cdot A = L \cdot U$ holds. Afterwards, the methods described in (a) can be used to compute the solution.

Sources and further reading. The presentation of the LU factorization in §1, in particular the characterization of its existence in terms of the leading principal submatrices, the factorization $P_1AP_2 = LU$ of a nonsingular matrix, and the block-wise derivation of Gaussian elimination with partial pivoting, owes much to James Demmel's textbook *Applied Numerical Linear Algebra* [4]. Storing the row interchanges in a pivot vector corresponds to the convention used in LAPACK [1]. The method of §2 is due to André-Louis Cholesky; it was published posthumously by Benoît [3]. A comprehensive treatment of the perturbation theory of §3, and of rounding error analysis in general, can be found in Higham's monograph [10].

§1 Gaussian Elimination

First, a brief overview of some basics from linear algebra that are particularly helpful in this section.

Definition 3.1.1 (Permutation matrix)

A permutation matrix P is an identity matrix with permuted rows.

Lemma 3.1.2 (Properties)

If P, P_1 and P_2 are $n \times n$ permutation matrices and X is an $n \times n$ matrix, then:

a) $P \cdot X$ *is* X *with permuted rows.*

b) $X \cdot P$ *is* X *with permuted columns.*

c) $P^{-1} = P^T$

d) $detP = \pm 1$

e) $P_1 \cdot P_2$ *is a permutation matrix.*

Remark 3.1.3

a) In this course, a vector $x \in \mathbb{R}^n$ denotes a column vector, and thus x^T denotes a row vector.

b) As a rule, uppercase letters are used for matrices and the corresponding lowercase letter for their entries:

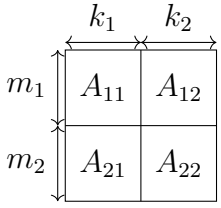
$$X \in \mathbb{R}^{m \times n} \text{ with } X = (x_{ij})_{1 \le i \le m,\, 1 \le j \le n}$$

c) For $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$, $C \in \mathbb{R}^{m \times n}$ the matrix product $C = A \cdot B$ is defined by

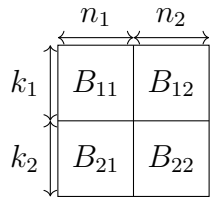
$$c_{ij} = \sum_{\ell=1}^{k} a_{i\ell} \cdot b_{\ell j}, \quad 1 \le i \le m,\ 1 \le j \le n$$

d) Matrices can also be viewed blockwise, for example:

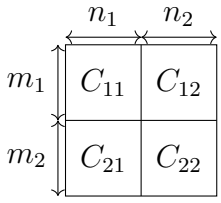
$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \in \mathbb{R}^{m \times k}$$



$$B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \in \mathbb{R}^{k \times n}$$



$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \in \mathbb{R}^{m \times n}$$



with $m_1 + m_2 = m$, $n_1 + n_2 = n$ and $k_1 + k_2 = k$. Then the matrix product $C = A \cdot B$ can be computed blockwise by

$$\begin{aligned} C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \\ &= \begin{pmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{pmatrix} = A \cdot B \end{aligned}$$

Caution: in general, the commutative law does not hold for matrices.

Lemma 3.1.4 (Gaussian elimination without row interchanges)

For an $n \times n$ matrix A the following statements are equivalent:

(1) There exist a unit lower triangular matrix L and a nonsingular upper triangular matrix U with

$$A = L \cdot U$$

(2) All leading principal submatrices are nonsingular.

Moreover: If the LU factorization in (1) exists, then the matrices L and U are uniquely determined.

Proof:

(1) $\Rightarrow$ (2)
For $1 \leq j \leq n$ let

$$A_{11} \text{ be the leading } j \times j \text{ principal submatrix of } A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix},$$
$$L_{11} \text{ be the leading } j \times j \text{ principal submatrix of } L = \begin{pmatrix} L_{11} & \\ L_{21} & L_{22} \end{pmatrix}$$
$$U_{11} \text{ be the leading } j \times j \text{ principal submatrix of } U = \begin{pmatrix} U_{11} & U_{12} \\ & U_{22} \end{pmatrix}$$

The determinant of a triangular matrix is the product of its diagonal elements. Hence, here we have

$$\det L = 1 = \det L_{11}$$
$$\det U = \prod_{i=1}^{n} u_{ii} \neq 0 \;\Rightarrow\; \det U_{11} = \prod_{i=1}^{j} u_{ii} \neq 0$$

By assumption $A = L \cdot U$, and thus

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = A = L \cdot U = \begin{pmatrix} L_{11} & \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ & U_{22} \end{pmatrix}$$
$$= \begin{pmatrix} L_{11} \cdot U_{11} & L_{11} \cdot U_{12} \\ L_{21} \cdot U_{11} & L_{21} \cdot U_{12} + L_{22} \cdot U_{22} \end{pmatrix}$$

Hence $A_{11} = L_{11} \cdot U_{11}$, and it follows that

$$\det A_{11} = \det\left(L_{11} \cdot U_{11}\right) = \underbrace{\det L_{11}}_{=1} \cdot \underbrace{\det U_{11}}_{\neq 0} \neq 0$$

(2) $\Rightarrow$ (1)
We prove this direction by induction on n.

Base case: $n = 1$
We have $A = A_{11} = (a_{11})$ with $a_{11} \neq 0$. With the nonsingular triangular matrices $L = (1)$ and $U = (a_{11})$, statement (1) is satisfied.

Induction step: $n - 1 \to n$

To bring the induction hypothesis into play, one can use the following ansatz:

$$\begin{pmatrix} \tilde{A} & b \\ c^T & \delta \end{pmatrix} = A \stackrel{!}{=} L \cdot U = \begin{pmatrix} \tilde{L} & \\ \ell^T & 1 \end{pmatrix} \begin{pmatrix} \tilde{U} & u \\ & \eta \end{pmatrix} = \begin{pmatrix} \tilde{L} \cdot \tilde{U} & \tilde{L} \cdot u \\ \ell^T \tilde{U} & \ell^T \cdot u + \eta \end{pmatrix}$$

where $\tilde{A}$, $\tilde{L}$ and $\tilde{U}$ are the leading $(n-1) \times (n-1)$ principal submatrices of A, L and U, respectively.

$$\left[\begin{array}{l} \text{The dimensions of the other blocks are thereby implicitly determined:} \\ c^T \text{ and } \ell^T \text{ are row vectors of dimension } 1 \times (n-1), \\ b \text{ and } u \text{ are column vectors of dimension } (n-1) \times 1, \\ \delta \text{ and } \eta \text{ are } 1 \times 1 \text{ blocks, i.e., scalars.} \end{array}\right]$$

By the induction hypothesis there exist unique nonsingular triangular matrices $\tilde{L}$ and $\tilde{U}$ with $\tilde{A} = \tilde{L} \cdot \tilde{U}$. For the remaining blocks the following must hold:

$$\begin{aligned} b &= \tilde{L} \cdot u \\ c^T &= \ell^T \tilde{U} \;\Leftrightarrow\; c = \tilde{U}^T \cdot \ell \\ \delta &= \ell^T \cdot u + \eta \end{aligned}$$

These equations uniquely determine the remaining blocks of the ansatz by

$$\begin{aligned} u &= \tilde{L}^{-1} \cdot b \\ \ell &= \tilde{U}^{-T} \cdot c \\ \eta &= \delta - \ell^T \cdot u \end{aligned}$$

That L has only ones on its diagonal follows from the ansatz. The ansatz also implies that U is an upper triangular matrix. Because of

$$0 \neq \det A = \underbrace{\det L}_{=1} \cdot \det U = \underbrace{\det \tilde{U}}_{\neq 0} \cdot \eta$$

U is also nonsingular.

□

It is interesting that, in its proof, this existence and uniqueness theorem describes an algorithm with which the LU factorization can actually be computed. Whether this algorithm is practical for carrying out the factorization with pencil and paper is another question. Nevertheless, we want to try it on a small example, because it is not only instructive but also gives us the opportunity to show how the computation can be reproduced in MATLAB.

Example 3.1.5 Let $A = \begin{pmatrix} 6 & 7 & 14 \\ 2 & 1 & 3 \\ 4 & 4 & 7 \end{pmatrix}$; we compute the LU factorizations of the leading principal submatrices A_{11}, A_{22} and A_{33} with the method described in the previous proof.

a) For $A_{11} = (6)$ we obtain the factorization $A_{11} = L_{11} \cdot U_{11}$ from the base case:

$$L_{11} = (1) \text{ and } U_{11} = (6).$$

b) For $A_{22} = \left(\begin{array}{c|c} 6 & 7 \\ \hline 2 & 1 \end{array}\right) = \begin{pmatrix} A_{11} & b \\ c^T & \delta \end{pmatrix}$, $L_{22} = \begin{pmatrix} L_{11} & \\ \ell^T & 1 \end{pmatrix}$ and $U_{22} = \begin{pmatrix} U_{11} & u \\ & \eta \end{pmatrix}$ form the ansatz for the factorization $A_{22} = L_{22} \cdot U_{22}$. The unknowns u, ℓ and η can be determined as in the induction step. In this case the computation is simple:

$$\begin{aligned} b = 7 &\stackrel{!}{=} L_{11} \cdot u = 1 \cdot u && \Rightarrow u = 7 \\ c = 2 &\stackrel{!}{=} U_{11}^T \cdot \ell = 6 \cdot \ell && \Rightarrow \ell = \tfrac{2}{6} = \tfrac{1}{3} \\ \eta &= \delta - \ell^T \cdot u = 1 - \tfrac{1}{3} \cdot 7 = -\tfrac{4}{3} \end{aligned}$$

This gives

$$L_{22} = \begin{pmatrix} 1 & \\ \frac{1}{3} & 1 \end{pmatrix}, U_{22} = \begin{pmatrix} 6 & 7 \\ & -\frac{4}{3} \end{pmatrix}.$$

c) For $A_{33} = \left(\begin{array}{cc|c} 6 & 7 & 14 \\ 2 & 1 & 3 \\ \hline 4 & 4 & 7 \end{array}\right) = \begin{pmatrix} A_{22} & b \\ c^T & \delta \end{pmatrix}$, $L_{33} = \begin{pmatrix} L_{22} & \\ \ell^T & 1 \end{pmatrix}$ and $U_{33} = \begin{pmatrix} U_{22} & u \\ & \eta \end{pmatrix}$ form the ansatz for the factorization $A_{33} = L_{33} \cdot U_{33}$. In this case the computation is somewhat more laborious, since the unknowns u and ℓ each have to be determined by forward substitution:

$$b = \begin{pmatrix} 14 \\ 3 \end{pmatrix} \stackrel{!}{=} L_{22} \cdot u = \begin{pmatrix} 1 & \\ \frac{1}{3} & 1 \end{pmatrix} \cdot u \quad \stackrel{\text{forward substitution}}{\Longrightarrow} u = \begin{pmatrix} 14 \\ -\frac{5}{3} \end{pmatrix}$$

$$c = \begin{pmatrix} 4 \\ 4 \end{pmatrix} \stackrel{!}{=} U_{22}^T \cdot \ell = \begin{pmatrix} 6 & \\ 7 & -\frac{4}{3} \end{pmatrix} \cdot \ell \quad \stackrel{\text{forward substitution}}{\Longrightarrow} \ell = \begin{pmatrix} \frac{2}{3} \\ \frac{1}{2} \end{pmatrix}$$

$$\eta = \delta - \ell^T \cdot u = 7 - \begin{pmatrix} \frac{2}{3} & \frac{1}{2} \end{pmatrix} \cdot \begin{pmatrix} 14 \\ -\frac{5}{3} \end{pmatrix} = -\tfrac{3}{2}$$

This gives

$$L_{33} = \begin{pmatrix} 1 & & \\ \frac{1}{3} & 1 & \\ \frac{2}{3} & \frac{1}{2} & 1 \end{pmatrix}, \quad U_{33} = \begin{pmatrix} 6 & 7 & 14 \\ & -\frac{4}{3} & -\frac{5}{3} \\ & & -\frac{3}{2} \end{pmatrix}.$$

d) This example can be reproduced with the following MATLAB script:

```
A = [ 6   7  14;
      2   1   3;
      4   4   7];

n = size(A, 2);        % n = number of columns of A
```

```
 6 L = eye(n);           % L = n x n identity matrix
 7 U = zeros(n);         % U = n x n zero matrix
 8
 9 for j = 1:n
10     if j == 1
11         % Corresponds to the base case
12         L(1,1) = 1;
13         U(1,1) = A(1,1);
14     else
15         % Corresponds to the induction step
16         % 1) b, c, delta correspond to the blocks in A
17         b = A(1:j-1, j);
18         c = A(j, 1:j-1).';
19         delta = A(j, j);
20
21         % 2) determine u and l by forward substitution,
22         %    then simply compute eta
23         u = L(1:j-1, 1:j-1) \ b;
24         l = U(1:j-1, 1:j-1).' \ c;
25         eta = delta - l.' * u;
26
27         % 3) insert l, u, eta into L and U.
28         L(j, 1:j-1) = l.';
29         U(1:j-1, j) = u;
30         U(j, j) = eta;
31     end
32 end
33
34 % print L, U and L * U
35 disp('L ='); disp(L);
36 disp('U ='); disp(U);
37 disp('L * U ='); disp(L * U);
```

In line 6 the matrix L is initialized with the identity matrix, so that in each step only the block below the diagonal has to be set. In MATLAB, matrix blocks can be accessed conveniently. For a 3×3 matrix A, for example, we have:

$$A = \left(\begin{array}{cc|c} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \end{array}\right) \mathrel{\widehat{=}} \left(\begin{array}{c|c} \texttt{A(1:2, 1:2)} & \texttt{A(1:2, 3)} \\ \hline \texttt{A(3, 1:2)} & \texttt{A(3,3)} \end{array}\right)$$

Accordingly, for a $j \times j$ matrix:

$$A = \left(\begin{array}{ccc|c} a_{11} & \dots & a_{1,j-1} & a_{1,j} \\ \vdots & & \vdots & \vdots \\ \hline a_{j-1,1} & \dots & a_{j-1,j-1} & a_{j-1,j} \\ \hline a_{j,1} & \dots & a_{j,j-1} & a_{j,j} \end{array}\right) \mathrel{\widehat{=}} \left(\begin{array}{c|c} \texttt{A(1:j-1, 1:j-1)} & \texttt{A(1:j-1, j)} \\ \hline \texttt{A(j, 1:j-1)} & \texttt{A(j, j)} \end{array}\right)$$

In MATLAB, the range operator `from:to` is used to select row and column ranges of a block. The MATLAB code accesses these blocks in L and U.

In line 23 the backslash operator is used to solve $b = L_{j-1,j-1}u$ for u. Since $L_{j-1,j-1}$ is a lower triangular matrix, MATLAB does this by forward substitution.

In MATLAB, the transposed matrix is obtained with the dot-apostrophe operator:

$$A^T \quad \widehat{=} \quad \texttt{A.'}$$

This operator is used in lines 18, 24, 25 and 28.

It is great that one can derive a programmable algorithm from a proof. However, it would be just as interesting to develop an algorithm that is suitable for computations with pencil and paper and is easy to remember.

Example 3.1.6 (Gaussian elimination as you know it, or should know it)

The following computation shows how a matrix A is transformed into row echelon form. In the first step, zeros are created below a_{11}, and in the second step, zeros below a_{22}.

$$\begin{pmatrix} 6 & 7 & 14 \\ 2 & 1 & 3 \\ 4 & 4 & 7 \end{pmatrix} \begin{matrix} \cdot\frac{1}{3} \\ - \\ \\ \end{matrix} \begin{matrix} \\ \\ \cdot\frac{2}{3} \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 6 & 7 & 14 \\ 0 & -\frac{4}{3} & -\frac{5}{3} \\ 0 & -\frac{2}{3} & -\frac{7}{3} \end{pmatrix} \begin{matrix} \\ \cdot\frac{1}{2} \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 6 & 7 & 14 \\ 0 & -\frac{4}{3} & -\frac{5}{3} \\ 0 & 0 & -\frac{3}{2} \end{pmatrix}$$

The procedure always follows the pattern that a multiple of one row is subtracted from another row. The multiple used in a computation step is noted as a multiplier at the arrows.

Interestingly, this procedure yields at the end the matrix U of the LU factorization, and this is no coincidence. We will see that the matrix L can also easily be read off from this computation. To this end, it is helpful to adapt the notation slightly. The positions where a zero was created can be used to store the multiplier that was used. The regions overwritten in this way can be marked by separating lines:

$$\begin{pmatrix} 6 & 7 & 14 \\ 2 & 1 & 3 \\ 4 & 4 & 7 \end{pmatrix} \begin{matrix} \cdot\frac{1}{3} \\ - \\ \\ \end{matrix} \begin{matrix} \\ \\ \cdot\frac{2}{3} \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 6 & 7 & 14 \\ \frac{1}{3} & -\frac{4}{3} & -\frac{5}{3} \\ \frac{2}{3} & -\frac{2}{3} & -\frac{7}{3} \end{pmatrix} \begin{matrix} \\ \cdot\frac{1}{2} \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 6 & 7 & 14 \\ \frac{1}{3} & -\frac{4}{3} & -\frac{5}{3} \\ \frac{2}{3} & \frac{1}{2} & -\frac{3}{2} \end{pmatrix}$$

Below the separating line, the entries of L below the diagonal are visible, i.e., the strictly lower triangular part of L. One can therefore read off

$$L = \begin{pmatrix} 1 & & \\ \frac{1}{3} & 1 & \\ \frac{2}{3} & \frac{1}{2} & 1 \end{pmatrix} \quad \text{and} \quad U = \begin{pmatrix} 6 & 7 & 14 \\ & -\frac{4}{3} & -\frac{5}{3} \\ & & -\frac{3}{2} \end{pmatrix}.$$

Lemma 3.1.7 (Gaussian elimination for the LU factorization)

Let A be an $n \times n$ matrix for which a unique LU factorization with nonsingular matrix U exists.

1. *In the case $n = 1$, $L = (1)$ and $U = A$.*
2. *In the case $n > 1$, consider the block structures of the matrices*

$$A = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix}, \quad L = \begin{pmatrix} 1 & \\ \ell_{21} & L_{22} \end{pmatrix} \quad \text{and} \quad U = \begin{pmatrix} u_{11} & u_{12}^T \\ & U_{22} \end{pmatrix}$$

with $(n-1)\times(n-1)$ *matrix blocks* A_{22}, L_{22} *and* U_{22}. *Then*

$$u_{11} = a_{11},\ u_{12}^T = a_{12}^T,\ \ell_{21} = \tfrac{1}{a_{11}} \cdot a_{21}$$

and L_{22} *and* U_{22} *are obtained from the LU factorization of*

$$\tilde{A}_{22} := A_{22} - \ell_{21} \cdot u_{12}^T$$

Proof:
Because of the uniqueness of the factorization, one only has to multiply out the factorization and show that it can be solved for the sought blocks of L and U.

The case $n = 1$ is clear, and for $n > 1$ we have

$$A = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \stackrel{!}{=} \begin{pmatrix} 1 & \\ \ell_{21} & L_{22} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12}^T \\ & U_{22} \end{pmatrix} = \begin{pmatrix} u_{11} & u_{12}^T \\ \ell_{21} \cdot u_{11} & \ell_{21} \cdot u_{12}^T + L_{22} \cdot U_{22} \end{pmatrix}$$

It follows that

$$u_{11} = a_{11},\ u_{12}^T = a_{12}^T,\ \ell_{21} = \tfrac{1}{a_{11}} \cdot a_{21}$$

and

$$L_{22} \cdot U_{22} = \tilde{A}_{22} = A_{22} - \ell_{21} \cdot u_{12}^T$$

□

The proof describes a recursive algorithm for computing the LU factorization, which can be implemented directly in `MATLAB`. The following code is kept as close as possible to the mathematical notation of the proof, but places no emphasis on efficiency or clever programming.

```
function [L, U] = lu_rec(A)
    n = size(A, 2);

    if n == 1
        L = 1;
        U = A;
    else
        a11 = A(1,1);
        a21 = A(2:n, 1);
        a12 = A(1, 2:n)';
        A22 = A(2:n, 2:n);

        u11 = a11;
        u12 = a12;
        l21 = 1/a11 * a21;
        A22_ = A22 - l21 * u12'; % A22_ stands for "A22 tilde"

        [L22, U22] = lu_rec(A22_);

        nix = zeros(n-1, 1);

        L = [1,    nix';
             l21, L22];

        U = [u11, u12';
             nix, U22];
    end
end
```

Example 3.1.8

The recursive algorithm can also be rewritten as an iterative algorithm. Concretely, this means that a loop is used instead. In this example we derive the iterative algorithm informally.

a) In the first step, A is transformed into a new matrix:

$$A = A^{(0)} = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \rightsquigarrow A^{(1)} = \begin{pmatrix} u_{11} & u_{12}^T \\ \ell_{21} & \widetilde{A}_{22} \end{pmatrix} \rightsquigarrow \dots$$

with

$$u_{11} = a_{11},\ u_{12} = a_{12},\ \ell_{21} = a_{21}/a_{11} \quad \text{and} \quad \widetilde{A}_{22} = A_{22} - \ell_{21} \cdot u_{12}^T$$

This notation distinguishes precisely between the original matrix $A = A^{(0)}$ and the newly created matrix $A^{(1)}$. To compute $A^{(1)}$, therefore, only a_{21} has to be replaced by ℓ_{21} and then A_{22} by $\widetilde{A}_{22}$, which can be expressed by

$$\begin{aligned} a_{21} &\leftarrow a_{21}/a_{11} \\ A_{22} &\leftarrow A_{22} - a_{21} \cdot a_{12}^T \end{aligned} \tag{3.1}$$

If the context makes clear that blocks are being replaced, the notation can be simplified so that the same names are used for the blocks in $A^{(0)}$ and $A^{(1)}$:

$$A = A^{(0)} = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \overset{(3.1)}{\rightsquigarrow} A^{(1)} = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \rightsquigarrow \dots$$

In MATLAB, the first step for an $n \times n$ matrix can be implemented by

```
A(2:n, 1) = A(2:n, 1) / A(1, 1);
A(2:n, 2:n) = A(2:n, 2:n) - A(2:n, 1) * A(1, 2:n);
```

b) In the second step, only the $(n-1) \times (n-1)$ block

$$A_{22} = \begin{pmatrix} a_{22} & a_{23}^T \\ a_{32} & A_{33} \end{pmatrix}$$

of $A^{(1)}$ is replaced by means of

$$\begin{aligned} a_{32} &\leftarrow a_{32}/a_{22} \\ A_{33} &\leftarrow A_{33} - a_{32} \cdot a_{23}^T \end{aligned}$$

and thus transformed into the matrix $A^{(2)}$. One proceeds analogously to the first step, and likewise for the MATLAB implementation:

```
A(3:n, 2) = A(3:n, 2) / A(2, 2);
A(3:n, 3:n) = A(3:n, 3:n) - A(3:n, 2) * A(2, 3:n);
```

c) The matrix A has been completely transformed into the LU factorization when, in this procedure, only a 1×1 trailing submatrix $A_{n,n}$ remains. In MATLAB, the computation can be implemented with a simple for loop. The following code demonstrates this for our Example 3.1.6 above on page 20:

```
1 A = [ 6   7  14;
2       2   1   3;
3       4   4   7];
4
5 n = size(A, 2);
6
7 for j = 1 : n
8     A(j+1:n, j) = A(j+1:n, j)/ A(j, j);
9     A(j+1:n, j+1:n) = A(j+1:n, j+1:n) - A(j+1:n, j)*A(j, j+1:n);
10 end
11
12 disp('A ='); disp(A);
```

d) This algorithm is used in the MATLAB function `lu_vec`. At the end, the matrices L and U are extracted from the modified matrix A:

```
1 function [L, U] = lu_vec(A)
2     n = size(A, 2);
3
4     for j = 1 : n
5         A(j+1:n, j) = A(j+1:n, j) / A(j, j);
6         A(j+1:n, j+1:n) = A(j+1:n, j+1:n) ...
7                                 - A(j+1:n, j) * A(j, j+1:n);
8     end
9     L = tril(A, -1) + eye(n);
10    U = triu(A);
11 end
```

In line 8 the function `tril` (triangular lower) is used to create a new matrix of the same dimension as A that contains all elements of A below the diagonal. An identity matrix of the same dimension is added to this matrix to obtain L.

In line 9 the upper triangular matrix U is formed from A by means of the function `triu` (triangular upper).

Motivation 3.1.9 (Componentwise computation of the LU factorization)

In the j-th step, operations of the form

$$x \leftarrow x/\alpha$$
$$A \leftarrow A - x \cdot y^T$$

with

$$\alpha = a_{j,j},\ x = \begin{pmatrix} a_{j+1,j} \\ \vdots \\ a_{n,j} \end{pmatrix},\ y^T = \begin{pmatrix} a_{j,j+1} & \dots & a_{j,n} \end{pmatrix}$$

and

$$A = \begin{pmatrix} a_{j+1,j+1} & \dots & a_{j+1,n} \\ \vdots & & \vdots \\ a_{n,j+1} & \dots & a_{n,n} \end{pmatrix}$$

are performed. Componentwise, these operations can be described as

Algorithm 1: lu (row-wise)

Input: $A \in \mathbb{R}^{n \times n}$
Output: L, U with $A = LU$

```
1  for j = 1 to n do
2      for i = j + 1 to n do
3          A(i, j) ← A(i, j)/A(j, j);
4          for k = j + 1 to n do
5              A(i, k) ← A(i, k) − A(i, j) · A(j, k);
6  L ← tril(A, −1) + I_n;      % create L with tril ("triangular lower") and the identity
     matrix
7  U ← triu(A);                % create U with triu ("triangular upper")
8  return L, U
```

```
function [L, U] = lu_rowwise(A)
    n = size(A, 2);
    for j = 1 : n
        for i = j+1 : n
            A(i, j) = A(i, j) / A(j, j);
            for k = j+1 : n
                A(i, k) = A(i, k) − A(i, j) * A(j, k);
            end
        end
    end
    L = tril(A, −1) + eye(n);
    U = triu(A);
end
```

$$a_{i,j} \leftarrow a_{i,j}/\alpha \qquad \text{for } j+1 \le i \le n, \tag{3.2}$$

$$a_{i,k} \leftarrow a_{i,k} - a_{i,j} \cdot a_{j,k} \qquad \text{for } j+1 \le i \le n,\ j+1 \le k \le n. \tag{3.3}$$

Note that $a_{i,j}$ can be used in (3.3) only after it has been updated by the operation in (3.2).

In Algorithm 1 these operations are carried out by means of two loops in such a way that the matrix is updated row by row. This corresponds exactly to the procedure in Example 3.1.6 on page 20, as it was also carried out in the computation by hand.

In contrast, in Algorithm 2 the matrix is traversed column by column. The arithmetic operations are then performed in the same order as in the function `lu_vec`, which is described in Example 3.1.8 d) on page 23.

Remark 3.1.10 (The necessity of pivoting)

The matrix $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ has no LU factorization. This means that, even in simple cases, one cannot always solve $Ax = b$ by means of an LU factorization. After all the proofs and algorithms, this result is somewhat sobering. However, the proof of the next theorem offers a way out.

Algorithm 2: lu (column-wise)

Input: $A \in \mathbb{R}^{n \times n}$
Output: L, U with $A = LU$

1 **for** $j = 1$ ***to*** n **do**
2 **for** $i = j + 1$ ***to*** n **do**
3 $A(i, j) \leftarrow A(i, j)/A(j, j)$;
4 **for** $k = j + 1$ ***to*** n **do**
5 **for** $i = j + 1$ ***to*** n **do**
6 $A(i, k) \leftarrow A(i, k) - A(i, j) \cdot A(j, k)$;
7 $L \leftarrow \text{tril}(A, -1) + I_n$;
8 $U \leftarrow \text{triu}(A)$;
9 **return** L, U

```
function [L, U] = lu_colwise(A)
    n = size(A, 2);
    for j = 1 : n
        for i = j+1 : n
            A(i, j) = A(i, j) / A(j, j);
        end
        for k = j+1 : n
            for i = j+1 : n
                A(i, k) = A(i, k) - A(i, j) * A(j, k);
            end
        end
    end
    L = tril(A, -1) + eye(n);
    U = triu(A);
end
```

Theorem 3.1.11 (LU factorization with permutation matrices)

Let A be a nonsingular $n \times n$ matrix. Then there exist permutation matrices P_1 and P_2, a unit lower triangular matrix L, and a nonsingular upper triangular matrix U such that

$$P_1 A P_2 = LU$$

holds. Moreover, only one of P_1 and P_2 is needed, i.e., one can choose either $P_1 = I$ or $P_2 = I$.

Proof: We prove the theorem by induction on n.

Base case: $n = 1$
By assumption, $A = (a_{11})$ with $a_{11} \neq 0$. With

$$L = (1),\ U = A = (a_{11}), P_1 = P_2 = I = (1)$$

the claim holds.

Induction step: $n-1 \to n$
A is nonsingular, that is:

$$\Rightarrow \text{Every row and every column of } A \text{ has an entry } \neq 0$$

$$\Rightarrow \exists P_1', P_2' : \; P_1' A P_2' = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \text{ with } a_{11} \neq 0$$

(here one can choose either $P_1' = I$ or $P_2' = I$)

Ansatz in order to exploit the induction hypothesis:

$$P_1' A P_2' = \begin{pmatrix} a_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} \overset{!}{=} \begin{pmatrix} 1 & \\ \ell_{21} & I \end{pmatrix} \begin{pmatrix} u_{11} & u_{12}^T \\ & \tilde{A}_{22} \end{pmatrix}$$

$$= \begin{pmatrix} u_{11} & u_{12}^T \\ \ell_{21} u_{11} & \ell_{21} u_{12}^T + \tilde{A}_{22} \end{pmatrix}$$

For this we need:

$$a_{11} = u_{11},\; a_{21} = \ell_{21} \cdot u_{11},\; a_{12}^T = u_{12}^T,\; A_{22} = \ell_{21} \cdot u_{12}^T + \tilde{A}_{22}$$

Because of $a_{11} \neq 0$ this is obviously possible with:

$$\begin{aligned} u_{11} &= a_{11} \\ \ell_{21} &= \frac{1}{a_{11}} \cdot a_{21} \\ u_{12} &= a_{12} \\ \tilde{A}_{22} &= A_{22} - \ell_{21} \cdot u_{12}^T \end{aligned}$$

In order to apply the induction hypothesis to the $(n-1) \times (n-1)$ matrix $\tilde{A}_{22}$, we need

$$0 \neq \det(\tilde{A}_{22})$$

Since A is nonsingular, we have:

$$0 \neq \pm \det(A) = \det(P_1' A P_2') = \underbrace{\det \begin{pmatrix} 1 & \\ \ell_{21} & I \end{pmatrix}}_{=1} \cdot \underbrace{\det \begin{pmatrix} u_{11} & u_{12}^T \\ & \tilde{A}_{22} \end{pmatrix}}_{= u_{11} \cdot \det(\tilde{A}_{22})} \overset{u_{11} = a_{11} \neq 0}{\Longrightarrow} \det(\tilde{A}_{22}) \neq 0$$

Hence, by the induction hypothesis:

$$\exists \tilde{P}_1,\, \tilde{P}_2,\, \tilde{L},\, \tilde{U} : \quad \tilde{P}_1 \cdot \tilde{A}_{22} \cdot \tilde{P}_2 = \tilde{L} \cdot \tilde{U}$$

The rest now follows by substitution and rearrangement:

$$P_1' A P_2' \overset{\text{see above}}{=} \begin{pmatrix} 1 & \\ \ell_{21} & I \end{pmatrix} \begin{pmatrix} u_{11} & u_{12}^T \\ & \tilde{A}_{22} \end{pmatrix} \overset{\text{IH}}{=} \begin{pmatrix} 1 & \\ \ell_{21} & I \end{pmatrix} \begin{pmatrix} u_{11} & u_{12}^T \\ & \tilde{P}_1^T \tilde{L} \tilde{U} \tilde{P}_2^T \end{pmatrix}$$

$$= \begin{pmatrix} 1 & \\ \ell_{21} & \tilde{P}_1^T \tilde{L} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12}^T \\ & \tilde{U} \tilde{P}_2^T \end{pmatrix}$$

$$= \begin{pmatrix} 1 & \\ & \tilde{P}_1^T \end{pmatrix} \underbrace{\begin{pmatrix} 1 & \\ \tilde{P}_1 \ell_{21} & \tilde{L} \end{pmatrix}}_{=L} \underbrace{\begin{pmatrix} u_{11} & u_{12}^T \tilde{P}_2 \\ & \tilde{U} \end{pmatrix}}_{=U} \begin{pmatrix} 1 & \\ & \tilde{P}_2^T \end{pmatrix}$$

Thus L is a unit lower triangular matrix and U is a nonsingular upper triangular matrix (since $\det(U) = u_{11} \cdot \det(\tilde{U}) \neq 0$). One last rearrangement yields:

$$\underbrace{\left[\begin{pmatrix} 1 & \\ & \tilde{P}_1 \end{pmatrix} \cdot P_1'\right]}_{=:P_1} \cdot A \cdot \underbrace{\left[P_2' \cdot \begin{pmatrix} 1 & \\ & \tilde{P}_2 \end{pmatrix}\right]}_{=:P_2} = L \cdot U$$

□

If one specifies how P_1' and P_2' are chosen in each step, the proof yields an algorithm for computing the factorization.

Corollary 3.1.12 (Partial pivoting (column pivoting))

One can choose $P_2' = I$ and P_1' such that a_{11} is the element of largest absolute value in the first column. Then for

$$\ell_{21} = \tfrac{1}{a_{11}} \cdot a_{21},$$

every entry has absolute value less than or equal to 1.

Example 3.1.13 (Gaussian elimination with partial pivoting)

The computational scheme used in Example 3.1.6 can be extended to partial pivoting by a simple modification.

(a) When computing by hand, even in the case $n = 3$ one would like to avoid writing down the permutation matrices $\widetilde{P}_1$ and P_1' separately in order to then compute

$$P = \begin{pmatrix} 1 & \\ & \widetilde{P}_1 \end{pmatrix} \cdot P_1'$$

A more practical method is to keep track of a permutation matrix in addition to A. Initially, this is $P^{(0)} = I_n$, the identity matrix. Whenever rows are swapped during the transformations, one also swaps the corresponding rows of the permutation matrix. In this way one obtains $P^{(1)}, P^{(2)}, \ldots$ one after another, where the last matrix is the desired permutation matrix P.

$$A = A^{(0)} = \begin{pmatrix} 4 & 4 & 7 \\ 6 & 7 & 14 \\ 2 & 1 & 3 \end{pmatrix} \quad \text{and} \quad P^{(0)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(1)} = \begin{pmatrix} 6 & 7 & 14 \\ 4 & 4 & 7 \\ 2 & 1 & 3 \end{pmatrix} \begin{matrix} \cdot\frac{2}{3} \\ - \\ \\ \end{matrix} \begin{matrix} \\ \\ \cdot\frac{1}{3} \\ - \end{matrix} \quad \text{and} \quad P^{(1)} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(2)} = \begin{pmatrix} 6 & 7 & 14 \\ \frac{2}{3} & -\frac{2}{3} & -\frac{7}{3} \\ \frac{1}{3} & -\frac{4}{3} & -\frac{5}{3} \end{pmatrix} \quad \text{and} \quad P^{(1)} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(3)} = \begin{pmatrix} 6 & 7 & 14 \\ \frac{1}{3} & -\frac{4}{3} & -\frac{5}{3} \\ \frac{2}{3} & -\frac{2}{3} & -\frac{7}{3} \end{pmatrix} \cdot\tfrac{1}{2} \quad \text{and} \quad P^{(2)} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(4)} = \begin{pmatrix} 6 & 7 & 14 \\ \frac{1}{3} & -\frac{4}{3} & -\frac{5}{3} \\ \frac{2}{3} & \frac{1}{2} & -\frac{3}{2} \end{pmatrix} \quad \text{and} \quad P^{(2)} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

This yields

$$L = \begin{pmatrix} 1 & & \\ \frac{1}{3} & 1 & \\ \frac{2}{3} & \frac{1}{2} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 6 & 7 & 14 \\ & -\frac{4}{3} & -\frac{5}{3} \\ & & -\frac{3}{2} \end{pmatrix}, \quad P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Note that in the transition from $A^{(2)}$ to $A^{(3)}$ the entire rows of $A^{(2)}$ were swapped. Here, the following part of the proof of Theorem 3.1.11 was applied:

> If $\widetilde{P} \cdot \widetilde{A}_{22} = \widetilde{L} \cdot \widetilde{U}$, then
> $$L = \begin{pmatrix} 1 & \\ \widetilde{P} \cdot \ell_{21} & \widetilde{L} \end{pmatrix}$$

Looking at $A^{(2)}$, we have

$$\widetilde{A}_{22} = \begin{pmatrix} -\frac{2}{3} & -\frac{7}{3} \\ -\frac{4}{3} & -\frac{5}{3} \end{pmatrix} \quad \text{and} \quad \ell_{21} = \begin{pmatrix} \frac{2}{3} \\ \frac{1}{3} \end{pmatrix}.$$

The matrices $\widetilde{L}$ and $\widetilde{U}$ are determined only in the next steps, but $\widetilde{P}$ is already fixed:

$$\widetilde{P} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

(b) In the case $n = 4$ one can see more clearly that with this method the complete rows must always be swapped.

$$A = A^{(0)} \begin{pmatrix} 2 & 7 & 17 & 17 \\ -3 & -8 & -11 & 1 \\ 3 & 12 & 9 & 9 \\ 6 & 12 & 6 & 6 \end{pmatrix} \quad \text{and} \quad P^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(1)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ -3 & -8 & -11 & 1 \\ 3 & 12 & 9 & 9 \\ 2 & 7 & 17 & 17 \end{pmatrix} \begin{matrix} \cdot -\frac{1}{2} \\ - \end{matrix} \quad \begin{matrix} \cdot\frac{1}{2} \\ - \end{matrix} \quad \begin{matrix} \cdot\frac{1}{3} \\ - \end{matrix}$$

$$\rightsquigarrow \quad A^{(2)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ -\frac{1}{2} & -2 & -8 & 4 \\ \frac{1}{2} & 6 & 6 & 6 \\ \frac{1}{3} & 3 & 15 & 15 \end{pmatrix} \quad \text{and} \quad P^{(1)} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(3)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ \frac{1}{2} & 6 & 6 & 6 \\ -\frac{1}{2} & -2 & -8 & 4 \\ \frac{1}{3} & 3 & 15 & 15 \end{pmatrix} \begin{matrix} -\frac{1}{3} \\ - \end{matrix} \quad \begin{matrix} \frac{1}{2} \\ - \end{matrix}$$

$$\rightsquigarrow \quad A^{(4)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ \frac{1}{2} & 6 & 6 & 6 \\ -\frac{1}{2} & -\frac{1}{3} & -6 & 6 \\ \frac{1}{3} & -\frac{1}{2} & 12 & 12 \end{pmatrix} \quad \text{and} \quad P^{(2)} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

$$\rightsquigarrow \quad A^{(5)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ \frac{1}{2} & 6 & 6 & 6 \\ \frac{1}{3} & -\frac{1}{2} & 12 & 12 \\ -\frac{1}{2} & -\frac{1}{3} & -6 & 6 \end{pmatrix} \begin{matrix} -\frac{1}{2} \\ - \end{matrix}$$

$$\rightsquigarrow \quad A^{(6)} = \begin{pmatrix} 6 & 12 & 6 & 6 \\ \frac{1}{2} & 6 & 6 & 6 \\ \frac{1}{3} & -\frac{1}{2} & 12 & 12 \\ -\frac{1}{2} & -\frac{1}{3} & -\frac{1}{2} & 12 \end{pmatrix} \quad \text{and} \quad P^{(3)} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

This yields

$$L = \begin{pmatrix} 1 & & & \\ \frac{1}{2} & 1 & & \\ \frac{1}{3} & \frac{1}{2} & 1 & \\ -\frac{1}{2} & -\frac{1}{3} & -\frac{1}{2} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 6 & 12 & 6 & 6 \\ & 6 & 6 & 6 \\ & & 12 & 12 \\ & & & 12 \end{pmatrix}, \quad P = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Looking at $A^{(2)}$, we have

$$\widetilde{A}_{22} = \begin{pmatrix} -2 & -8 & 4 \\ 6 & 6 & 6 \\ 3 & 15 & 15 \end{pmatrix} \quad \text{and} \quad \ell_{21} = \begin{pmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{3} \end{pmatrix}.$$

This ℓ_{21} should now be replaced by $\widetilde{P} \cdot \ell_{21}$, where $\widetilde{P}$ is the permutation matrix used for the LU factorization of the trailing submatrix, i.e., $\widetilde{P} \cdot \widetilde{A}_{22} = \widetilde{L} \cdot \widetilde{U}$.

Here, $\widetilde{P} \cdot \widetilde{A}_{22}$ can be characterized by pairwise row swaps as follows:

- First, rows 1 and 2 of $\widetilde{A}_{22}$ are swapped.
- Then rows 2 and 3 of $\widetilde{A}_{22}$ are swapped.

In the course of the algorithm, ℓ_{21} is replaced step by step by $\widetilde{P} \cdot \ell_{21}$ by performing the corresponding row swaps:

- In the transition $A^{(2)} \rightsquigarrow A^{(3)}$, ℓ_{21} is replaced by

$$\begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ \frac{1}{3} \end{pmatrix}$$

- In the transition $A^{(4)} \rightsquigarrow A^{(5)}$, ℓ_{21} is replaced by

$$\begin{pmatrix} \frac{1}{2} \\ \frac{1}{3} \\ -\frac{1}{2} \end{pmatrix}$$

Remark 3.1.14 (Simple implementation)

1. Numerical stability will be investigated only later. The idea, however, is that this strategy reduces cancellation effects.
2. This strategy does not determine the choice of P_1' uniquely, since there may be several entries of equal absolute value in the first column.
3. In `MATLAB`, `[m, idx] = max(abs(A(:, 1)))` returns in `m` the largest absolute value in the first column of `A`. In addition, `idx` contains the index at which this value was first found.

4. In the j-th step, the element of largest absolute value is searched for in `A(j:n, j)`, i.e., in the j-th column on or below the diagonal.

   With `[m, idx] = max(abs(A(j:n, j)))`, `idx` does not contain a value from j to n but a value from 1 to $n-j+1$. If the value of largest absolute value is found on the diagonal, then `idx` has the value 1 and not j. The actual row index is therefore obtained as `idx + j - 1`.

5. For example, `A([1 2], :) = A([2 1], :)` swaps the first and second rows of A.

6. The following code implements the LU factorization with partial pivoting. Here, `p` instead of `idx` is used for the row index of the entry of largest absolute value (the so-called pivot element):

```
function [L, U, P] = lu_pp(A)
    n = size(A, 2);
    P = eye(n);
    for j = 1 : n
        % partial pivoting
        [~, p] = max(abs(A(j:n, j)));
        p = p + j - 1;
        if p ~= j
            A([j, p], :) = A([p, j], :);
            P([j, p], :) = P([p, j], :);
        end

        % compute j-th column in L and update submatrix of A
        A(j+1:n, j) = A(j+1:n, j) / A(j, j);
        A(j+1:n, j+1:n) = A(j+1:n, j+1:n) ...
                          - A(j+1:n, j) * A(j, j+1:n);
    end
    L = tril(A, -1) + eye(n);
    U = triu(A);
end
```

Definition 3.1.15 (Transposition matrix)

A transposition matrix $T_{i,j}$ with $i, j \in \{1, \ldots, n\}$ is obtained from the identity matrix by swapping the i-th and j-th rows.

Remark 3.1.16 (Properties of a transposition matrix)

(a) In the case $i < j$, $T_{i,j}$ has the form

$$T_{i,j} = \begin{pmatrix}
 & & & i & & & & j & & & \\
 & & & \downarrow & & & & \downarrow & & & \\
1 & & & & & & & & & & \\
 & \ddots & & & & & & & & & \\
 & & 1 & & & & & & & & \\
 & & & 0 & 0 & \cdots & 0 & 1 & & & \leftarrow i \\
 & & & 0 & 1 & & & 0 & & & \\
 & & & \vdots & & \ddots & & \vdots & & & \\
 & & & 0 & & & 1 & 0 & & & \\
 & & & 1 & 0 & \cdots & 0 & 0 & & & \leftarrow j \\
 & & & & & & & & 1 & & \\
 & & & & & & & & & \ddots & \\
 & & & & & & & & & & 1
\end{pmatrix}$$

(b) The entries of the transposition matrix $T_{i,j}$ can be described as follows:

$$(T_{i,j})_{k,\ell} = \begin{cases} 1, & \text{if } (k = i \text{ and } \ell = j) \text{ or } (k = j \text{ and } \ell = i), \\ 1, & \text{if } k = \ell \text{ and } k \notin \{i, j\}, \\ 0, & \text{otherwise.} \end{cases}$$

All diagonal entries are one, except at the positions (i, i) and (j, j). There are also ones at the positions (i, j) and (j, i). All other entries are zero.

(c) Obviously, $T_{i,j}$ is symmetric and is also obtained from the identity matrix by swapping the i-th and j-th columns. Moreover, $T_{i,j} = T_{j,i}$.

(d) The matrix $T_{i,j} \cdot A$ is obtained from A by swapping the i-th and j-th rows. One can first convince oneself of this with a simple example with $n = 4$, in which one considers $T_{i,j}$ column by column and A row by row:

$$T_{4,1} \cdot A = \begin{pmatrix} e_4 & e_2 & e_3 & e_1 \end{pmatrix} \cdot \begin{pmatrix} a_1^T \\ a_2^T \\ a_3^T \\ a_4^T \end{pmatrix} = \underbrace{e_4 \cdot a_1^T + e_2 \cdot a_2^T + e_3 \cdot a_3^T + e_1 \cdot a_4^T}_{\text{sort the summands with respect to } e_1, \ldots, e_4}$$

$$= e_1 \cdot a_4^T + e_2 \cdot a_2^T + e_3 \cdot a_3^T + e_4 \cdot a_1^T = \begin{pmatrix} e_1 & e_2 & e_3 & e_4 \end{pmatrix} \cdot \begin{pmatrix} a_4^T \\ a_2^T \\ a_3^T \\ a_1^T \end{pmatrix}$$

$$= \begin{pmatrix} a_4^T \\ a_2^T \\ a_3^T \\ a_1^T \end{pmatrix}$$

In order to describe the general case formally, we next summarize some basics about permutations and transpositions.

(e) *Permutations and transpositions*

A permutation $\sigma \in S_n$ is a bijective map from $\{1, \ldots, n\}$ to $\{1, \ldots, n\}$. Every such map can be represented uniquely by a table of values:

ℓ	1	2	...	n
$\sigma(\ell)$	$\sigma(1)$	$\sigma(2)$	...	$\sigma(n)$

For example, for $n = 4$ the following table describes a permutation $\sigma \in S_4$:

ℓ	1	2	3	4
$\sigma(\ell)$	4	3	1	2

Transpositions are special cases of permutations in which only two elements are swapped. The transposition $\tau_{i,j} \in S_n$ is defined by

$$\tau_{i,j}(\ell) = \begin{cases} i, & \ell = j, \\ j, & \ell = i, \\ \ell, & \text{otherwise} \end{cases}$$

and swaps the i-th element with the j-th element. The mapping rule makes it possible to describe transpositions in general, which is often advantageous in proofs. A concrete transposition such as $\tau_{1,4}$ for $n = 4$ can of course also be described by the table of values

ℓ	1	2	3	4
$\tau_{1,4}(\ell)$	4	2	3	1

which is equivalent but often more intuitive.

For transpositions, obviously $\tau_{i,j} = \tau_{j,i}$ and, moreover, $\tau_{i,j} \circ \tau_{i,j} = \mathrm{id}$. It follows that every transposition is its own inverse, i.e., $\tau_{i,j}^{-1} = \tau_{i,j}$.

(f) The example in (d) can be rewritten using the notation for transpositions, which also makes the connection between a transposition and a transposition matrix clear:

$$T_{4,1} \cdot A = \begin{pmatrix} e_{\tau_{4,1}(1)} & e_{\tau_{4,1}(2)} & e_{\tau_{4,1}(3)} & e_{\tau_{4,1}(4)} \end{pmatrix} \cdot \begin{pmatrix} a_1^T \\ a_2^T \\ a_3^T \\ a_4^T \end{pmatrix}$$

$$= \sum_{\ell=1}^{4} e_{\tau_{4,1}(\ell)} \cdot a_\ell^T \overset{\substack{\tau_{4,1}(\ell)=\widetilde{\ell} \\ \ell=\tau_{4,1}^{-1}(\widetilde{\ell})=\tau_{4,1}(\widetilde{\ell})}}{=\!=\!=} \sum_{\widetilde{\ell}=1}^{4} e_{\widetilde{\ell}} \cdot a_{\tau_{4,1}(\widetilde{\ell})}^T$$

$$= \begin{pmatrix} e_1 & e_2 & e_3 & e_4 \end{pmatrix} \begin{pmatrix} a_{\tau_{4,1}(1)}^T \\ a_{\tau_{4,1}(2)}^T \\ a_{\tau_{4,1}(3)}^T \\ a_{\tau_{4,1}(4)}^T \end{pmatrix} = \begin{pmatrix} a_4^T \\ a_2^T \\ a_3^T \\ a_1^T \end{pmatrix}$$

(g) In general,

$$T_{i,j} \cdot A = \begin{pmatrix} e_{\tau_{i,j}(1)} & \dots & e_{\tau_{i,j}(n)} \end{pmatrix} \cdot \begin{pmatrix} a_1^T \\ \vdots \\ a_n^T \end{pmatrix} = \sum_{\ell=1}^{n} e_{\tau_{i,j}(\ell)} \cdot a_\ell^T = \sum_{\ell=1}^{n} e_\ell \cdot a_{\tau_{i,j}(\ell)}^T$$

$$= \begin{pmatrix} e_1 & \dots & e_n \end{pmatrix} \begin{pmatrix} a_{\tau_{i,j}(1)}^T \\ \vdots \\ a_{\tau_{i,j}(n)}^T \end{pmatrix} = \begin{pmatrix} a_{\tau_{i,j}(1)}^T \\ \vdots \\ a_{\tau_{i,j}(n)}^T \end{pmatrix}$$

(h) *Composition of transpositions*

For $n = 4$ we consider the following examples of transpositions

ℓ	1	2	3	4
$\tau_{1,4}(\ell)$	4	2	3	1

"swap 1 and 4"

as well as

ℓ	1	2	3	4
$\tau_{2,3}(\ell)$	1	3	2	4

"swap 2 and 3"

and

ℓ	1	2	3	4
$\tau_{3,4}(\ell)$	1	2	4	3

"swap 3 and 4"

In Example 3.1.13 (b), these transpositions correspond to the row swaps that are performed one after another in this order. Formally, this sequence of transpositions can be described as the composition

$$\sigma = \tau_{1,4} \circ \tau_{2,3} \circ \tau_{3,4}$$

A table of values for this composition can be created by successively swapping the numbers from 1 to 4. Swapping the entries in positions i and j of the value row of a map f yields $f \circ \tau_{i,j}$, so each further swap is composed from the right:

ℓ	1	2	3	4
$\tau_{1,4}(\ell)$	4	2	3	1
$\tau_{1,4} \circ \tau_{2,3}(\ell)$	4	3	2	1
$\sigma = \tau_{1,4} \circ \tau_{2,3} \circ \tau_{3,4}(\ell)$	4	3	1	2

This composition produces the permutation $\sigma \in S_4$ given by

ℓ	1	2	3	4
$\sigma(\ell)$	4	3	1	2

(i) *Multiplication of transposition matrices*

Multiplying transposition matrices yields a permutation matrix. The following worked example shows not only how the multiplication can be carried out by row swaps of an identity matrix, but also the connection to the composition of transpositions:

$$P := T_{3,4} \cdot T_{2,3} \cdot T_{1,4} = T_{3,4} \cdot \Big(T_{2,3} \cdot \big(T_{1,4} \cdot I \big) \Big)$$

$$= T_{3,4} \cdot \left(T_{2,3} \cdot \left(T_{1,4} \cdot \begin{pmatrix} e_1^T \\ e_2^T \\ e_3^T \\ e_4^T \end{pmatrix} \right) \right) = T_{3,4} \cdot \left(T_{2,3} \cdot \begin{pmatrix} e_4^T \\ e_2^T \\ e_3^T \\ e_1^T \end{pmatrix} \right) = T_{3,4} \cdot \begin{pmatrix} e_4^T \\ e_3^T \\ e_2^T \\ e_1^T \end{pmatrix}$$

$$= \begin{pmatrix} e_4^T \\ e_3^T \\ e_1^T \\ e_2^T \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \overset{\sigma \text{ from (h)}}{=\!=\!=} \begin{pmatrix} e_{\sigma(1)}^T \\ e_{\sigma(2)}^T \\ e_{\sigma(3)}^T \\ e_{\sigma(4)}^T \end{pmatrix}$$

Implementation 3.1.17 (Gaussian elimination with partial pivoting)

In Example 3.1.13 we showed how Gaussian elimination with partial pivoting can be carried out by hand (with pencil and paper, so to speak). In Remark 3.1.14 the method was implemented in MATLAB as the function `lu_pp`. However, no attention was paid to a memory-efficient representation of the permutation matrix. In addition, at the end of the function the matrices L and U were extracted from the overwritten matrix A, which requires additional memory.

Unfortunately, in MATLAB it is fundamentally impossible to implement the LU factorization memory-efficiently (or efficiently at all). In several places, efficiency is deliberately sacrificed for ease of use. This makes MATLAB excellent for developing new algorithms and testing them for correctness, but less suitable for assessing efficiency. For example, when a matrix is passed as an argument to a MATLAB function, a copy of the matrix is created internally (which costs time and memory), and this copy is then passed to the function. The following MATLAB implementation can therefore only indicate how a memory-efficient implementation would be possible in principle. In a later section, the underlying algorithm is implemented in the programming language Julia in order to show that this memory efficiency can also be demonstrated with benchmarks.

```
1  function [A, piv] = lu_pp_inplace(A)
2      n = size(A, 2);
3      piv = zeros(n, 1);
4      for j = 1 : n
5          % partial pivoting
6          [~, p] = max(abs(A(j:n, j)));
7          p = p + j - 1;
8          piv(j) = p;
9          if p ~= j
10             A([j, p], :) = A([p, j], :);
11         end
12
13         % compute j-th column in L and update submatrix of A
14         A(j+1:n, j) = A(j+1:n, j) / A(j, j);
15         A(j+1:n, j+1:n) = A(j+1:n, j+1:n) ...
16                           - A(j+1:n, j) * A(j, j+1:n);
17     end
18 end
```

The essential changes are the following:

- Line 1: The function returns the overwritten $n \times n$ matrix A and a so-called pivot vector piv. In line 3, piv is created as a column vector of dimension n.
- In lines 6 and 7, as before, the index p of the element of largest absolute value, the pivot element, in the first column of the trailing submatrix is determined.
- The essential change now comes in line 8. Instead of swapping the rows of a permutation matrix P, we merely record in the vector piv that in the j-th step the j-th row is swapped with the p-th row. The content of piv thus represents the transposition matrices that were applied:
  - In the 1st step, row 1 was swapped with row piv_1, i.e., the remaining matrix was multiplied from the left by T_{1,piv_1}.
  - In the 2nd step, row 2 was swapped with row piv_2, i.e., the remaining matrix was multiplied from the left by T_{2,piv_2}.
  - …
  - In the n-th step, row n was swapped with row piv_n, i.e., the remaining matrix was multiplied from the left by T_{n,piv_n}.

  For the entries of piv, obviously

$$\text{piv}_j \geq j \quad \forall j \in \{1, \ldots, n\}$$

and hence always $\text{piv}_n = n$. The last entry thus always represents $T_{n,n} = I$ and is actually unnecessary. Saving this entry, however, has no effect on efficiency; it would merely make the code and algorithms that work with pivot vectors less readable.

Implementation 3.1.18 (Applying a pivot vector as a permutation matrix)

To multiply a matrix X from the left by the permutation matrix P represented by a pivot vector piv, one can proceed as in Remark 3.1.16 (h). So no matrices are actually multiplied; rows are merely swapped successively. In this way, the following MATLAB function computes $P \cdot X$ and returns the permuted matrix:

```
1  function X = apply_piv(X, piv)
2      [m, n] = size(X);
3      for i = 1 : m
4          if piv(i) ~= i
5              % X([i, piv(i)], :) = X([piv(i), i], :);
6              for j = 1 : n
7                  tmp          = X(i, j);
8                  X(i, j)      = X(piv(i), j);
9                  X(piv(i), j) = tmp;
10             end
11         end
12     end
13 end
```

The MATLAB code in line 5, which is commented out, swaps rows i and piv_i. This commented-out code was replaced by lines 6 to 10, in which the rows are swapped by means of a loop and elementwise access. In this form it is easier to see that the function does not require any additional memory. It also becomes clear that at most $\mathcal{O}(n \cdot m)$ swaps of individual elements are performed.

If X is an $n \times n$ matrix, the cost of computing $P \cdot X$ is therefore $\mathcal{O}(n^2)$ and not $\mathcal{O}(n^3)$ as for a "normal" matrix multiplication.

Remark 3.1.19 (Cost of the LU factorization)

1. In benchmarks, the cost of row swaps is not taken into account. In fact, the algorithm for the LU factorization can always be formulated such that no swaps have to be performed. This holds for the LU factorization with partial pivoting as well as for any other pivoting strategy. That swaps are nevertheless performed in practice has a technical reason (keyword: CPU caches). The swaps make memory accesses in the subsequent steps considerably more efficient. Overall, this significantly speeds up the algorithm.

2. In the past, when determining the computational cost, one distinguished between addition, subtraction, multiplication and division, since these operations had different costs. On most of today's architectures, however, the cost of these arithmetic operations is almost the same, which is why one usually no longer distinguishes between them. It is conceivable, however, that this might change again with future developments in hardware architecture.

```
1 for j = 1 : n
2     A(j+1:n, j) = A(j+1:n, j) / A(j, j);
3     A(j+1:n, j+1:n) = A(j+1:n, j+1:n) ...
4                       - A(j+1:n, j) * A(j, j+1:n);
```

```
5 end
```

To see the relevant arithmetic operations more clearly, one can rewrite this such that the components are accessed elementwise. For the computational cost it does not matter whether the access here is row-wise or column-wise:

```
1 for j = 1 : n
2     for i = j+1 : n
3         % One operation:
4         A(i, j) = A(i, j) / A(j, j);
5     end
6     for k =  j+1 : n
7         for i = j+1 : n
8             % Two operations:
9             A(i, k) = A(i, k) - A(i, j) * A(j, k);
10        end
11    end
12 end
```

The computational cost is thus

$$\begin{aligned}
\sum_{j=1}^{n}\sum_{i=j+1}^{n} 1 + 2\sum_{j=1}^{n}\sum_{k=j+1}^{n}\sum_{i=j+1}^{n} 1 &= \sum_{j=1}^{n}(n-j) + 2\sum_{j=1}^{n}(n-j)^2 \\
&= \sum_{j=0}^{n-1} j + 2\sum_{j=0}^{n-1} j^2 \\
&= \frac{n\cdot(n-1)}{2} + 2\cdot\frac{(n-1)\cdot n\cdot(2n-1)}{6} \\
&= \frac{2n^3}{3} - \frac{n^2}{2} - \frac{n}{6} \\
&= \frac{2n^3}{3} + \mathcal{O}(n^2)
\end{aligned}$$

§2 Real Symmetric Positive Definite Matrices

Definition 3.2.1 (Real positive definite matrix)

A real $n \times n$ matrix A is positive definite if

$$x^T \cdot A \cdot x > 0 \quad \forall x \in \mathbb{R}^n \backslash \{\mathbf{0}\}$$

holds.

Lemma 3.2.2

If A is a positive definite real $n \times n$ matrix, then all leading principal submatrices of A are nonsingular. By Lemma 3.1.4 it therefore follows that A has a unique LU factorization (without pivoting).

Proof:
Let $j \in \{1, \ldots, n\}$ be arbitrary but fixed and let A_{11} denote the leading $j \times j$ principal submatrix of $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$. If A_{11} is singular, there exists an $\mathbf{x}_1 \neq \mathbf{0}_j$ with $A_{11} \cdot \mathbf{x}_1 = \mathbf{0}_j$. For

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{0}_{n-j} \end{pmatrix} \in \mathbb{R}^n$$

it then follows that

$$\mathbf{x} \neq \mathbf{0}_n \quad \text{and} \quad \mathbf{x}^T \cdot A \cdot \mathbf{x} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{0}_{n-j} \end{pmatrix}^T \cdot \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{0}_{n-j} \end{pmatrix} = \mathbf{x}_1^T \cdot A_{11} \cdot \mathbf{x}_1 = 0$$

which would contradict the assumption. ∇

Lemma 3.2.3 *If A is a positive definite real $n \times n$ matrix, then there exists a unique factorization*

$$A = L \cdot D \cdot \widehat{L}^T,$$

where D is an invertible diagonal matrix and L and $\widehat{L}$ are both unit lower triangular matrices.

Proof: By Lemma 3.2.2, a positive definite matrix A has a unique factorization of the form

$$A = L \cdot U,$$

where U is an invertible upper triangular matrix. Hence the diagonal entries of U satisfy $u_{ii} \neq 0$ for $1 \leq i \leq n$. If we define $D := \operatorname{diag}(u_{11}, \ldots, u_{nn})$, then

$$D^{-1} = \operatorname{diag}\left(\frac{1}{u_{11}}, \ldots, \frac{1}{u_{nn}}\right)$$

exists, and therefore

$$\widehat{L}^T := D^{-1} \cdot U$$

is well defined. For the entries of $\widehat{L}^T$ we have

$$e_i^T \cdot \widehat{L}^T \cdot e_j = \left(e_i^T \cdot D^{-1}\right) \cdot (U \cdot e_j) = \frac{1}{u_{ii}} \cdot e_i^T \cdot U \cdot e_j = \begin{cases} \frac{u_{ij}}{u_{ii}}, & j > i, \\ 1, & i = j, \\ 0, & j < i, \end{cases}$$

so that $\widehat{L}^T$ is an upper triangular matrix and hence $\widehat{L}$ is a lower triangular matrix, each with ones on the diagonal. This factorization is also unique: for any factorization $A = L \cdot D \cdot \widehat{L}^T$ of the stated form, $U := D \cdot \widehat{L}^T$ is an upper triangular matrix with $\operatorname{diag}(u_{11}, \ldots, u_{nn}) = D$, so $A = L \cdot U$ is the unique LU factorization of A; hence L, D, and $\widehat{L}^T = D^{-1} \cdot U$ are uniquely determined. ∇

Lemma 3.2.4 (Cholesky factorization with diagonal matrix)

For a real $n \times n$ matrix A the following statements are equivalent:

(1) A is symmetric and positive definite.

(2) There exist a unique unit lower triangular matrix L and a diagonal matrix $D = \operatorname{diag}(d_1, \ldots, d_n)$ with $d_1, \ldots, d_n > 0$ such that

$$A = L \cdot D \cdot L^T$$

holds.

Proof:
(1) $\Rightarrow$ (2)
From Lemma 3.2.3 it follows that A has a unique factorization of the form

$$A = L \cdot D \cdot \widehat{L}^T, \tag{3.4}$$

where $D = \operatorname{diag}(d_1, \ldots, d_n)$ is a diagonal matrix and L and $\widehat{L}$ are unit lower triangular matrices.

Since A is symmetric, we have:

$$A = A^T = \left(L \cdot D \cdot \widehat{L}^T\right)^T = \widehat{L} \cdot D \cdot L^T. \tag{3.5}$$

From (3.4) and (3.5) it follows that

$$A = L \cdot D \cdot \widehat{L}^T = \widehat{L} \cdot D \cdot L^T,$$

and by the uniqueness of the factorization in (3.4) we must have $L = \widehat{L}$.

It remains to show that D has only positive entries on the diagonal. If $d_i \leq 0$, then choosing $x = L^{-T} \cdot e_i \neq \mathbf{0}$ would lead to

$$x^T \cdot A \cdot x = e_i^T \cdot L^{-1} \cdot L \cdot D \cdot L^T \cdot L^{-T} \cdot e_i = e_i^T \cdot D \cdot e_i = d_i \leq 0.$$

This, however, contradicts the positive definiteness of A.

(2) $\Rightarrow$ (1)
From $d_1 > 0, \ldots, d_n > 0$ it follows that

$$x^T \cdot A \cdot x = x^T \cdot L \cdot D \cdot L^T \cdot x \overset{y := L^T \cdot x}{=\!=\!=} y^T \cdot D \cdot y = d_1 \cdot y_1^2 + \cdots + d_n \cdot y_n^2 \geq 0 \quad \forall x \in \mathbb{R}^n.$$

Since L, and hence also L^T, is invertible, we have

$$y = L^T \cdot x = \mathbf{0} \;\Leftrightarrow\; x = L^{-T} \cdot y = \mathbf{0}.$$

It therefore follows that

$$x^T \cdot A \cdot x \overset{y := L^T \cdot x}{=\!=\!=} d_1 \cdot y_1^2 + \cdots + d_n \cdot y_n^2 > 0 \quad \forall x \neq \mathbf{0}.$$

Thus A is positive definite. The symmetry of A follows directly from

$$A = L \cdot D \cdot L^T = A^T.$$



Corollary 3.2.5 (Classical Cholesky factorization)

Let A be a symmetric positive definite $n \times n$ matrix. Let L be the unit lower triangular matrix and $D = \operatorname{diag}(d_1, \ldots, d_n)$ the diagonal matrix from Lemma 3.2.4 such that

$$A = L \cdot D \cdot L^T$$

holds. If one now defines

$$D^{1/2} := \operatorname{diag}\left(\sqrt{d_1}, \ldots, \sqrt{d_n}\right), \quad \text{and} \quad \widetilde{L} := L \cdot D^{1/2},$$

then

$$A = \widetilde{L} \cdot \widetilde{L}^T.$$

This factorization is called the classical Cholesky factorization of A.

Example 3.2.6 (Computation by hand)

From Lemma 3.2.4 it follows that the Cholesky factorization with diagonal matrix

$$A = L \cdot D \cdot L^T$$

can be computed by means of the LU factorization. Afterwards (if desired) the classical Cholesky factorization $A = \widetilde{L} \cdot \widetilde{L}^T$ can be determined by computing $\widetilde{L} := L \cdot D^{1/2}$.

Because of the equivalence statement in Lemma 3.2.4 it is not necessary to check beforehand whether a symmetric matrix is also positive definite. In fact, computing the Cholesky factorization is a common method for testing real symmetric matrices for positive definiteness.

(a) First one computes as in the LU factorization without pivoting:

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 8 & 12 \\ 3 & 12 & 27 \end{pmatrix} \begin{matrix} \cdot 2 \\ - \\ \end{matrix} \quad \begin{matrix} \cdot 3 \\ \\ - \end{matrix} \quad \rightsquigarrow \quad \begin{pmatrix} 1 & 2 & 3 \\ \boxed{2} & 4 & 6 \\ \boxed{3} & 6 & 18 \end{pmatrix} \begin{matrix} \\ \cdot \frac{3}{2} \\ - \end{matrix} \quad \rightsquigarrow \quad \begin{pmatrix} 1 & 2 & 3 \\ \boxed{2} & 4 & 6 \\ 3 & \boxed{\frac{3}{2}} & 9 \end{pmatrix}$$

This yields:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & \frac{3}{2} & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 9 \end{pmatrix}, \quad D^{1/2} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

From this we finally obtain:

$$\widetilde{L} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 2 & 0 \\ 3 & 3 & 3 \end{pmatrix}.$$

(b) One recognizes that a symmetric matrix A is not positive definite by the fact that entries arise on the diagonal that are not positive. In this case the procedure can be aborted:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 3 \end{pmatrix} \begin{matrix} \cdot 1 \\ - \end{matrix} \begin{matrix} \cdot 1 \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 2 \end{pmatrix} \begin{matrix} \cdot 2 \\ - \end{matrix} \rightsquigarrow \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 1 & 0 \end{pmatrix}$$

This yields

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \; D = \operatorname{diag}(d_1, d_2, d_3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

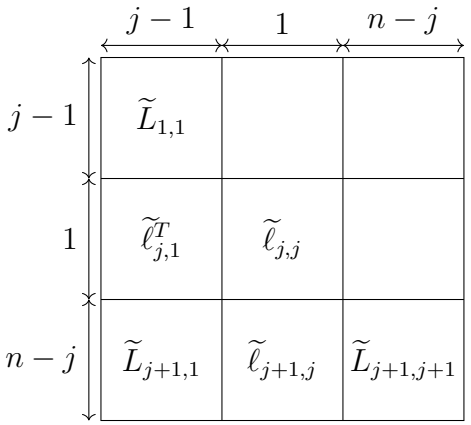
Because $d_3 \leq 0$, A is not positive definite.

Derivation 3.2.7 (Direct computation of the classical Cholesky factorization)

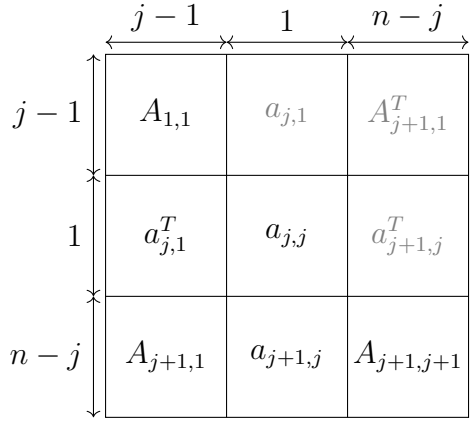
Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite, and let $\widetilde{L}$ be the lower triangular matrix of the classical Cholesky factorization with positive diagonal entries. The existence and uniqueness of this factorization are used in what follows.

The challenge is that the algorithm accesses only the lower triangular part of A and overwrites it in place. For a $j \in \{1, \ldots, n\}$ we consider the following block structures for $\widetilde{L}$ and A:

$$\widetilde{L} = \begin{pmatrix} \widetilde{L}_{1,1} & & \\ \widetilde{\ell}_{j,1}^T & \widetilde{\ell}_{j,j} & \\ \widetilde{L}_{j+1,1} & \widetilde{\ell}_{j+1,j} & \widetilde{L}_{j+1,j+1} \end{pmatrix}$$



$$A = \begin{pmatrix} A_{1,1} & a_{j,1} & A_{j+1,1}^T \\ a_{j,1}^T & a_{j,j} & a_{j+1,j}^T \\ A_{j+1,1} & a_{j+1,j} & A_{j+1,j+1} \end{pmatrix}$$



In the j-th step our algorithm computes the j-th column of $\widetilde{L}$, i.e., the entries $\widetilde{\ell}_{j,j}$ and $\widetilde{\ell}_{j+1,j}$. From the identity $A = \widetilde{L} \cdot \widetilde{L}^T$ and by fully multiplying out the block structures we obtain a total of nine equations.

Among them are the following two equations (see the side calculation further below), which suffice to compute the desired entries $\widetilde{\ell}_{j,j}$ and $\widetilde{\ell}_{j+1,j}$:

$$a_{j,j} = \widetilde{\ell}_{j,1}^T \cdot \widetilde{\ell}_{j,1} + \widetilde{\ell}_{j,j}^2, \quad a_{j+1,j} = \widetilde{L}_{j+1,1} \cdot \widetilde{\ell}_{j,1} + \widetilde{\ell}_{j+1,j} \cdot \widetilde{\ell}_{j,j}.$$

Rearranging gives:

$$\widetilde{\ell}_{j,j} = \sqrt{a_{j,j} - \widetilde{\ell}_{j,1}^T \cdot \widetilde{\ell}_{j,1}}, \quad \widetilde{\ell}_{j+1,j} = \frac{a_{j+1,j} - \widetilde{L}_{j+1,1} \cdot \widetilde{\ell}_{j,1}}{\widetilde{\ell}_{j,j}}.$$

Hence the j-th column of $\widetilde{L}$ can be computed once the first $j-1$ columns of $\widetilde{L}$ are known.

If A is overwritten in place, we obtain:

$$a_{j,j} \leftarrow \sqrt{a_{j,j} - a_{j,1}^T \cdot a_{j,1}}, \quad a_{j+1,j} \leftarrow \frac{a_{j+1,j} - A_{j+1,1} \cdot a_{j,1}}{a_{j,j}}.$$

In MATLAB notation these blocks of A can be accessed by

$$\begin{aligned} a_{j,j} &\triangleq \texttt{A(j,j)} \\ a_{j,1}^T &\triangleq \texttt{A(j, 1:j-1)} \\ A_{j+1,1} &\triangleq \texttt{A(j+1:n, 1:j-1)} \\ a_{j+1,j} &\triangleq \texttt{A(j+1:n, j)} \end{aligned}$$

With this, the algorithm reads (in MATLAB notation):

Algorithm 3: Classical Cholesky factorization

Input: $A \in \mathbb{R}^{n \times n}$
Output: tril(A) is overwritten with $\widetilde{L}$

1 **for** $j \leftarrow 1 : n$ **do**
2 $\quad A(j,j) \leftarrow \sqrt{A(j,j) - A(j,1:j{-}1) \cdot A(j,1:j{-}1)^T}$
3 $\quad A(j{+}1:n,j) \leftarrow \big(A(j{+}1:n,j) - A(j{+}1:n,1:j{-}1) \cdot A(j,1:j{-}1)^T\big)/A(j,j)$

The cost is $\frac{1}{3}n^3 + \mathcal{O}(n^2)$ operations, which is half the cost of an LU factorization.

Side calculation: In the computation of $\widetilde{L} \cdot \widetilde{L}^T$ the relevant blocks $\widetilde{\ell}_{j,j}$ and $\widetilde{\ell}_{j+1,j}$ are colored:

$$\widetilde{L} \cdot \widetilde{L}^T = \begin{pmatrix} \widetilde{L}_{1,1} & & \\ \widetilde{\ell}_{j,1}^T & \widetilde{\ell}_{j,j} & \\ \widetilde{L}_{j+1,1} & \widetilde{\ell}_{j+1,j} & \widetilde{L}_{j+1,j+1} \end{pmatrix} \cdot \begin{pmatrix} \widetilde{L}_{1,1}^T & \widetilde{\ell}_{j,1} & \widetilde{L}_{j+1,1}^T \\ & \widetilde{\ell}_{j,j} & \widetilde{\ell}_{j+1,j}^T \\ & & \widetilde{L}_{j+1,j+1}^T \end{pmatrix}$$

$$= \begin{pmatrix} * & * & * \\ * & \widetilde{\ell}_{j,1}^T \cdot \widetilde{\ell}_{j,1} + \widetilde{\ell}_{j,j} \cdot \widetilde{\ell}_{j,j} & * \\ * & \widetilde{L}_{j+1,1} \cdot \widetilde{\ell}_{j,1} + \widetilde{\ell}_{j+1,j} \cdot \widetilde{\ell}_{j,j} & * \end{pmatrix}$$

$$\stackrel{!}{=} \begin{pmatrix} * & * & * \\ * & a_{j,j} & * \\ * & a_{j+1,j} & * \end{pmatrix} = A$$

§3 Conditioning of Linear Systems

Definition 3.3.1 (Error and residual)

Let $x = A^{-1}b$ be the exact (but in general unknown) solution of the linear system and $\widetilde{x}$ the computed solution. Then

$$e := x - \widetilde{x} \qquad \text{is called the error, and}$$
$$r := A \cdot e = A \cdot (x - \widetilde{x}) = b - A \cdot \widetilde{x} \qquad \text{the residual.}$$

Remark 3.3.2 (Outlook: error estimators)

(a) The error e can in general not be computed, since x is unknown.

(b) The residual r, on the other hand, can be computed from $\widetilde{x}$ at a cost of $\mathcal{O}(n^2)$. Because of the relation

$$e = A^{-1} \cdot r$$

the error can (at least theoretically) be computed from the residual. If $\|\cdot\|$ is a vector norm, then for a compatible matrix norm we have

$$\|e\| = \|A^{-1} \cdot r\| \leq \|A^{-1}\| \cdot \|r\|. \tag{3.6}$$

(c) In general A^{-1}, and hence $\|A^{-1}\|$, is unknown, since otherwise the solution x would also be known. The cost of computing A^{-1} is $\mathcal{O}(n^3)$, i.e., of the same order as the cost of computing $\widetilde{x}$. An estimate of the form (3.6) is therefore, for the time being, mainly of theoretical use.

(d) For the matrix norms $\|\cdot\|_1$ and $\|\cdot\|_\infty$ there are numerical methods that can estimate $\|A^{-1}\|$ at a cost of $\mathcal{O}(n^2)$. This gives:

$$\|e\|_\infty = \|A^{-1} \cdot r\|_\infty \leq \|A^{-1}\|_\infty \cdot \|r\|_\infty \approx \text{estimate}_{\|A^{-1}\|_\infty}(A) \cdot \|r\|_\infty,$$

or

$$\|e\|_1 = \|A^{-1} \cdot r\|_1 \leq \|A^{-1}\|_1 \cdot \|r\|_1 \approx \text{estimate}_{\|A^{-1}\|_1}(A) \cdot \|r\|_1.$$

Motivation 3.3.3

In Chapter 2 the condition of a problem was motivated as the "sensitivity of a function evaluation with respect to perturbations of the input data" and defined for real functions of one variable. Here we now consider the special case of a linear system. The solution $x = A^{-1} \cdot b$ thus depends on the data A and b, whose perturbation can be modeled as follows:

- The matrix $\widetilde{A} = A + \delta A$ is the coefficient matrix A perturbed by δA, and
- the vector $\widetilde{b} = b + \delta b$ is the right-hand side b perturbed by δb.

With exact arithmetic, a numerical method then delivers the solution of

$$(A + \delta A) \cdot \widetilde{x} = (b + \delta b)\,.$$

In what follows, we will derive for compatible norms an estimate of the form

$$\frac{\|\delta x\|}{\|x\|} := \frac{\|e\|}{\|x\|} = \frac{\|\widetilde{x} - x\|}{\|x\|} \leq C \cdot \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right)$$

The quantities appearing here can be interpreted as follows:

- $\frac{\|\delta A\|}{\|A\|}$ and $\frac{\|\delta b\|}{\|b\|}$ are measures of the relative input error of A and b, respectively,
- $\frac{\|\delta x\|}{\|x\|} = \frac{\|e\|}{\|x\|}$ is a measure of the relative output error, and
- the constant C describes the sensitivity of the relative output error with respect to the relative input errors. Here C depends on the problem, i.e., on A and b. We will see, however, that C depends mainly on the quantity $\kappa(A) := \|A\| \cdot \|A^{-1}\|$, which we will define as the condition number of A.

Definition 3.3.4 (Condition number of a matrix)

The condition number of an invertible matrix A with respect to a matrix norm $\|\cdot\|$ is

$$\kappa_{\|\cdot\|}(A) := \|A\| \cdot \|A^{-1}\|$$

If the matrix norm is clear from the context, one also simply writes $\kappa(A)$.

Remark 3.3.5

(a) In Chapter 2 (Definition 2.2.1) the condition number of a function $f : \mathbb{R} \to \mathbb{R}$ was defined as

$$\kappa_f(x) = \frac{|f'(x)| \cdot |x|}{|f(x)|}.$$

For a function $f : \mathbb{R}^n \to \mathbb{R}^m$ the condition is defined analogously by

$$\kappa_f(x) := \frac{\|f'(x)\| \cdot \|x\|}{\|f(x)\|},$$

where a vector norm $\|\cdot\|$ and the matrix norm induced by it are used.

If A is a real $n \times n$ matrix, then the condition of the function

$$f : \mathbb{R}^n \to \mathbb{R}^n, \quad f(x) = A \cdot x,$$

is given by

$$\kappa_f(x) = \frac{\|A\| \cdot \|x\|}{\|A \cdot x\|} = \frac{\|A\|}{\frac{\|A \cdot x\|}{\|x\|}}.$$

It was shown in the exercises that

$$\inf_{x \neq \mathbf{0}} \frac{\|A \cdot x\|}{\|x\|} = \frac{1}{\|A^{-1}\|}$$

holds. It follows that:

$$\sup_{x \neq \mathbf{0}} \kappa_f(x) = \sup_{x \neq \mathbf{0}} \frac{\|A\|}{\frac{\|A \cdot x\|}{\|x\|}} = \frac{\|A\|}{\inf\limits_{x \neq \mathbf{0}} \frac{\|A \cdot x\|}{\|x\|}} = \|A\| \cdot \|A^{-1}\| = \kappa_{\|\cdot\|}(A).$$

Thus $\kappa_{\|\cdot\|}(A)$ is the least upper bound for $\kappa_f(x)$ that holds for all $x \neq \mathbf{0}$. It represents the "worst case" value of the condition, i.e., the maximal possible sensitivity of the function.

(b) In a finite-dimensional vector space all norms are equivalent to each other. This means:

If V is a vector space and $\|\cdot\|$ and $|||\cdot|||$ are both norms, then (proof in Appendix §3)

$$\exists C_1, C_2 > 0 : \quad C_1 \cdot \|x\| \leq |||x||| \leq C_2 \cdot \|x\| \quad \forall x \in V.$$

It also follows that the condition numbers with respect to different norms can be estimated against each other and are therefore equivalent in the same sense. For example,

$$\exists C_1, C_2 > 0 : \quad C_1 \cdot \kappa_{\|\cdot\|_\infty}(A) \leq \kappa_{\|\cdot\|_2}(A) \leq C_2 \cdot \kappa_{\|\cdot\|_\infty}(A).$$

Even if A^{-1} is known, it is numerically considerably more expensive to compute $\kappa_{\|\cdot\|_2}(A)$ (see the exercises) than $\kappa_{\|\cdot\|_\infty}(A)$ or $\kappa_{\|\cdot\|_1}(A)$.

Lemma 3.3.6

Let $B \in \mathbb{R}^{n \times n}$ and let $\|B\| < 1$ hold for a submultiplicative matrix norm $\|\cdot\|$. Then $I - B$ is invertible, and we have:

$$(I - B)^{-1} = \sum_{k=0}^{\infty} B^k \quad \textit{and} \quad \left\|(I - B)^{-1}\right\| \leq \frac{1}{1 - \|B\|}.$$

Remark 3.3.7 (Convergence of matrix series)

If $(B_k)_{k=0}^{\infty}$ is a sequence of $n \times n$ matrices, then the series $\sum_{k=0}^{\infty} B_k$ converges if and only if there exists a limit $S \in \mathbb{R}^{n\times n}$ to which the sequence of partial sums $(S_N)_{N=0}^{\infty}$ with $S_N = \sum_{k=0}^{N} B_k$ converges, i.e.,

$$\forall \varepsilon > 0 \; \exists N(\varepsilon) : \; \left\| S - \sum_{k=0}^{N} B_k \right\| < \varepsilon \quad \forall N > N(\varepsilon)$$

must hold. Fortunately, a finite-dimensional normed vector space is always a Banach space (proof in Appendix §3.5) and convergence can be established with the Cauchy criterion

$$\forall \varepsilon > 0 \; \exists N(\varepsilon) : \; \|S_N - S_M\| = \left\| \sum_{k=M+1}^{N} B_k \right\| < \varepsilon \quad \forall N \geq M > N(\varepsilon)$$

without having to know the limit. If

$$\sum_{k=0}^{\infty} \|B_k\| < \infty \tag{3.7}$$

holds, then the Cauchy criterion is satisfied.

Proof: From the absolute convergence (3.7) it follows that the tail of the series converges to zero. Thus

$$\lim_{N\to\infty} \sum_{k=N}^{\infty} \|B_k\| = 0 \quad \Longleftrightarrow \quad \forall \varepsilon > 0 \; \exists N(\varepsilon) : \; \sum_{k=N}^{\infty} \|B_k\| < \varepsilon \quad \forall N \geq N(\varepsilon)$$

and from this it follows that

$$\left\| \sum_{k=M+1}^{N} B_k \right\| \leq \sum_{k=M+1}^{N} \|B_k\| \leq \sum_{k=M}^{\infty} \|B_k\| < \varepsilon \quad \forall N \geq M > N(\varepsilon)$$

□

Proof of Lemma 3.3.6: Here one considers the sequence of matrices $(B_k)_{k=0}^{\infty}$ with $B_k := B^k$. From submultiplicativity and $\|B\| < 1$ it follows that

$$\left\| \sum_{k=0}^{\infty} B^k \right\| \leq \sum_{k=0}^{\infty} \left\| B^k \right\| \leq \sum_{k=0}^{\infty} \|B\|^k \overset{\text{geometric series}}{=\!=\!=} \frac{1}{1 - \|B\|} < \infty \tag{3.8}$$

and hence the convergence of the series $\sum_{k=0}^{\infty} B^k$ to a matrix S. In terms of the partial sums we thus have

$$\lim_{N\to\infty} \underbrace{\sum_{k=0}^{N} B^k}_{=:S_N} = \lim_{N\to\infty} S_N = S$$

From (3.8) it also follows that

$$\lim_{N\to\infty} \left\|B^N\right\| = 0 \quad \Longleftrightarrow \quad \lim_{N\to\infty} B^N = \mathbf{0}$$

If one now considers

$$\begin{aligned}(I-B)\cdot S_N = (I-B)\sum_{k=0}^{N} B^k &= \sum_{k=0}^{N}(I-B)\cdot B^k \\ &= \sum_{k=0}^{N}\left(B^k - B^{k+1}\right) = \sum_{k=0}^{N} B^k - \sum_{k=0}^{N} B^{k+1} \\ &= \sum_{k=0}^{N} B^k - \sum_{k=1}^{N+1} B^k \\ &= I - B^{N+1}\end{aligned}$$

it follows that

$$\lim_{N\to\infty}(I-B)\cdot S_N = (I-B)\cdot \lim_{N\to\infty} S_N = I \quad \Longleftrightarrow \quad S = \lim_{N\to\infty} S_N = (I-B)^{-1}$$

□

Derivation 3.3.8 (Condition of a linear system)

We consider the equations for the perturbed and the unperturbed linear system:

$$(A+\delta A)\cdot \widetilde{x} = b + \delta b, \tag{3.9}$$

$$A\cdot x = b \quad \Longleftrightarrow \quad (A+\delta A)\cdot x = b + \delta A\cdot x. \tag{3.10}$$

Subtracting (3.10) from (3.9), we obtain

$$\begin{aligned}(A+\delta A)\cdot(\widetilde{x}-x) &= \delta b - \delta A\cdot x, \\ \Longleftrightarrow A\cdot\left(I + A^{-1}\cdot\delta A\right)\cdot(\widetilde{x}-x) &= \delta b - \delta A\cdot x.\end{aligned} \tag{3.11}$$

In the case $\|A^{-1}\|\cdot\|\delta A\| = \kappa(A)\cdot\frac{\|\delta A\|}{\|A\|} < 1$, which implies $\|A^{-1}\cdot\delta A\| \le \|A^{-1}\|\cdot\|\delta A\| < 1$, the inverse $(I + A^{-1}\cdot\delta A)^{-1}$ exists, and we have

$$\begin{aligned}\left\|\left(I + A^{-1}\cdot\delta A\right)^{-1}\right\| &\le \frac{1}{1-\|A^{-1}\cdot\delta A\|} \\ &\le \frac{1}{1-\|A^{-1}\|\cdot\|\delta A\|} \quad \text{since } \|A^{-1}\cdot\delta A\| \le \|A^{-1}\|\cdot\|\delta A\|, \\ &= \frac{1}{1-\|A^{-1}\|\cdot\|A\|\cdot\frac{\|\delta A\|}{\|A\|}}, \\ &= \frac{1}{1-\kappa(A)\cdot\frac{\|\delta A\|}{\|A\|}},\end{aligned} \tag{3.12}$$

where $\kappa(A) = \|A\|\cdot\|A^{-1}\|$ is the condition number of A.

Under this assumption, (3.11) can be solved for $\delta x = \widetilde{x} - x$, which gives

$$\delta x = \widetilde{x} - x = \left(I + A^{-1} \cdot \delta A\right)^{-1} \cdot A^{-1} \cdot (\delta b - \delta A \cdot x)\,.$$

For the norm of the absolute error we obtain the following estimate, where the triangle inequality, the compatibility of the matrix norm, and (3.12) are used:

$$\begin{aligned} \|\delta x\| &= \left\| \left(I + A^{-1} \cdot \delta A\right)^{-1} \cdot A^{-1} \cdot (\delta b - \delta A \cdot x) \right\| \\ &\leq \left\| \left(I + A^{-1} \cdot \delta A\right)^{-1} \right\| \cdot \left\|A^{-1}\right\| \cdot (\|\delta b\| + \|\delta A\| \cdot \|x\|) \\ &\leq \frac{1}{1 - \kappa(A) \cdot \frac{\|\delta A\|}{\|A\|}} \cdot \left\|A^{-1}\right\| \cdot (\|\delta b\| + \|\delta A\| \cdot \|x\|)\,. \end{aligned} \tag{3.13}$$

In the case $x \neq \mathbf{0}$ we obtain for $\frac{\|\delta x\|}{\|x\|}$ an upper bound that depends on the quantities $\frac{\|\delta A\|}{\|A\|}$ and $\frac{\|\delta b\|}{\|b\|}$ as well as on the condition number $\kappa(A)$:

$$\begin{aligned} \frac{\|\delta x\|}{\|x\|} &\leq \frac{\|A^{-1}\|}{1 - \kappa(A) \cdot \frac{\|\delta A\|}{\|A\|}} \cdot \left(\frac{\|\delta b\|}{\|x\|} + \|\delta A\| \right) \\ &\leq \frac{\kappa(A)}{1 - \kappa(A) \cdot \frac{\|\delta A\|}{\|A\|}} \cdot \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right) \quad \text{since } \|b\| = \|A \cdot x\| \leq \|A\| \cdot \|x\|. \end{aligned} \tag{3.14}$$

Remark 3.3.9 (Interpretation of the condition number)

(a) The estimate (3.14) shows that the condition number $\kappa(A)$ is a key quantity for a linear system. It describes the sensitivity of the solution x to changes in the coefficient matrix A and the right-hand side b. Such perturbations can occur, for example, when A and b are not exactly representable or, in certain applications, are known only inaccurately.

This estimate is independent of additional errors that may arise during the numerical computation. This means that (3.14) is a bound for the unavoidable error that arises solely from the inaccuracies in the input data.

(b) The term $\kappa(A) \cdot \frac{\|\delta A\|}{\|A\|}$ is often negligible, i.e., we have $\kappa(A) \cdot \frac{\|\delta A\|}{\|A\|} \ll 1$. In this case the estimate (3.14) is often written as a qualitative estimate:

$$\frac{\|\delta x\|}{\|x\|} \lesssim \kappa(A) \cdot \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right).$$

(c) The estimate (3.14) has the following illustrative consequences: With t-digit decimal floating-point arithmetic, the relative error of the input data (we use the maximum norm here as an example) can be of the order

$$\frac{\|\delta A\|}{\|A\|} \approx 5 \cdot 10^{-t} \quad \text{and} \quad \frac{\|\delta b\|}{\|b\|} \approx 5 \cdot 10^{-t}$$

If $\kappa_{\|\cdot\|_\infty} \approx 10^{\alpha}$ with $5 \cdot 10^{\alpha - t} \ll 1$, then (3.14) leads to the qualitative estimate

$$\frac{\|\delta x\|_\infty}{\|x\|_\infty} \lesssim 10^{1+\alpha-t} = 10^{-(t-\alpha-1)}.$$

This says that the change can amount to up to one unit in the $(t-\alpha-1)$-th decimal place relative to $\|x\|_\infty$. This yields the following rule of thumb:

If a linear system $A \cdot x = b$ is solved with t-digit decimal floating-point arithmetic and the condition number is $\kappa_{\|\cdot\|_\infty} \approx 10^\alpha$, then, because of the unavoidable errors in the input data A and b, only $t - \alpha - 1$ decimal digits of the computed solution $\widetilde{x}$ (relative to the component of largest absolute value) are reliable.

Chapter 4 Linear Least Squares Problems

In the natural sciences and many other fields of application it is often necessary to find a mathematical model that describes a certain relationship between quantities. Such models frequently depend on unknown parameters that have to be determined from experimental data.

A simple example of this is the linear function $f(t) = \alpha \cdot t + \beta$, where α and β are the parameters to be determined. These parameters can be estimated by measurements in which $y_i = f(t_i)$ is determined for different values of t_i. Since measurements are always subject to measurement errors, i.e., we only have $y_i \approx f(t_i)$, one performs several measurements in order to increase the accuracy.

Suppose we are given m measurements $(t_1, y_1), (t_2, y_2), \ldots, (t_m, y_m)$, where m is considerably larger than the number of unknown parameters. Figure 4.1 illustrates this for the case $m = 4$. The problem now consists in determining the parameters α and β such that the function f fits the measured data as well as possible.

Sources and further reading. The material of this chapter is covered in all standard texts on numerical linear algebra, for example [4], [18] and [8]. Householder transformations go back to Householder [12]. The compact representation of the Householder vectors described at the end of §2 corresponds to the representation used in LAPACK [1]. Givens rotations (§3) are named after Givens [7].

§1 The Method of Least Squares

In order to determine the parameters α and β, one requires that the squared deviation between the model values and the measured values be minimized. This leads to the optimization problem

$$g(\alpha, \beta) = \sum_{i=1}^{m} \left(f(t_i) - y_i\right)^2 = \sum_{i=1}^{m} \left(\alpha \cdot t_i + \beta - y_i\right)^2 \to \min\,. \tag{4.1}$$

To simplify the problem we convert the sum representation into matrix form. If we define

$$A := \begin{pmatrix} t_1 & 1 \\ \vdots & \vdots \\ t_m & 1 \end{pmatrix}, \quad b := \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}, \quad x := \begin{pmatrix} \alpha \\ \beta \end{pmatrix},$$

then the objective function can be written in compact form as

$$g(\alpha, \beta) = \|A \cdot x - b\|_2^2 = (A \cdot x - b)^T \cdot (A \cdot x - b)\,. \tag{4.2}$$

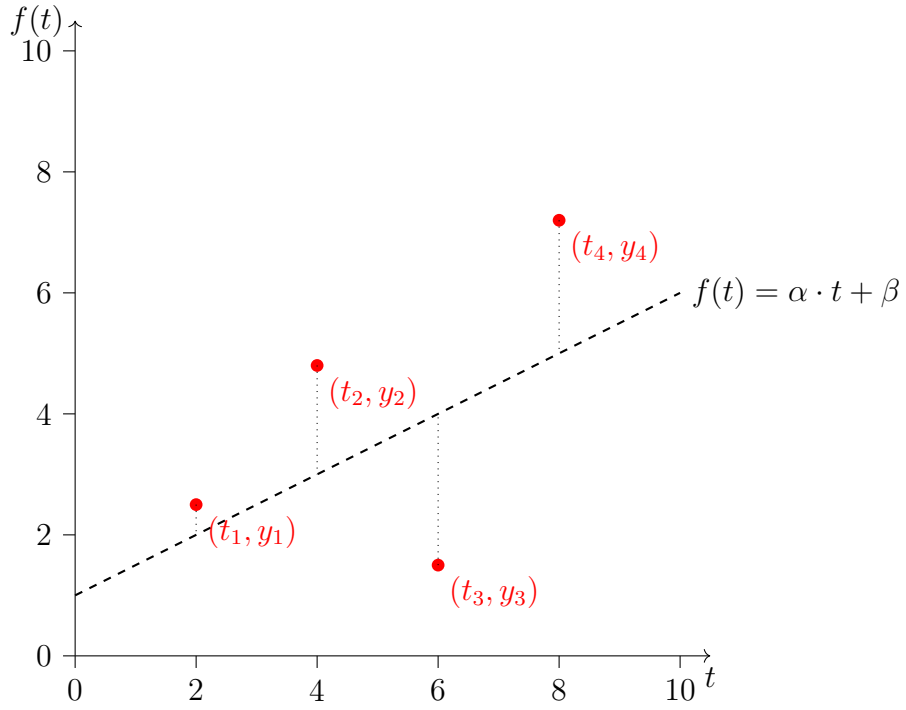


Figure 4.1: Linear model with $m = 4$ measurement points.

Here the vector $A \cdot x$ corresponds to the modeled values, and b contains the measured values. Multiplying out gives

$$g(\alpha, \beta) = \|A \cdot x - b\|_2^2 = x^T \cdot A^T \cdot A \cdot x - 2 \cdot b^T \cdot A \cdot x + b^T \cdot b. \tag{4.3}$$

This allows us to treat the problem analytically. At a local minimizer the gradient of the objective function must vanish:

$$\nabla g(x) = 2 \cdot A^T \cdot A \cdot x - 2 \cdot A^T \cdot b \stackrel{!}{=} \mathbf{0}.$$

This leads to the normal equations

$$A^T \cdot A \cdot x = A^T \cdot b. \tag{4.4}$$

A local minimum is present if the Hessian $H_g(x) = 2 \cdot A^T \cdot A$ is positive definite. The matrix $A^T \cdot A$ is at least positive semidefinite, because

$$x^T \cdot A^T \cdot A \cdot x = \|A \cdot x\|_2^2 \geq 0 \quad \forall x \in \mathbb{R}^2.$$

If $\operatorname{rank}(A) = 2$, it follows moreover that

$$x^T \cdot A^T \cdot A \cdot x > 0 \quad \forall x \in \mathbb{R}^2 \setminus \{\mathbf{0}\}.$$

Then $H_g(x)$ is positive definite and the local minimizer is unique. Since the objective function is quadratic and the Hessian is positive definite everywhere, one can show that the local minimum found is at the same time the global minimum, which is unique as well.

Numerically, the problem can be solved by solving the normal equations (4.4). To this end one first sets up the coefficient matrix $A^T A$ and the right-hand side $A^T b$. Afterwards

the Cholesky factorization can be used to solve the system efficiently. The Cholesky factorization, developed by André-Louis Cholesky at the beginning of the 20th century, was originally designed exactly for this purpose, in particular for solving normal equations in geodesy.

However, this is not necessarily the best method. On the one hand, the additional computation of A^TA and A^Tb causes additional computational cost. On the other hand, forming the matrix A^TA worsens the condition of the problem, which can amplify numerical instabilities. For this reason, alternative methods such as the QR factorization, which work directly on the original matrix A without computing A^TA explicitly, are often preferred in practice.

These numerical drawbacks suggest looking for alternatives that work more efficiently and more stably with the original matrix A. The following theorem nevertheless remains of central importance: it guarantees the existence and uniqueness of the solution of the normal equations and thus forms the basis for developing alternative methods that can compute the same solution in different ways.

Theorem 4.1.1 (Normal equations)

Let $A \in \mathbb{R}^{m\times n}$ with $m \geq n$ and $\operatorname{rank}(A) = n$. Then:

$$\|A \cdot \widehat{x} - b\|_2 = \min_{x\in\mathbb{R}^n} \|A \cdot x - b\|_2 \quad \Longleftrightarrow \quad \widehat{x} = \left(A^T \cdot A\right)^{-1} \cdot A^T \cdot b.$$

The minimization problem, and hence the linear least squares problem, therefore has a unique solution.

Proof: Let $g(x) = \|A \cdot x - b\|_2^2$, where $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ and $x \in \mathbb{R}^n$. The objective function $g(x)$ is quadratic and differentiable. The Hessian of $g(x)$ is given by

$$H_g(x) = 2 \cdot A^T \cdot A.$$

Since $\operatorname{rank}(A) = n$ is assumed, $A^T \cdot A$ is symmetric, positive definite and invertible. Hence $g(x)$ has exactly one stationary point x_0, which is determined by the normal equations

$$A^T \cdot A \cdot x_0 = A^T \cdot b.$$

The solution is unique and reads

$$x_0 = \left(A^T \cdot A\right)^{-1} \cdot A^T \cdot b.$$

To show that x_0 is the global minimum, we consider the difference

$$g(x) - g(x_0) = \|A \cdot x - b\|_2^2 - \|A \cdot x_0 - b\|_2^2.$$

By rearranging and using the fact that the normal equations hold, one obtains

$$g(x) - g(x_0) = \|(A \cdot x - b) - (A \cdot x_0 - b)\|_2^2 = \|A \cdot (x - x_0)\|_2^2.$$

(The complete derivation is treated in an exercise.) Since A has full rank, we have $\|A \cdot (x - x_0)\|_2^2 > 0$ for $x \neq x_0$. Thus

$$g(x) > g(x_0) \quad \text{for all } x \neq x_0.$$

It follows that x_0 is the unique solution of the minimization problem and that $g(x_0)$ is the global minimum. ∎

§2 QR Factorization: Householder Transformations

Before we turn to the QR factorization, we define the notion of an orthogonal matrix. Orthogonal matrices play a central role in the QR factorization, since they describe transformations that preserve the Euclidean distance between vectors. The columns of an orthogonal matrix are orthogonal with respect to the Euclidean inner product

$$\langle x, y \rangle = x^T \cdot y$$

and have norm 1 with respect to the Euclidean norm induced by it. This norm is the 2-norm, i.e.,

$$\|x\|_2 = \sqrt{x^T \cdot x}.$$

An orthogonal matrix Q describes a linear map that leaves lengths and angles invariant. Such maps can be interpreted as rotations or reflections of the space.

Definition 4.2.1 (Orthogonal matrix)

A square matrix Q is called orthogonal if $Q^T \cdot Q = I$.

That an orthogonal matrix is a length-preserving map is recorded in the following lemma.

Lemma 4.2.2

If $x \in \mathbb{R}^n$ and $Q \in \mathbb{R}^{n \times n}$ is an orthogonal matrix, then

$$\|Q \cdot x\|_2 = \|x\|_2.$$

Proof: From $Q^T \cdot Q = I$ it follows that

$$\|Q \cdot x\|_2^2 = (Q \cdot x)^T \cdot (Q \cdot x) = x^T \cdot Q^T \cdot Q \cdot x = x^T \cdot I \cdot x = \|x\|_2^2.$$

Since $\| \cdot \|_2 \geq 0$, this implies

$$\|Q \cdot x\|_2 = \|x\|_2.$$

□

The following theorem provides the basis for an alternative method for solving the linear least squares problem that is numerically more stable than directly solving the normal equations.

Theorem 4.2.3 (QR factorization)

Let $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ and $\operatorname{rank}(A) = n$. *Then there exist*

$$Q \in \mathbb{R}^{m \times m}, \quad R \in \mathbb{R}^{m \times n}, \quad R_1 \in \mathbb{R}^{n \times n},$$

where R_1 is an invertible upper triangular matrix, such that

$$A = QR, \quad Q^T Q = I, \quad R = \begin{pmatrix} R_1 \\ \mathbf{0} \end{pmatrix}.$$

Before the theorem is proved, and a numerical method for computing Q and R is developed along the way, the following corollary shows how the QR factorization can be used to solve the linear least squares problem.

Corollary 4.2.4

Let $A \in \mathbb{R}^{m\times n}$ with $m > n$ and $\operatorname{rank}(A) = n$. Furthermore, let a factorization $A = Q \cdot R$ as in Theorem 4.2.3 be known. Considering the block structures

$$Q = \begin{pmatrix} Q_1 & Q_2 \end{pmatrix}, \quad Q_1 \in \mathbb{R}^{m\times n},\ Q_2 \in \mathbb{R}^{m\times(m-n)},$$

$$R = \begin{pmatrix} R_1 \\ \mathbf{0} \end{pmatrix}, \qquad R_1 \in \mathbb{R}^{n\times n},$$

we have for the linear least squares problem with $b \in \mathbb{R}^m$:

$$\|A \cdot \widehat{x} - b\|_2 = \min_{x\in\mathbb{R}^n} \|A \cdot x - b\|_2 \quad \Longleftrightarrow \quad \widehat{x} = R_1^{-1} \cdot Q_1^T \cdot b.$$

Proof:
We have

$$Q^T \cdot A = \begin{pmatrix} Q_1^T \\ Q_2^T \end{pmatrix} \cdot A = \begin{pmatrix} Q_1^T \cdot A \\ Q_2^T \cdot A \end{pmatrix} = \begin{pmatrix} R_1 \\ \mathbf{0} \end{pmatrix} = R, \quad \text{and} \quad Q^T \cdot b = \begin{pmatrix} Q_1^T \cdot b \\ Q_2^T \cdot b \end{pmatrix}.$$

By Lemma 4.2.2 it follows that

$$\begin{aligned} \|A \cdot x - b\|_2^2 &= \left\|Q^T \cdot (A \cdot x - b)\right\|_2^2 = \left\|Q^T \cdot A \cdot x - Q^T \cdot b\right\|_2^2 \\ &= \left\| \begin{pmatrix} Q_1^T \cdot A \cdot x - Q_1^T \cdot b \\ Q_2^T \cdot A \cdot x - Q_2^T \cdot b \end{pmatrix} \right\|_2^2 = \left\| \begin{pmatrix} R_1 \cdot x - Q_1^T \cdot b \\ -Q_2^T \cdot b \end{pmatrix} \right\|_2^2 \\ &= \left\|R_1 \cdot x - Q_1^T \cdot b\right\|_2^2 + \left\|Q_2^T \cdot b\right\|_2^2. \end{aligned}$$

Since R_1 is invertible, one can choose $\widehat{x} = R_1^{-1} \cdot Q_1^T \cdot b$ and obtains

$$\|A \cdot \widehat{x} - b\|_2^2 = \left\|Q_2^T \cdot b\right\|_2^2 \leq \left\|R_1 \cdot x - Q_1^T \cdot b\right\|_2^2 + \left\|Q_2^T \cdot b\right\|_2^2 = \|A \cdot x - b\|_2^2 \quad \forall x \in \mathbb{R}^n.$$

It follows that

$$\|A \cdot \widehat{x} - b\|_2^2 \leq \|A \cdot x - b\|_2^2 \quad \forall x \in \mathbb{R}^n \quad \Longleftrightarrow \quad \|A \cdot \widehat{x} - b\|_2 = \min_{x\in\mathbb{R}^n} \|A \cdot x - b\|_2$$

From Theorem 4.1.1 it follows that $\widehat{x}$ is the unique solution of the linear least squares problem. ☐

For proving the existence of the QR factorization and for computing this factorization, we first motivate the so-called Householder transformation.

Motivation 4.2.5 (Householder transformation; reflection of a vector)

The reflection of a vector in a (hyper)plane is a linear map that can be represented by an orthogonal matrix, provided the reflection fixes the origin. For $\mathbb{R}^2$ we show here, using the methods of analytic geometry, how a matrix H can be constructed that reflects an arbitrary vector $a \in \mathbb{R}^2$ with $a \neq \mathbf{0}$ onto the x-axis.

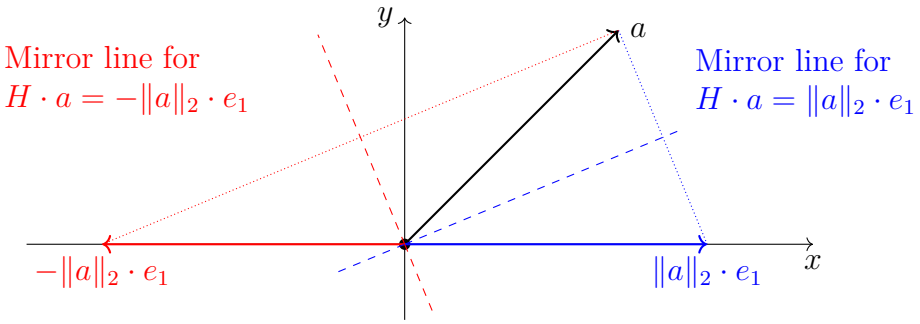


Figure 4.2: Ways of reflecting a onto the x-axis.

Figure 4.2 illustrates that there are two ways of reflecting a and thus of determining H. Formally, this can be derived as follows: The matrix H is to be orthogonal, i.e., it is to satisfy

$$H^T \cdot H = I \quad \text{and at the same time} \quad H \cdot a = \alpha \cdot e_1.$$

Thus the vector a is to be mapped by H to a multiple of e_1. It follows that

$$|\alpha| = \|\alpha \cdot e_1\|_2 = \|H \cdot a\|_2 \overset{H^T \cdot H = I}{=\!=\!=} \|a\|_2 \quad \Longrightarrow \quad \alpha = \pm\|a\|_2.$$

So H can be chosen such that either

$$H \cdot a = \|a\|_2 \cdot e_1 \quad \text{or} \quad H \cdot a = -\|a\|_2 \cdot e_1$$

holds.

For the further derivation we make use of an important result from analytic geometry. While classical geometry needs no coordinates, analytic geometry, by introducing a coordinate system, allows an algebraic description of geometric facts. This connection is particularly helpful for deriving transformations such as the Householder transformation in a way that is both intuitive and precise. In $\mathbb{R}^2$ we have

$$a^T \cdot b = \|a\|_2 \cdot \|b\|_2 \cdot \cos\alpha \quad \text{with } \alpha = \angle(a, b),$$

where $\|\cdot\|_2$ is the Euclidean norm and $\angle(a, b)$ is the angle enclosed by a and b. Figure 4.3 illustrates this relationship by showing the projection of a onto b, the angle α and the geometric meaning of the inner product.

This relationship connects geometric and algebraic aspects as follows:

- **Geometrically:** The inner product describes the length of the projection of a onto b, multiplied by the length of b.
- **Algebraically:** It allows the geometric quantity to be computed directly from the components of the vectors, without the angle $\angle(a, b)$ having to be known.

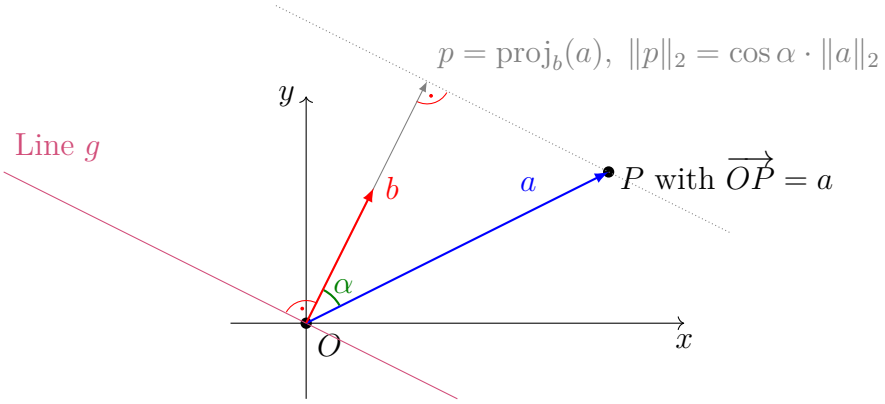


Figure 4.3: Illustration of the Euclidean inner product. The picture shows the projection $\text{proj}_b(a)$ of a onto b, the enclosed angle α, and the orthogonal relationship between b and a line.

For us, the case where b is a unit vector, i.e., $\|b\|_2 = 1$, will be important. In this case we have

$$a^T b \overset{\|b\|_2=1}{=\!=\!=} \|a\|_2 \cdot \cos\alpha \quad \text{with } \alpha = \angle(a, b).$$

In this case the inner product computes a kind of "oriented" length of the projection of a onto b. The sign indicates whether a points in the same direction as b $(+)$ or in the opposite direction $(-)$. If one identifies a with the position vector of a point P and b with a normal vector of a line g, then the inner product yields, in the same sense, an "oriented" distance of P from the line g. The sign indicates on which side of the line the point P lies.

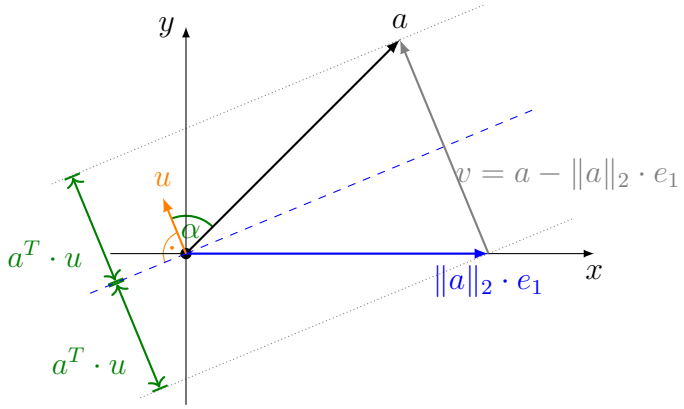


Figure 4.4: Construction of the Householder transformation with $H \cdot a = \|a\|_2 \cdot e_1$.

Figure 4.4 sketches how a Householder transformation can be constructed that maps a by a reflection onto $\|a\|_2 \cdot e_1$. Here we consider a case in which a does not lie on the x-axis, i.e., $|a_1| \neq \|a\|_2$. If one defines the connecting vector

$$v := a - \|a\|_2 \cdot e_1 \neq \mathbf{0}, \quad \text{since } |a_1| \neq \|a\|_2,$$

then it is perpendicular to the mirror line. The normalized vector

$$u := \frac{v}{\|v\|_2}$$

is therefore a normal vector of the mirror line. The quantity

$$a^T \cdot u \overset{\|u\|_2=1}{=\!=\!=} \|a\|_2 \cdot \cos\alpha \quad \text{with } \alpha = \angle(a,u)$$

corresponds to the "oriented" length of a in the direction of u.

Now the equation

$$a - 2 \cdot \left(a^T \cdot u\right) \cdot u = \|a\|_2 \cdot e_1 \tag{4.5}$$

must hold. This can be justified and illustrated geometrically: If one interprets the vector addition on the left-hand side, one starts at a and "walks" orthogonally to the mirror line in the direction of $-u$. The distance covered equals twice the distance of a from the mirror line. One crosses the mirror line and ends up on the opposite side at $\|a\|_2 \cdot e_1$.

Rearranging (4.5) yields

$$\begin{aligned}\|a\|_2 \cdot e_1 &= a - 2 \cdot \left(a^T \cdot u\right) \cdot u = a - 2 \cdot u \cdot \left(u^T \cdot a\right) = a - 2 \cdot \left(u \cdot u^T\right) \cdot a \\ &= \left(I - 2 \cdot u \cdot u^T\right) \cdot a.\end{aligned}$$

Substituting $u = \frac{v}{\|v\|_2}$, we obtain the matrix of the Householder transformation as

$$H = I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \quad \text{with } v = a - \|a\|_2 \cdot e_1 \quad \text{and } H \cdot a = \|a\|_2 \cdot e_1.$$

If a is instead to be mapped onto $-\|a\|_2 \cdot e_1$, one proceeds analogously and obtains

$$H = I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \quad \text{with } v = a + \|a\|_2 \cdot e_1 \quad \text{and } H \cdot a = -\|a\|_2 \cdot e_1.$$

That the matrix H is indeed orthogonal will be shown in the proof of Lemma 4.2.6, in which the Householder transformation is considered more generally in $\mathbb{R}^n$.

If a already lies on the x-axis, H can be chosen as the identity matrix. Otherwise one has to take into account that rounding errors occur in the numerical implementation of the Householder transformation. To avoid cancellation and thereby improve numerical stability, one exploits the fact that there are two ways of reflecting a vector a onto the x-axis. We will confirm experimentally in the exercises that it is advantageous to choose the reflection for which the distance between a and $H \cdot a$ is as large as possible. This is achieved by proceeding as follows: If the first component a_1 is nonnegative, one reflects a onto $-\|a\|_2 \cdot e_1$, otherwise onto $\|a\|_2 \cdot e_1$. This procedure is taken into account in the following lemma in the definition of the Householder transformation.

Lemma 4.2.6 (Householder transformation)

Let $a \in \mathbb{R}^n \setminus \{\mathbf{0}\}$. Then there exists a symmetric orthogonal matrix H such that

$$H \cdot a = \alpha \cdot e_1 \quad \textit{with} \quad |\alpha| = \|a\|_2.$$

In the case $\|a\|_2 = |a_1|$, one can choose $H = I$. Otherwise the matrix

$$H = I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v}$$

with

$$v = a - \alpha \cdot e_1 \quad \textit{and} \quad \alpha = \begin{cases} -\|a\|_2, & \textit{if } a_1 \geq 0, \\ \|a\|_2, & \textit{if } a_1 < 0 \end{cases} \tag{4.6}$$

has these properties.

Remark: With (4.6), cancellation in the first component is avoided in the numerical computation of v. In general, this ensures that $v_1 = a_1 - \alpha \neq 0$ and hence $v \neq \mathbf{0}$, so that H is well defined.

The vector v used to define the Householder matrix H is also called the reflection vector; in the following, however, we simply call it the Householder vector.

Proof: We distinguish two cases:

Case 1: $\|a\|_2 = |a_1|$ In this case $a_2 = \cdots = a_n = 0$, and thus $a = a_1 \cdot e_1$. Obviously one can choose $H = I$ and $\alpha = a_1$, since $I \cdot a = a = a_1 \cdot e_1$ and $|a_1| = \|a\|_2$. Moreover, I is both symmetric and orthogonal.

Case 2: $\|a\|_2 \neq |a_1|$ For the representation

$$H = I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v},$$

we first show that H is symmetric:

$$H^T = \left(I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \right)^T \overset{(v^T)^T = v}{=\!=\!=} I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} = H.$$

Hence H is symmetric.

To show orthogonality, we consider the product $H \cdot H^T$:

$$\begin{aligned} H \cdot H^T = H \cdot H &= \left(I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \right) \cdot \left(I - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \right) \\ &= I - 4 \cdot \frac{v \cdot v^T}{v^T \cdot v} + 4 \cdot \frac{(v \cdot v^T) \cdot (v \cdot v^T)}{(v^T \cdot v) \cdot (v^T \cdot v)} \\ &= I - 4 \cdot \frac{v \cdot v^T}{v^T \cdot v} + 4 \cdot \frac{v \cdot (v^T \cdot v) \cdot v^T}{(v^T \cdot v) \cdot (v^T \cdot v)} \\ &\overset{v^T \cdot v \in \mathbb{R}}{=\!=\!=} I - 4 \cdot \frac{v \cdot v^T}{v^T \cdot v} + 4 \cdot \frac{v \cdot v^T}{v^T \cdot v} = I. \end{aligned}$$

Hence H is orthogonal.

Algebraic Verification of $H \cdot a = \alpha \cdot e_1$: To show that $H \cdot a = \alpha \cdot e_1$ holds, we carry out an algebraic verification. From

$$2 \cdot \frac{v^T \cdot a}{v^T \cdot v} = 2 \cdot \frac{(a - \alpha \cdot e_1)^T \cdot a}{(a - \alpha \cdot e_1)^T \cdot (a - \alpha \cdot e_1)} = 2 \cdot \frac{a^T \cdot a - \alpha \cdot e_1^T \cdot a}{a^T \cdot a - 2 \cdot \alpha \cdot a^T \cdot e_1 + \alpha^2}$$

$$= 2 \cdot \frac{\|a\|_2^2 - \alpha \cdot a_1}{\|a\|_2^2 - 2 \cdot \alpha \cdot a_1 + \|a\|_2^2} = 2 \cdot \frac{\|a\|_2^2 - \alpha \cdot a_1}{2 \cdot \|a\|_2^2 - 2 \cdot \alpha \cdot a_1} = 1$$

it follows that

$$H \cdot a = a - 2 \cdot \frac{v \cdot v^T}{v^T \cdot v} \cdot a = a - 2 \cdot \frac{v^T \cdot a}{v^T \cdot v} \cdot v = a - v = \alpha \cdot e_1.$$

∎

Example 4.2.7 (Determining and applying a Householder transformation)

This example uses simple calculations to illustrate how Householder transformations can be determined and applied. In addition, it shows how the QR factorization is computed iteratively with Householder transformations.

(a) In the first part of the example we consider

$$a = \begin{pmatrix} 4 \\ 3 \end{pmatrix}.$$

We determine the Householder matrix H_1 according to Lemma 4.2.6.

We have $\|a\|_2 = \sqrt{16+9} = 5$, and since $a_1 = 4 \geq 0$ we get $\alpha = -5$. We thus obtain

$$v = a + 5 \cdot e_1 = \begin{pmatrix} 9 \\ 3 \end{pmatrix}, \quad v^T v = 81 + 9 = 90.$$

The Householder matrix is then

$$\begin{aligned} H_1 &= I - 2 \cdot \frac{v \cdot v^T}{v^T v} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{2}{90} \cdot \begin{pmatrix} 81 & 27 \\ 27 & 9 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{5} \cdot \begin{pmatrix} 9 & 3 \\ 3 & 1 \end{pmatrix} = \frac{1}{5} \cdot \left(\begin{pmatrix} 5 & 0 \\ 0 & 5 \end{pmatrix} - \begin{pmatrix} 9 & 3 \\ 3 & 1 \end{pmatrix} \right) \\ &= \frac{1}{5} \cdot \begin{pmatrix} -4 & -3 \\ -3 & 4 \end{pmatrix}. \end{aligned}$$

As a check, one can verify that

$$H_1 \cdot a = \frac{1}{5} \cdot \begin{pmatrix} -4 & -3 \\ -3 & 4 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} -5 \\ 0 \end{pmatrix}$$

holds.

(b) In the second part of the example we consider the matrix

$$A = \begin{pmatrix} -1 & -4 \\ -2 & 3 \\ -2 & 2 \end{pmatrix}.$$

A Householder transformation H_2 is to map the first column a_1 to a multiple of e_1. The second column a_2 is transformed with the same Householder transformation, so that

$$H_2 \cdot A = \begin{pmatrix} H_2 \cdot a_1 & H_2 \cdot a_2 \end{pmatrix}$$

is computed.

We have $\|a_1\|_2 = \sqrt{1+4+4} = 3$. Since $a_{11} = -1 < 0$, we get $\alpha = 3$. We thus obtain

$$v = a_1 - 3 \cdot e_1 = \begin{pmatrix} -4 \\ -2 \\ -2 \end{pmatrix}, \quad v^T v = 16 + 4 + 4 = 24.$$

The vector v was chosen such that

$$H_2 \cdot a_1 = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix}$$

holds.

To compute $H_2 \cdot a_2$, one first computes

$$v^T a_2 = \begin{pmatrix} -4 & -2 & -2 \end{pmatrix} \cdot \begin{pmatrix} -4 \\ 3 \\ 2 \end{pmatrix} = 16 - 6 - 4 = 6.$$

From this it follows that

$$\begin{aligned} H_2 \cdot a_2 &= a_2 - 2 \cdot \frac{v \cdot v^T}{v^T v} \cdot a_2 = a_2 - 2 \cdot \frac{6}{24} \cdot v \\ &= \begin{pmatrix} -4 \\ 3 \\ 2 \end{pmatrix} - \frac{1}{2} \cdot \begin{pmatrix} -4 \\ -2 \\ -2 \end{pmatrix} = \begin{pmatrix} -2 \\ 4 \\ 3 \end{pmatrix}. \end{aligned}$$

Altogether we obtain

$$H_2 \cdot A = \begin{pmatrix} 3 & -2 \\ 0 & 4 \\ 0 & 3 \end{pmatrix}.$$

For didactic reasons, the matrix H_2 is set up in full:

$$\begin{aligned} H_2 &= I - \frac{2}{v^T v} \cdot v \cdot v^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{2}{24} \cdot \begin{pmatrix} -4 \\ -2 \\ -2 \end{pmatrix} \cdot \begin{pmatrix} -4 & -2 & -2 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{12} \cdot \begin{pmatrix} 16 & 8 & 8 \\ 8 & 4 & 4 \\ 8 & 4 & 4 \end{pmatrix} = \frac{1}{12} \cdot \left(\begin{pmatrix} 12 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 12 \end{pmatrix} - \begin{pmatrix} 16 & 8 & 8 \\ 8 & 4 & 4 \\ 8 & 4 & 4 \end{pmatrix} \right) \\ &= \frac{1}{12} \cdot \begin{pmatrix} -4 & -8 & -8 \\ -8 & 8 & -4 \\ -8 & -4 & 8 \end{pmatrix}. \end{aligned}$$

(c) In the third part of the example we finally compute the QR factorization of the matrix A from (b). After the first Householder transformation H_2 we obtain:

$$H_2 \cdot A = \begin{pmatrix} 3 & -2 \\ 0 & 4 \\ 0 & 3 \end{pmatrix}.$$

Now we consider the lower right block of the transformed matrix:

$$A_{\text{lower}} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}.$$

For this block we use the Householder transformation H_1 from (a) in order to further triangularize the second column. This transformation is embedded into a block structure in which H_1 sits in the lower right corner and the rest of the matrix is an identity matrix:

$$\widetilde{H}_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -4/5 & -3/5 \\ 0 & -3/5 & 4/5 \end{pmatrix}.$$

Applying $\widetilde{H}_1$ to $H_2 \cdot A$ yields:

$$\widetilde{H}_1 \cdot H_2 \cdot A = \begin{pmatrix} 3 & -2 \\ 0 & -5 \\ 0 & 0 \end{pmatrix}.$$

This completely describes the QR factorization:

$$A = Q \cdot R,$$

where

$$Q^T = \widetilde{H}_1 \cdot H_2 \quad \text{and} \quad R = \begin{pmatrix} 3 & -2 \\ 0 & -5 \\ 0 & 0 \end{pmatrix}.$$

From this it follows that:

$$\begin{aligned} Q = H_2^T \cdot \widetilde{H}_1^T &= \frac{1}{12} \cdot \begin{pmatrix} -4 & -8 & -8 \\ -8 & 8 & -4 \\ -8 & -4 & 8 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & -4/5 & -3/5 \\ 0 & -3/5 & 4/5 \end{pmatrix} \\ &= \frac{1}{60} \cdot \begin{pmatrix} -4 & -8 & -8 \\ -8 & 8 & -4 \\ -8 & -4 & 8 \end{pmatrix} \cdot \begin{pmatrix} 5 & 0 & 0 \\ 0 & -4 & -3 \\ 0 & -3 & 4 \end{pmatrix} \\ &= \frac{1}{60} \begin{pmatrix} -20 & 56 & -8 \\ -40 & -20 & -40 \\ -40 & -8 & 44 \end{pmatrix}, \end{aligned}$$

where Q is an orthogonal matrix, since both Householder transformations are orthogonal.

We are now perfectly equipped to prove the existence of a QR factorization constructively and in general.

Proof: Of Theorem 4.2.3 (QR factorization)

Base case: $n = 1$
The matrix $A = (a_1) \in \mathbb{R}^{m\times 1}$ consists of a single column vector a_1. Since $\operatorname{rank}(A) = 1$, we have $a_1 \neq \mathbf{0}$. By Lemma 4.2.6 there exist an orthogonal matrix $H \in \mathbb{R}^{m\times m}$ and an $\alpha \in \mathbb{R}$ such that the first column a_1 is mapped to $\alpha \cdot e_1$ with $\alpha \neq 0$. With $Q = H$ and $R = \alpha \cdot e_1$ the claim is then proved.

Induction step: $n - 1 \to n$

Let $A = (a_1, a_2, \ldots, a_n) \in \mathbb{R}^{m\times n}$, where $a_1, \ldots, a_n$ are the columns of A. Since by assumption $\operatorname{rank}(A) = n$, the columns of A are linearly independent; in particular, $a_1 \neq \mathbf{0}$. By Lemma 4.2.6 there exist an orthogonal matrix $H \in \mathbb{R}^{m\times m}$ and an $\alpha \in \mathbb{R}$ such that the first column a_1 is mapped to $\alpha \cdot e_1$ with $\alpha \neq 0$. For the block structure of $H \cdot A$ we obtain:

$$H \cdot A = \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix} \qquad \begin{array}{c|cc} & 1 & n-1 \\ \hline 1 & \alpha & \widetilde{a}_{12}^T \\ m-1 & & \widetilde{A}_{22} \end{array}$$

Here the first column of A has been reduced to a simple form, while the rank of the remaining matrix is analyzed next.

In order to be able to apply the induction hypothesis, we must show that $\operatorname{rank}(\widetilde{A}_{22}) = n-1$. Suppose $\operatorname{rank}(\widetilde{A}_{22}) < n - 1$; then the columns of $\widetilde{A}_{22}$ are linearly dependent. Hence there exists an $x_2 \in \mathbb{R}^{n-1}$ with

$$\widetilde{A}_{22} \cdot x_2 = \mathbf{0} \quad \text{and} \quad x_2 \neq \mathbf{0}$$

If we define

$$x_1 := -\frac{\widetilde{a}_{12}^T \cdot x_2}{\alpha} \quad \text{and} \quad x := \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

then

$$x \neq \mathbf{0} \quad \text{and} \quad H \cdot A \cdot x = \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \alpha \cdot x_1 + \widetilde{a}_{12}^T \cdot x_2 \\ \widetilde{A}_{22} \cdot x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ \mathbf{0} \end{pmatrix}.$$

Since H is orthogonal, we have:

$$H \cdot A \cdot x = \mathbf{0} \quad \Longleftrightarrow \quad A \cdot x = \mathbf{0}.$$

However, this contradicts the fact that the columns of A are linearly independent. Therefore $\operatorname{rank}(\widetilde{A}_{22}) = n - 1$ must hold. The induction hypothesis is applicable. There exist $\widetilde{Q} \in \mathbb{R}^{(m-1)\times(m-1)}$ and $\widetilde{R} \in \mathbb{R}^{(m-1)\times(n-1)}$ such that

$$\widetilde{A}_{22} = \widetilde{Q} \cdot \widetilde{R}, \quad \widetilde{Q}^T \cdot \widetilde{Q} = I_{m-1}, \quad \widetilde{R} = \begin{pmatrix} \widetilde{R}_1 \\ \mathbf{0} \end{pmatrix},$$

where $\widetilde{R}_1$ is an invertible $(n-1) \times (n-1)$ matrix. With this representation we obtain

$$H \cdot A = \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix} = \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{Q} \cdot \widetilde{R} \end{pmatrix} = \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix} \cdot \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{R} \end{pmatrix}.$$

Transforming back with $H^T = H$ yields for A:

$$A = \underbrace{\left[H \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}\right]}_{=:Q} \cdot \underbrace{\begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{R} \end{pmatrix}}_{=:R}.$$

Since Q, as a product of orthogonal matrices, is again orthogonal, it remains to show that R has the required structure, as is shown in the following. We have

$$R = \begin{pmatrix} R_1 \\ \mathbf{0} \end{pmatrix} \quad \text{with } R_1 = \begin{pmatrix} \alpha & \widetilde{a}_{12}^T \\ & \widetilde{R}_1 \end{pmatrix}.$$

Since R_1 is an upper triangular matrix, we obtain for the determinant:

$$\det(R_1) = \alpha \cdot \det(\widetilde{R}_1).$$

Since both $\alpha \neq 0$ and $\det(\widetilde{R}_1) \neq 0$, it follows that $\det(R_1) \neq 0$. Hence R_1 is invertible. ☐

Remark 4.2.8 (Efficient storage of the QR factorization)

Often it is not necessary to set up the matrix Q explicitly, since it frequently suffices to know the Householder vectors. When computing the QR factorization, the matrix A can be overwritten in such a way that the part on and above the diagonal contains the matrix R, while the part below the diagonal is used to store the Householder vectors.

For an $a \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ one constructs a Householder vector $\widetilde{v}$ such that

$$H = I - \tau \cdot \widetilde{v} \cdot \widetilde{v}^T \quad \text{with } \tau \in \mathbb{R},\ \widetilde{v} = \begin{pmatrix} 1 \\ \widetilde{v}_2 \\ \vdots \\ \widetilde{v}_n \end{pmatrix} \in \mathbb{R}^n$$

holds. Here τ and $\widetilde{v}$ can be generated from the vector v constructed in Lemma 4.2.6 as follows:

- In the case $\|a\|_2 = |a_1|$ (then $H = I$) one sets $\tau = 0$ and $\widetilde{v} = e_1$.
- In the case $\|a\|_2 \neq |a_1|$, the vector v from the lemma satisfies $v \neq \mathbf{0}$. Then also $v_1 \neq 0$, and one sets

$$\tau = \frac{2 \cdot v_1^2}{v^T \cdot v} \quad \text{and} \quad \widetilde{v} = \frac{1}{v_1} \cdot v.$$

If the Householder transformation is used for the QR factorization of an $m \times n$ matrix A, then in the first step $\widetilde{v} \in \mathbb{R}^m$. The nontrivial entries $\widetilde{v}_2, \ldots, \widetilde{v}_m$ can be stored below $r_{1,1}$ in the first column as follows:

$$H \cdot \begin{pmatrix} a_{1,1} & a_{1,2} & \ldots & a_{1,n} \\ a_{2,1} & a_{2,2} & \ldots & a_{2,n} \\ \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & \ldots & a_{m,n} \end{pmatrix} = \left(\begin{array}{c|ccc} r_{1,1} & * & \ldots & * \\ \widetilde{v}_2 & \widetilde{a}_{2,2} & \ldots & \widetilde{a}_{2,n} \\ \vdots & \vdots & & \vdots \\ \widetilde{v}_m & \widetilde{a}_{m,2} & \ldots & \widetilde{a}_{m,n} \end{array}\right)$$

In addition, the value τ is stored externally.

In the case $m \geq n$, a QR factorization thus requires additional storage for only n values in total, which considerably simplifies the storage compared with an explicit construction of the matrix Q.

§3 QR Factorization with Givens Rotations

A Householder transformation annihilates all entries of a column below the diagonal in one step. Givens rotations are more selective: a Givens rotation annihilates a single entry. A QR factorization is then obtained by annihilating the entries below the diagonal one after the other. This is particularly advantageous if only a few entries below the diagonal are nonzero.

Motivation 4.3.1 (Givens rotation; rotation of a vector in $\mathbb{R}^2$)

The rotation of the plane about the origin by the angle φ (counterclockwise for $\varphi > 0$) maps e_1 to $(\cos\varphi, \sin\varphi)^T$ and e_2 to $(-\sin\varphi, \cos\varphi)^T$. With $c = \cos\varphi$ and $s = \sin\varphi$ it is therefore described by the matrix

$$\begin{pmatrix} c & -s \\ s & c \end{pmatrix}, \qquad c^2 + s^2 = 1.$$

(If one identifies $\mathbb{R}^2$ with $\mathbb{C}$, the rotation is just the multiplication by $e^{i\varphi} = c + is$: $(c+is)(x_1 + ix_2) = (cx_1 - sx_2) + i(sx_1 + cx_2)$.)

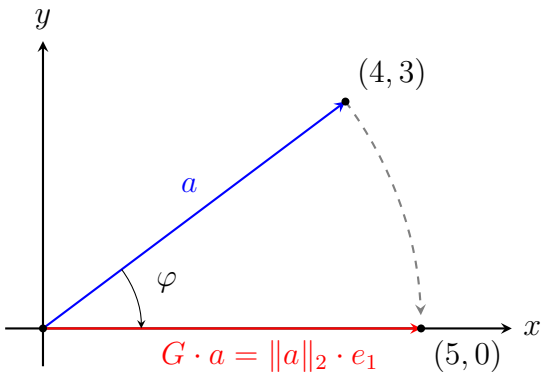


Figure 4.5: Rotation of $a = (4,3)^T$ onto the x-axis. The rotation is clockwise, i.e., $\varphi < 0$.

For a given $x = (x_1, x_2)^T \neq \mathbf{0}$ we look for c, s such that

$$\begin{pmatrix} c & -s \\ s & c \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} r \\ 0 \end{pmatrix}, \quad \text{i.e.} \quad x_1 c - x_2 s = r \quad \text{and} \quad x_1 s = -x_2 c.$$

Figure 4.5 illustrates this for $x = (4,3)^T$. Since a rotation preserves lengths, $|r| = \|x\|_2$ must hold. This also follows by direct computation:

$$r^2 = (x_1 c - x_2 s)^2 = x_1^2 c^2 - 2x_1 x_2 cs + x_2^2 s^2 \overset{x_1 s = -x_2 c}{=\!=\!=} x_1^2 c^2 + x_2^2 c^2 + x_1^2 s^2 + x_2^2 s^2$$

$$= (x_1^2 + x_2^2) \cdot (c^2 + s^2) \overset{c^2+s^2=1}{=\!=\!=} x_1^2 + x_2^2.$$

The sign of r can be chosen freely. We set

$$r = \operatorname{sign}(x_1) \cdot \sqrt{x_1^2 + x_2^2}. \tag{4.7}$$

For $x_1 \neq 0$ we obtain

$$x_1 c - x_2 s = r \xleftrightarrow{s=-x_2 c/x_1} x_1 c + \frac{x_2^2 c}{x_1} = r \iff c \cdot (x_1^2 + x_2^2) = x_1 r$$

$$\iff c = \frac{x_1 r}{x_1^2 + x_2^2} = \frac{x_1 r}{r^2} = \frac{x_1}{r}$$

and then

$$x_1 s = -x_2 c = -\frac{x_1 x_2}{r} \iff s = -\frac{x_2}{r}.$$

In general, the angle φ cannot be recovered from $\cos\varphi$ alone, since the cosine is bijective only on certain intervals. The sign choice in (4.7), however, restricts the angle implicitly: for $x_1 \neq 0$ it gives $c = x_1/r = |x_1|/\|x\|_2 > 0$ and hence $\varphi \in (-\pi/2, \pi/2)$. In the special case $x_1 = 0$ one sets $c = 0$ and $s = 1$, i.e., $\varphi = \pi/2$, and obtains $r = -x_2$. Altogether, $\varphi \in (-\pi/2, \pi/2]$ and $c \geq 0$.

Unlike the sign choice for Householder transformations, the choice in (4.7) does not serve to avoid cancellation: c and s are computed without subtractions for either sign. It ensures $c \geq 0$, which we will use for storing the rotations.

In $\mathbb{R}^n$ a rotation acts in the plane spanned by two unit vectors e_p and e_q and leaves all other components unchanged.

Definition 4.3.2 (Givens rotation)

Let $x \in \mathbb{R}^n$ and $1 \leq p < q \leq n$. Let $r = \operatorname{sign}(x_p) \cdot \sqrt{x_p^2 + x_q^2}$ if $x_p \neq 0$ and $r = -x_q$ if $x_p = 0$. With

$$c = \begin{cases} \frac{x_p}{r}, & x_p \neq 0, \\ 0, & x_p = 0, \end{cases} \qquad s = \begin{cases} -\frac{x_q}{r}, & x_p \neq 0, \\ 1, & x_p = 0, \end{cases}$$

the matrix

$$G(p, q, \varphi) = \begin{pmatrix} I_{p-1} & & & & \\ & c & & -s & \\ & & I_{q-p-1} & & \\ & s & & c & \\ & & & & I_{n-q} \end{pmatrix} \in \mathbb{R}^{n \times n}$$

is called a Givens rotation. It agrees with the identity matrix I_n except for the entries at the positions (p, p), (p, q), (q, p) and (q, q).

Lemma 4.3.3 (Givens rotation)

Let $G = G(p, q, \varphi)$ be the Givens rotation for $x \in \mathbb{R}^n$ from Definition 4.3.2. Then G is orthogonal, and

$$(G \cdot x)_i = \begin{cases} r, & i = p, \\ 0, & i = q, \\ x_i, & \textit{otherwise.} \end{cases}$$

Proof: In both cases of the definition $c^2+s^2 = 1$: for $x_p \neq 0$ we have $c^2+s^2 = (x_p^2+x_q^2)/r^2 = 1$, and for $x_p = 0$ we have $c^2 + s^2 = 0 + 1 = 1$. The matrix $G^T \cdot G$ agrees with I_n outside the rows and columns p and q. For the entries in these rows and columns, the product reduces to the 2×2 block

$$\begin{pmatrix} c & s \\ -s & c \end{pmatrix} \cdot \begin{pmatrix} c & -s \\ s & c \end{pmatrix} = \begin{pmatrix} c^2 + s^2 & -cs + sc \\ -sc + cs & s^2 + c^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Hence $G^T \cdot G = I_n$. For the product with x we have

$$(G \cdot x)_i = \begin{cases} cx_p - sx_q, & i = p, \\ sx_p + cx_q, & i = q, \\ x_i, & \text{otherwise.} \end{cases}$$

For $i = p$ this means

$$(G \cdot x)_p = cx_p - sx_q = \begin{cases} \frac{x_p}{r}x_p + \frac{x_q}{r}x_q = \frac{x_p^2+x_q^2}{r} = r, & x_p \neq 0, \\ -x_q, & x_p = 0, \end{cases}$$

and for $i = q$

$$(G \cdot x)_q = sx_p + cx_q = \begin{cases} -\frac{x_q}{r}x_p + \frac{x_p}{r}x_q = 0, & x_p \neq 0, \\ 1 \cdot 0 + 0 \cdot x_q = 0, & x_p = 0. \end{cases}$$

□

Remark 4.3.4 (Applying a Givens rotation)

A Givens rotation is never set up explicitly. For $y \in \mathbb{R}^n$ only the two components

$$\begin{pmatrix} (G \cdot y)_p \\ (G \cdot y)_q \end{pmatrix} = \begin{pmatrix} c & -s \\ s & c \end{pmatrix} \cdot \begin{pmatrix} y_p \\ y_q \end{pmatrix}$$

change. Once c and s are known, applying the rotation to y costs 6 floating-point operations (4 multiplications and 2 additions) instead of the $\mathcal{O}(n^2)$ operations of a matrix–vector product. Determining c and s (one square root and a few divisions) is not included in this count. Accordingly, $G \cdot A$ changes only the rows p and q of a matrix A. To determine c and s, only x_p and x_q are needed. In an implementation, $\sqrt{x_p^2 + x_q^2}$ should be computed in a way that avoids overflow, for example with MATLAB's `hypot`.

For the QR factorization of $A \in \mathbb{R}^{m\times n}$ with $m \geq n$, the entries below the diagonal are annihilated column by column. The entry $a_{i,j}$ with $i > j$ is annihilated with the Givens rotation $G(j, i, \varphi)$, which is determined from the current values of $a_{j,j}$ and $a_{i,j}$ (that is, $p = j$, $q = i$). This rotation combines the rows j and i. In the columns $1, \ldots, j-1$ both rows contain only zeros at this point, which therefore remain zero. Zeros produced earlier are thus not destroyed, and the rotation has to be applied only to the columns $j+1, \ldots, n$. As in the case of Householder transformations, A is overwritten: R is stored on and above the diagonal, and the value s of the rotation that annihilated $a_{i,j}$ is stored in place of $a_{i,j}$.

Algorithm 4: QR factorization with Givens rotations

Input: $A \in \mathbb{R}^{m\times n}$ with $m \geq n$
Output: A overwritten with R (on and above the diagonal) and the values s of the Givens rotations (below the diagonal)

1 **for** $j = 1, \ldots, n$ **do**
2 **for** $i = j+1, \ldots, m$ **do**
3 Determine c, s, r from $x_p = a_{j,j}$ and $x_q = a_{i,j}$ according to Definition 4.3.2
4 $a_{j,j} \leftarrow r$
5 **for** $l = j+1, \ldots, n$ **do**
6 $\begin{pmatrix} a_{j,l} \\ a_{i,l} \end{pmatrix} \leftarrow \begin{pmatrix} c & -s \\ s & c \end{pmatrix} \cdot \begin{pmatrix} a_{j,l} \\ a_{i,l} \end{pmatrix}$
7 $a_{i,j} \leftarrow s$

If $G(j, i)$ denotes the rotation used in the step (j, i) of Algorithm 4, then

$$Q^T = \big(G(n, m) \cdots G(n, n+1)\big) \cdots \big(G(2, m) \cdots G(2, 3)\big) \cdot \big(G(1, m) \cdots G(1, 2)\big) \tag{4.8}$$

satisfies $Q^T \cdot A = R$. The rotation applied first is on the far right; for $m = n$ the first group of factors is empty. As a product of orthogonal matrices, Q^T and hence Q are orthogonal, and $A = Q \cdot R$. If $\operatorname{rank}(A) = n$, then also $\operatorname{rank}(R) = n$, so that the upper $n \times n$ block R_1 of R is invertible. This gives a second constructive proof of Theorem 4.2.3.

Remark 4.3.5 (Storage of the Givens rotations and computation of $Q^T b$)

As with Householder transformations, Q is not set up explicitly. It suffices to know the rotations, and storing s in $a_{i,j}$ is enough: since $c \geq 0$ (see Motivation 4.3.1), c is uniquely determined by s,

$$c = \sqrt{1 - s^2}.$$

To solve a linear system or a linear least squares problem, one needs $Q^T \cdot b$ for $b \in \mathbb{R}^m$. Following (4.8), the rotations are applied to b in the same order as in Algorithm 4:

Algorithm 5: Computation of $Q^T b$ from the stored Givens rotations

Input: A as computed by Algorithm 4, $b \in \mathbb{R}^m$
Output: b overwritten with $Q^T \cdot b$

```
1 for j = 1, ..., n do
2     for i = j + 1, ..., m do
3         s ← a_{i,j},   c ← sqrt(1 − s^2)
4         (b_j, b_i)^T ← [c −s; s c] · (b_j, b_i)^T
```

Mathematically, this representation is exact. In floating-point arithmetic it can be inaccurate, see Remark 4.3.7.

Example 4.3.6 (QR factorization with Givens rotations)

(a) As in Example 4.2.7 we first consider $a = (4,3)^T$ (see Figure 4.5). We have $r = \operatorname{sign}(4) \cdot \sqrt{16+9} = 5$, hence $c = 4/5$ and $s = -3/5$, and

$$G \cdot a = \frac{1}{5} \cdot \begin{pmatrix} 4 & 3 \\ -3 & 4 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 5 \\ 0 \end{pmatrix}.$$

The Householder transformation in Example 4.2.7 reflects a onto $(-5,0)^T$, whereas the rotation turns it clockwise onto $(5,0)^T$.

(b) We compute the QR factorization of

$$A = \begin{pmatrix} 4 & 1 \\ 3 & 3 \\ 12 & 0 \end{pmatrix}$$

with Algorithm 4. We write down the matrices $G(j,i) \cdot A$ with the newly produced zeros; the algorithm stores the value s in their place.

Step $(j,i) = (1,2)$**:** With $x_p = 4$, $x_q = 3$ we get $r = 5$, $c = 4/5$, $s = -3/5$ as in (a). The rotation combines the rows 1 and 2; in the second column it yields

$$\begin{pmatrix} \frac{4}{5} & \frac{3}{5} \\ -\frac{3}{5} & \frac{4}{5} \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} \frac{13}{5} \\ \frac{9}{5} \end{pmatrix}, \qquad \text{so} \qquad G(1,2) \cdot A = \begin{pmatrix} 5 & \frac{13}{5} \\ 0 & \frac{9}{5} \\ 12 & 0 \end{pmatrix}.$$

Step $(j,i) = (1,3)$**:** Now $x_p = 5$ and $x_q = 12$ (the current entries $a_{1,1}$ and $a_{3,1}$). Hence $r = \sqrt{25+144} = 13$, $c = 5/13$, $s = -12/13$. The rotation combines the rows 1 and 3; in the second column it yields

$$\begin{pmatrix} \frac{5}{13} & \frac{12}{13} \\ -\frac{12}{13} & \frac{5}{13} \end{pmatrix} \cdot \begin{pmatrix} \frac{13}{5} \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ -\frac{12}{5} \end{pmatrix}, \qquad \text{so} \qquad G(1,3) \cdot G(1,2) \cdot A = \begin{pmatrix} 13 & 1 \\ 0 & \frac{9}{5} \\ 0 & -\frac{12}{5} \end{pmatrix}.$$

Step $(j,i) = (2,3)$**:** With $x_p = 9/5$ and $x_q = -12/5$ we get $r = \sqrt{81/25 + 144/25} = 3$, $c = 3/5$ and $s = 4/5$. There are no further columns to which the rotation would have to be applied ($l = 3 > n$). This time $s > 0$: the vector $(9/5, -12/5)^T$ lies below the x-axis and is rotated counterclockwise.

Altogether, the algorithm overwrites A with

$$\begin{pmatrix} 13 & 1 \\ -\frac{3}{5} & 3 \\ -\frac{12}{13} & \frac{4}{5} \end{pmatrix}, \qquad \text{i.e.} \qquad R = \begin{pmatrix} 13 & 1 \\ 0 & 3 \\ 0 & 0 \end{pmatrix}, \quad Q^T = G(2,3) \cdot G(1,3) \cdot G(1,2).$$

From the stored values, $c = \sqrt{1-s^2}$ is recovered as $4/5$, $5/13$ and $3/5$, respectively. As a check, $A = Q \cdot R$ with orthogonal Q implies $A^T \cdot A = R^T \cdot Q^T \cdot Q \cdot R = R^T \cdot R$, and indeed

$$A^T \cdot A = \begin{pmatrix} 16+9+144 & 4+9+0 \\ 4+9+0 & 1+9+0 \end{pmatrix} = \begin{pmatrix} 169 & 13 \\ 13 & 10 \end{pmatrix} = \begin{pmatrix} 13 & 0 \\ 1 & 3 \end{pmatrix} \cdot \begin{pmatrix} 13 & 1 \\ 0 & 3 \end{pmatrix} = R_1^T \cdot R_1.$$

Remark 4.3.7 (Stable storage of Givens rotations)

If $|s|$ is close to 1, i.e., if $|x_p| \ll |x_q|$, the reconstruction $c = \sqrt{1-s^2}$ in Algorithm 5 suffers from cancellation, as in Example 2.1.6 c). If the stored value is $\widetilde{s} = s(1+\delta)$ with $|\delta| \le u$, then

$$1 - \widetilde{s}^2 = 1 - s^2 - 2\delta s^2 + \mathcal{O}(\delta^2) = c^2 \cdot \left(1 - \frac{2s^2}{c^2} \cdot \delta\right) + \mathcal{O}(\delta^2).$$

The relative error is amplified by the factor $2s^2/c^2$, the relative condition number of $s \mapsto 1 - s^2$ (Definition 2.2.1). For $x_p = 10^{-9}$ and $x_q = 1$, for instance, $c \approx 10^{-9}$, but $\mathrm{fl}(s) = -1$ and the reconstruction yields $c = 0$.

The remedy is to store the one of c and s that is smaller in modulus, and to compute the other one from a number $\ge \frac{1}{2}$, where no cancellation occurs. Following Stewart's scheme as presented in [8], adapted to our convention $c \ge 0$, both values are encoded in a single number

$$\rho = \begin{cases} \operatorname{sign}(s), & c = 0, \\ s/2, & |s| < c, \\ 2\operatorname{sign}(s)/c, & \text{otherwise}, \end{cases} \qquad (c,s) = \begin{cases} (0,\rho), & |\rho| = 1, \\ \left(\sqrt{1-4\rho^2},\ 2\rho\right), & |\rho| < 1, \\ \left(\frac{2}{|\rho|},\ \operatorname{sign}(\rho)\sqrt{1-\frac{4}{\rho^2}}\right), & |\rho| > 1. \end{cases}$$

The three cases can be told apart, since $|s| < c$ implies $|\rho| = |s|/2 < 1/(2\sqrt{2})$, and otherwise $0 < c \le 1/\sqrt{2}$ gives $|\rho| \ge 2\sqrt{2}$. In both square roots the argument is $\ge \frac{1}{2}$. Since $c \ge 0$, only the sign of s has to be encoded, and decoding returns exactly the pair (c,s) used in Algorithm 4, not just $\pm(c,s)$. This matters because R was computed with this very rotation. In Algorithm 4, ρ is stored in $a_{i,j}$ instead of s. In Algorithm 5, c and s are obtained by decoding. In Example 4.3.6 one would store $\rho = -3/10$, $-26/5$ and $10/3$ instead of $s = -3/5$, $-12/13$ and $4/5$.

Remark 4.3.8 (Givens or Householder?)

For a general dense matrix, a short count shows that Algorithm 4 needs about

$$6 \cdot \sum_{j=1}^{n} (m-j)(n-j) \approx 3n^2 \left(m - \frac{n}{3}\right)$$

floating-point operations (6 for each pair of entries that is rotated; determining c and s adds only $\mathcal{O}(mn)$). This is about 1.5 times the cost of the QR factorization with Householder transformations, which needs about $2n^2(m - n/3)$ operations [8]. Givens rotations pay off if only a few entries below the diagonal have to be annihilated. An example are upper Hessenberg matrices, i.e., matrices with $a_{i,j} = 0$ for $i > j + 1$:

$$\begin{pmatrix} * & * & \cdots & * \\ * & * & \cdots & * \\ & \ddots & \ddots & \vdots \\ & & * & * \end{pmatrix}.$$

For $A \in \mathbb{R}^{n \times n}$ of this form, only the $n - 1$ rotations $G(j, j + 1)$ are needed, at a cost of $\mathcal{O}(n^2)$ instead of $\mathcal{O}(n^3)$ operations. In Algorithm 4 one then skips every rotation for an entry $a_{i,j}$ that is already zero and stores $s = 0$ (i.e., $c = 1$, $G = I$). For $a_{j,j} \neq 0$ this is exactly what Definition 4.3.2 yields for $x_q = 0$ anyway.

§4 Singular Value Decomposition

Theorem 4.4.1 (Singular value decomposition)

Let $A \in \mathbb{R}^{m \times n}$. Then there exist orthogonal matrices $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ such that

$$A = U \cdot \Sigma \cdot V^T,$$

where $\Sigma \in \mathbb{R}^{m \times n}$ is a diagonal matrix:

$$\Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_p),$$

with $p = \min\{m, n\}$ and $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$.

Proof:
W.l.o.g. let $m \geq n$; then

$$\Sigma = \begin{pmatrix} \widehat{\Sigma} \\ \mathbf{0} \end{pmatrix} \quad \text{with} \quad \widehat{\Sigma} = \begin{pmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{pmatrix} \in \mathbb{R}^{n \times n}$$

In the case $m < n$ one considers the singular value decomposition of $A^T = U \cdot \Sigma \cdot V^T$ and obtains $A = V \cdot \Sigma^T \cdot U^T$.

Base case: $n = 1$
In the case $A = \mathbf{0}$ the claim follows with $\Sigma = \mathbf{0}$ and arbitrary orthogonal matrices U and V.

In the case $A = (a_1) \neq \mathbf{0}$ let

$$\sigma_1 = \|a_1\|_2 = \sqrt{a_1^T \cdot a_1} \text{ and } u_1 = \frac{1}{\sigma_1} \cdot a_1.$$

Because of $u_1^T \cdot u_1 = \frac{1}{\sigma_1^2} \cdot a_1^T \cdot a_1 = 1$ we have $\|u_1\|_2 = 1$. If we extend u_1 to an orthogonal matrix U, then with

$$U = (u_1, \widetilde{U}) \in \mathbb{R}^{m \times m},\ V = (1) \in \mathbb{R}^{1 \times 1} \quad \text{and} \quad \Sigma = \operatorname{diag}(\sigma_1) \in \mathbb{R}^{m \times 1}$$

we obtain the desired decomposition, since

$$U \cdot \Sigma \cdot V^T = u_1 \cdot \sigma_1 = a_1 = A.$$

Induction step: $n - 1 \to n$
Here one actually proceeds almost analogously to the base case. In the case $A = \mathbf{0}$, the claim is satisfied with $\Sigma = \mathbf{0}$ and arbitrary orthogonal matrices U and V. In the case $A \neq \mathbf{0}$ only the construction of V has to be adapted. First one defines

$$\sigma_1 := \|A\|_2 = \max_{x \neq \mathbf{0}} \frac{\|A \cdot x\|_2}{\|x\|_2} > 0.$$

Since the maximum is attained, there exists a $v_1 \in \mathbb{R}^n$ with $\|v_1\|_2 = 1$ and $\|A \cdot v_1\|_2 = \sigma_1$. This vector can be extended to an orthogonal matrix V such that:

$$V = (v_1, \widetilde{V}) \in \mathbb{R}^{n \times n} \quad \text{with} \quad V^T \cdot V = I_n \quad \text{and} \quad \sigma_1 = \|A \cdot v_1\| > 0$$

Now one defines

$$u_1 = \frac{A \cdot v_1}{\|A \cdot v_1\|_2}$$

and extends u_1 to an orthogonal matrix $U = (u_1, \widetilde{U}) \in \mathbb{R}^{m \times m}$. Then it follows that

$$U^T \cdot A \cdot V = \begin{pmatrix} u_1^T \\ \widetilde{U}^T \end{pmatrix} \cdot A \cdot \begin{pmatrix} v_1 & \widetilde{V} \end{pmatrix} = \begin{pmatrix} u_1^T \cdot A \cdot v_1 & u_1^T \cdot A \cdot \widetilde{V} \\ \widetilde{U}^T \cdot A \cdot v_1 & \widetilde{U}^T \cdot A \cdot \widetilde{V} \end{pmatrix}$$

Here

$$u_1^T \cdot A \cdot v_1 = \frac{(A \cdot v_1)^T \cdot (A \cdot v_1)}{\|A \cdot v_1\|_2} = \|A \cdot v_1\|_2 = \sigma_1$$

and

$$\widetilde{U}^T \cdot A \cdot v_1 = \widetilde{U}^T \cdot u_1 \cdot \|Av_1\|_2 = \mathbf{0}.$$

That, in addition, $u_1^T \cdot A \cdot \widetilde{V} = \mathbf{0}^T$ holds follows from

$$\begin{aligned}
\sigma_1^2 &= \|A\|_2^2 = \|U^T \cdot A \cdot V\|_2^2 = \|V^T \cdot A^T \cdot U\|_2^2 \\
&\geq \|V^T \cdot A^T \cdot U \cdot e_1\|_2^2 = \|V^T \cdot A^T \cdot u_1\|_2^2 \\
&= \left\| \begin{pmatrix} v_1^T \cdot A^T \cdot u_1 \\ \widetilde{V}^T \cdot A^T \cdot u_1 \end{pmatrix} \right\|_2^2 = \|v_1^T \cdot A^T \cdot u_1\|_2^2 + \|\widetilde{V}^T \cdot A^T \cdot u_1\|_2^2 \\
&= \|u_1^T \cdot A \cdot v_1\|_2^2 + \|u_1^T \cdot A \cdot \widetilde{V}\|_2^2 \\
&= \sigma_1^2 + \|u_1^T \cdot A \cdot \widetilde{V}\|_2^2
\end{aligned}$$

Hence

$$U^T \cdot A \cdot V = \begin{pmatrix} \sigma_1 & \\ & \widetilde{U}^T \cdot A \cdot \widetilde{V} \end{pmatrix} \tag{4.9}$$

By the induction hypothesis, the remaining matrix has the singular value decomposition

$$\widetilde{U}^T \cdot A \cdot \widetilde{V} = U_1 \cdot \Sigma_1 \cdot V_1^T$$

with

$$\Sigma_1 = \operatorname{diag}(\sigma_2, \ldots, \sigma_n) \quad \text{and} \quad \sigma_2 \geq \cdots \geq \sigma_n.$$

Using this decomposition in (4.9), one obtains

$$U^T \cdot A \cdot V = \begin{pmatrix} \sigma_1 & \\ & U_1 \cdot \Sigma_1 \cdot V_1^T \end{pmatrix} = \begin{pmatrix} 1 & \\ & U_1 \end{pmatrix} \begin{pmatrix} \sigma_1 & \\ & \Sigma_1 \end{pmatrix} \begin{pmatrix} 1 & \\ & V_1^T \end{pmatrix}$$

This gives the singular value decomposition of A:

$$A = \left[U \cdot \begin{pmatrix} 1 & \\ & U_1 \end{pmatrix} \right] \cdot \begin{pmatrix} \sigma_1 & \\ & \Sigma_1 \end{pmatrix} \cdot \left[\begin{pmatrix} 1 & \\ & V_1^T \end{pmatrix} \cdot V^T \right]$$

That $\sigma_1 \geq \sigma_2$ holds follows from

$$\sigma_1 = \|A\|_2 = \left\| \begin{pmatrix} \sigma_1 & \\ & \Sigma_1 \end{pmatrix} \right\|_2 = \max_{k=1,\ldots,n} \sqrt{\sigma_k^2} \overset{\sigma_k \geq 0}{=\!=} \max_{k=1,\ldots,n} \sigma_k$$

□

Corollary 4.4.2 *With the notation of Theorem 4.4.1 we have:*

(a) $\|A\|_2 = \sigma_1$, *i.e., the 2-norm of A equals the largest singular value.*

(b) If $m = n$ and A is invertible, then $\|A^{-1}\|_2 = \frac{1}{\sigma_n}$. From this it follows that $\kappa_2(A) = \frac{\sigma_1}{\sigma_n}$ is the condition number of A with respect to the 2-norm.

(c) The symmetric matrix $A^T A \in \mathbb{R}^{n \times n}$ has the eigenvalues $\sigma_1^2, \ldots, \sigma_p^2$. If $m < n$, $n - m$ additional eigenvalues 0 are added.

(d) For $m \geq n$ and $\operatorname{rank}(A) = n$,

$$x_0 = V \cdot \Sigma^+ \cdot U^T \cdot b$$

with

$$\Sigma^+ = \begin{pmatrix} \widehat{\Sigma}^{-1} & \mathbf{0} \end{pmatrix} = \operatorname{diag}\left(\frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_n}, 0, \ldots, 0 \right) \in \mathbb{R}^{n \times m}$$

is the solution of the linear least squares problem $\min\limits_{x \in \mathbb{R}^n} \|Ax - b\|_2$.

Definition 4.4.3 (Pseudoinverse)

Let $A \in \mathbb{R}^{m \times n}$ *with* $m \geq n$ *and* $\operatorname{rank}(A) = n$*, and let*

$$A = U \cdot \Sigma \cdot V^T,$$

be a singular value decomposition of A *with the notation of Theorem 4.4.1. Then*

$$A^+ = V \cdot \Sigma^+ \cdot U^T$$

with

$$\Sigma^+ = \begin{pmatrix} \widehat{\Sigma}^{-1} & \mathbf{0} \end{pmatrix} = \operatorname{diag}\left(\frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_n}, 0, \ldots, 0\right) \in \mathbb{R}^{n \times m}$$

is called the pseudoinverse of A*.*

Chapter 5 Numerical Computation of Eigenvalues and Eigenvectors

At first glance, eigenvalues could be determined by finding the roots of the characteristic polynomial. Numerically, however, this makes no sense, since the explicit computation of the characteristic polynomial is inefficient for larger matrices and susceptible to rounding errors. Moreover, the computational cost and the instability grow exponentially with increasing matrix size.

Instead, iterative methods are used to determine eigenvalues. These methods, however, often require starting values or information about the location of the eigenvalues. Therefore, we begin this chapter with methods for estimating the geometric location of the eigenvalues.

Sources and further reading. The inclusion theorems of §1 go back to Gershgorin [6] and Bendixson [2]. The QR algorithm of §4 was introduced independently by Francis [5] and Kublanovskaya [15]; for Wilkinson's analysis of its convergence see [19]. Textbook treatments of the methods in this chapter can be found, for example, in [4], [18] and [8].

§1 Estimating the Location of Eigenvalues

Since real matrices can also have complex eigenvalues, it makes sense to consider the more general case of complex matrices right away. To analyze the location of the eigenvalues, we begin with the definition of the spectrum of a matrix.

Definition 5.1.1 (Spectrum of a matrix)
The spectrum of a matrix $A \in \mathbb{C}^{n\times n}$ is the set

$$\sigma(A) = \{\lambda \in \mathbb{C} : \det(A - \lambda \cdot I) = 0\},$$

consisting of all eigenvalues of A.

Having defined the spectrum of a matrix, the following theorem provides a simple way to estimate the eigenvalues of a matrix by its matrix norm.

Theorem 5.1.2 (Estimate by a matrix norm)

Let $\|\cdot\|$ be a matrix norm compatible with a vector norm and let λ be an eigenvalue of A. Then $|\lambda| \leq \|A\|$.

Proof: Let λ be an eigenvalue of A and $v \neq \mathbf{0}$ an associated eigenvector. Due to the compatibility of the matrix norm with the vector norm, we have

$$|\lambda|\|v\| = \|\lambda v\| = \|Av\| \leq \|A\|\|v\|.$$

This implies $|\lambda| \leq \|A\|$. ∎

In many cases this estimate is very rough. Moreover, computing the matrix norm can be extremely expensive. In particular, determining $\|A\|_2$ requires the computation of the eigenvalues of $A^T A$. Therefore, one looks for alternative estimates.

Theorem 5.1.3 ((Row-oriented) Gershgorin discs)
Let $A \in \mathbb{C}^{n\times n}$. Then

$$\sigma(A) \subseteq \mathcal{S}_{\mathcal{R}} := \bigcup_{i=1}^{n} \mathcal{R}_i, \quad \textit{where} \quad \mathcal{R}_i := \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j\neq i} |a_{ij}| \right\}.$$

Proof: Let $\lambda \in \sigma(A)$ and $Ax = \lambda x$ with $x \neq 0$. Then there exists an index i such that $|x_i| \geq |x_j|$ for all j. The i-th row of the equation yields

$$\lambda x_i = \sum_{j=1}^{n} a_{ij} x_j = a_{ii} x_i + \sum_{j\neq i} a_{ij} x_j.$$

Dividing by x_i and applying the triangle inequality, we obtain

$$|\lambda - a_{ii}| \leq \sum_{j\neq i} |a_{ij}|.$$

∎

Since $\sigma(A) = \sigma(A^T)$, it follows immediately:

Corollary 5.1.4 ((Column-oriented) Gershgorin discs)

Let $A \in \mathbb{C}^{n\times n}$. Then

$$\sigma(A) \subseteq \mathcal{S}_{\mathcal{C}} := \bigcup_{j=1}^{n} \mathcal{C}_j, \quad \textit{where} \quad \mathcal{C}_j := \left\{ z \in \mathbb{C} : |z - a_{jj}| \leq \sum_{i\neq j} |a_{ij}| \right\}.$$

From Theorem 5.1.3 and Corollary 5.1.4 we in turn obtain the first Gershgorin theorem:

Theorem 5.1.5 (First Gershgorin theorem)

Let $A \in \mathbb{C}^{n\times n}$. Then for all eigenvalues $\lambda \in \sigma(A)$

$$\lambda \in \mathcal{S}_{\mathcal{R}} \cap \mathcal{S}_{\mathcal{C}}$$

With the second Gershgorin theorem, the location of the eigenvalues can be analyzed even more precisely.

Theorem 5.1.6 (Second Gershgorin theorem)
Let $A \in \mathbb{C}^{n\times n}$, let M_1 be the union of k of the (row-oriented) Gershgorin discs $\mathcal{R}_i$ and M_2 the union of the remaining ones. If M_1 and M_2 are disjoint, then M_1 contains exactly k eigenvalues, counted with their algebraic multiplicity. (Applied to A^T, the same holds for the column-oriented discs $\mathcal{C}_j$.)

Proof:
We define $D := \operatorname{diag}(a_{11}, \ldots, a_{nn})$ and consider the family of matrices parametrized by $t \in \mathbb{R}$

$$A(t) := D + t \cdot (A - D), \quad 0 \leq t \leq 1.$$

We have $A(0) = D$ and $A(1) = A$. For $t = 0$ the statement is obviously true because of Theorem 5.1.3. Since the eigenvalues of $A(t)$ coincide with the roots of the characteristic polynomial of $A(t)$, and these are continuous functions of the coefficients of the polynomial, the eigenvalues of $A(t)$ depend continuously on the parameter t, i.e. they can be numbered as continuous functions $\lambda_1(t), \ldots, \lambda_n(t)$ on $[0, 1]$. Moreover, with $r_i := \sum_{j\neq i} |a_{ij}|$, the Gershgorin discs of $A(t)$ have the centers a_{ii} and the radii $t \cdot r_i \leq r_i$, so they are contained in the corresponding discs $\mathcal{R}_i$ of A. By Theorem 5.1.3, $\lambda_l(t) \in M_1 \cup M_2$ for all $t \in [0, 1]$ and all l. For $t = 0$, exactly k of the eigenvalues $\lambda_l(0) = a_{ll}$ (counted with multiplicity) lie in M_1, namely the centers of the discs forming M_1. Since M_1 and M_2 are disjoint and closed, a continuous path $t \mapsto \lambda_l(t)$ in $M_1 \cup M_2$ cannot jump from M_1 to M_2 or vice versa (the interval $[0, 1]$ is connected). The statement therefore follows by letting t run from 0 to 1. ☐

In order to formulate the third Gershgorin theorem, we need a property of matrices that is introduced in the following definition. The proof of this theorem as well as further Gershgorin-type estimates can be found in [11], §6.2; they are based on methods from complex analysis. These, however, would lead us too far afield at this point.

Definition 5.1.7 (Reducible matrix)

A matrix $A \in \mathbb{C}^{n\times n}$ is called reducible *if there exist a permutation matrix P and square matrices B_{11}, B_{22} such that*

$$PAP^T = \begin{pmatrix} B_{11} & B_{12} \\ 0 & B_{22} \end{pmatrix}.$$

If A is not reducible, it is called irreducible.

Remark 5.1.8
A matrix $A \in \mathbb{C}^{n\times n}$ is irreducible if and only if the associated directed graph is strongly connected. This means that for every pair of nodes (i, j) in the graph there is a directed path from i to j.

Let us consider the matrix

$$A = \begin{pmatrix} 4 & 0 & -3 \\ 0 & -1 & 1 \\ -1 & 1 & -2 \end{pmatrix}.$$

The associated graph is shown in Figure 5.1. For every pair of nodes there exists a directed path:

- From node 1 to node 2 via the path $1 \to 3 \to 2$.
- From node 2 to node 1 via the path $2 \to 3 \to 1$.
- Between all other pairs of nodes there are directed paths as well, either directly or via intermediate nodes.

Since the graph is strongly connected, the matrix A is irreducible by definition.

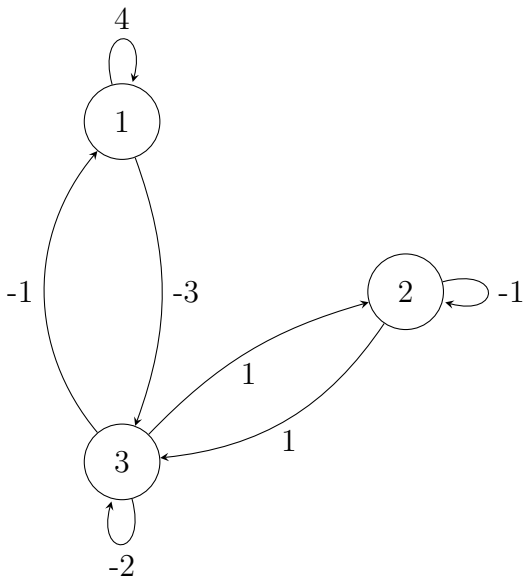


Figure 5.1: The weighted directed graph of the matrix A.

Theorem 5.1.9 (Third Gershgorin theorem)

Let $A \in \mathbb{C}^{n\times n}$ be irreducible. If an eigenvalue $\lambda \in \sigma(A)$ lies on the boundary of the union $\mathcal{S}_\mathcal{R}$ of the Gershgorin discs, then it lies on the boundary of every Gershgorin disc $\mathcal{R}_i$, $i = 1, \ldots, n$. (Since A^T is irreducible as well, the same holds for the column-oriented discs $\mathcal{C}_j$ and their union $\mathcal{S}_\mathcal{C}$.)

Example 5.1.10 Let us consider the irreducible matrix

$$A = \begin{pmatrix} 4 & 0 & -3 \\ 0 & -1 & 1 \\ -1 & 1 & -2 \end{pmatrix}.$$

with eigenvalues

$$\lambda_1 \approx 4.4766,\ \lambda_2 \approx -2.9458,\ \lambda_3 \approx -0.5308$$

The row-oriented Gershgorin discs of this matrix are given by

$$\begin{aligned} \mathcal{R}_1 &= \{z \in \mathbb{C} : |z-4| \leq 3\}, \\ \mathcal{R}_2 &= \{z \in \mathbb{C} : |z+1| \leq 1\}, \\ \mathcal{R}_3 &= \{z \in \mathbb{C} : |z+2| \leq 2\}. \end{aligned}$$

The column-oriented Gershgorin discs of this matrix are given by

$$\begin{aligned} \mathcal{C}_1 &= \{z \in \mathbb{C} : |z-4| \leq 1\}, \\ \mathcal{C}_2 &= \{z \in \mathbb{C} : |z+1| \leq 1\}, \\ \mathcal{C}_3 &= \{z \in \mathbb{C} : |z+2| \leq 4\}. \end{aligned}$$

By the first Gershgorin theorem (Theorem 5.1.5), the eigenvalues of the matrix lie within

$$(\mathcal{R}_1 \cup \mathcal{R}_2 \cup \mathcal{R}_3) \cap (\mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3)$$

This is illustrated in Figure 5.2.

Together with the second Gershgorin theorem (Theorem 5.1.6) it furthermore follows that two eigenvalues lie in $\mathcal{R}_2 \cup \mathcal{R}_3 = \mathcal{R}_3$ and one eigenvalue lies in $\mathcal{C}_1$.

The Gershgorin discs $\mathcal{R}_1$, $\mathcal{R}_2$ and $\mathcal{R}_3$ have no common boundary point, and neither do the Gershgorin discs $\mathcal{C}_1$, $\mathcal{C}_2$ and $\mathcal{C}_3$. Since A is an irreducible matrix, it therefore follows from the third Gershgorin theorem (Theorem 5.1.9) that no eigenvalue lies on the boundary of $\mathcal{S}_\mathcal{R}$ or of $\mathcal{S}_\mathcal{C}$. Hence the eigenvalues lie in the interior of $(\mathcal{R}_1 \cup \mathcal{R}_2 \cup \mathcal{R}_3) \cap (\mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3)$.

Another way to estimate the spectrum of a matrix is provided by Bendixson's theorem. In order to formulate it, we first define the Rayleigh quotient and the numerical range of a matrix.

Definition 5.1.11 (Numerical range of a matrix)
Let $A \in \mathbb{K}^{n\times n}$, where $\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$. The set of all Rayleigh quotients

$$\varrho(A; x) := \frac{x^H A x}{x^H x}, \quad x \in \mathbb{C}^n \setminus \{\mathbf{0}\},$$

is called the numerical range *(also called the field of values) of the matrix A:*

$$W(A) := \{\varrho(A; x) : x \in \mathbb{C}^n \setminus \{\mathbf{0}\}\}.$$

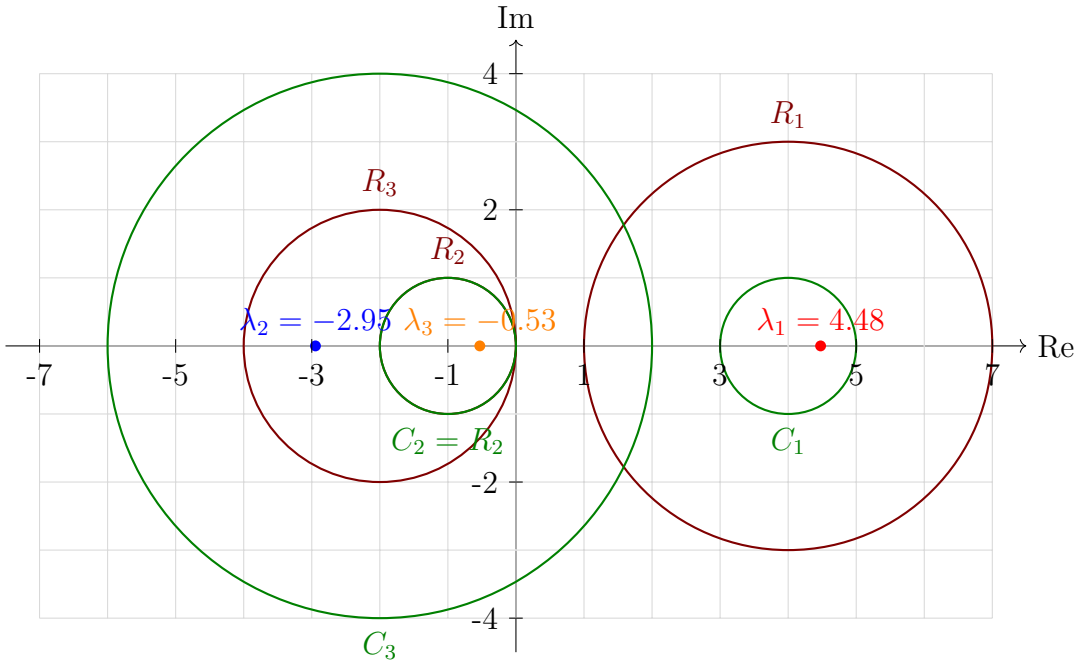


Figure 5.2: Estimate by Gershgorin discs.

Remark 5.1.12

(i) Even for a real matrix $A \in \mathbb{R}^{n\times n}$, all complex vectors $x \in \mathbb{C}^n \setminus \{0\}$ are taken into account in the numerical range.

(ii) The numerical range contains the spectrum: $\sigma(A) \subseteq W(A)$. If $x \in \mathbb{C}^n \setminus \{0\}$ is an eigenvector of A, then the associated Rayleigh quotient $\varrho(A; x)$ is exactly the eigenvalue belonging to x.

Lemma 5.1.13 (Properties of the numerical range)
Let $A \in \mathbb{K}^{n\times n}$, where $\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$. Then:

(i) $W(A)$ is connected.

(ii) If A is Hermitian, then all eigenvalues are real, and

$$W(A) = [\lambda_{\min}(A), \lambda_{\max}(A)] \subset \mathbb{R}.$$

(iii) If A is skew-Hermitian (i.e. $A = -A^H$), then all eigenvalues are purely imaginary, and $W(A)$ is a purely imaginary set, namely the convex hull of the eigenvalues of A.

Theorem 5.1.14 (Bendixson)
Let $A \in \mathbb{K}^{n\times n}$, where $\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$. Define $R := W\left(\frac{A+A^H}{2}\right) + W\left(\frac{A-A^H}{2}\right)$ as a "rectangle", where $A + B := \{a + b : a \in A, b \in B\}$. Then

$$W(A) \subseteq R.$$

Proof:
Let $x \in \mathbb{C}^n \setminus \{0\}$. Then:

$$\frac{x^H A x}{x^H x} = \frac{1}{x^H x} x^H \cdot \left(\frac{A + A^H}{2} + \frac{A - A^H}{2} \right) x.$$

The right-hand side can be split into:

$$\frac{1}{x^H x} x^H \cdot \left(\frac{A + A^H}{2} \right) x + \frac{1}{x^H x} x^H \cdot \left(\frac{A - A^H}{2} \right) x.$$

The two terms lie in $W\left(\frac{A+A^H}{2}\right)$ and $W\left(\frac{A-A^H}{2}\right)$, respectively. Hence the entire expression lies in R. ∎

Remark 5.1.15
Since $\frac{1}{2}(A + A^H)$ is Hermitian and $\frac{1}{2}(A - A^H)$ is skew-Hermitian, Bendixson's theorem states that for every $\lambda \in \sigma(A)$ the following holds:

$$\lambda_{\min}\left(\frac{1}{2}(A + A^H)\right) \leq \mathrm{Re}(\lambda) \leq \lambda_{\max}\left(\frac{1}{2}(A + A^H)\right),$$
$$\min\{\mathrm{Im}(\mu) : \mu \in \sigma\left(\frac{1}{2}(A - A^H)\right)\} \leq \mathrm{Im}(\lambda) \leq \max\{\mathrm{Im}(\mu) : \mu \in \sigma\left(\frac{1}{2}(A - A^H)\right)\}.$$

Example 5.1.16 Let us consider again the matrix

$$A = \begin{pmatrix} 4 & 0 & -3 \\ 0 & -1 & 1 \\ -1 & 1 & -2 \end{pmatrix}.$$

from Example 5.1.10 with eigenvalues

$$\lambda_1 \approx 4.4766,\ \lambda_2 \approx -2.9458,\ \lambda_3 \approx -0.5308$$

In order to apply Theorem 5.1.14, we compute

$$\frac{A^H + A}{2} = \begin{pmatrix} 4 & 0 & -2 \\ 0 & -1 & 1 \\ -2 & 1 & -2 \end{pmatrix} \quad \text{and} \quad \frac{A - A^H}{2} = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

With Theorem 5.1.3 one can estimate each of the numerical ranges and obtains

$$-5 \leq \mathrm{Re}(\lambda) \leq 6 \quad \text{and} \quad -1 \leq \mathrm{Im}(\lambda) \leq 1$$

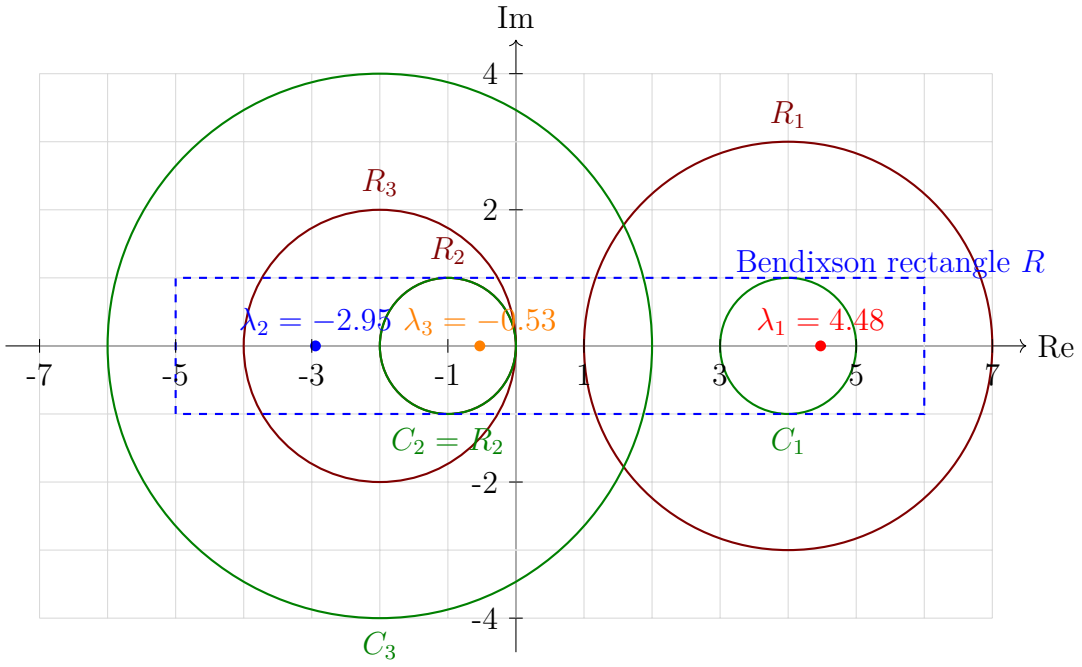


Figure 5.3: Estimate by Gershgorin discs and the Bendixson rectangle.

§2 The Power Method

The power method, also called power iteration or vector iteration, is a very simple yet effective method for determining the eigenvalue of largest modulus. To understand the basic idea of the method, let us first assume that the matrix $A \in \mathbb{C}^{n\times n}$ is diagonalizable. Then there is a basis $\{v_1, \ldots, v_n\}$ of $\mathbb{C}^n$ consisting of eigenvectors v_i of A. Furthermore, let the eigenvalues of A be ordered in the form

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|,$$

where λ_1 has algebraic multiplicity 1. If we now start from an initial vector $x^{(0)}$, it can be written as a linear combination of the eigenvectors v_i:

$$x^{(0)} = \sum_{i=1}^{n} \alpha_i v_i.$$

If we now define the iterates $a^{(k)}, k \in \mathbb{N}$, by $a^{(k)} := A^k x^{(0)}$, we obtain

$$a^{(k)} = A^k x^{(0)} = A^k \left(\sum_{i=1}^{n} \alpha_i v_i \right) = \sum_{i=1}^{n} \alpha_i \lambda_i^k v_i = \lambda_1^k \sum_{i=1}^{n} \alpha_i \left(\frac{\lambda_i}{\lambda_1} \right)^k v_i.$$

If the coefficient α_1 is nonzero (if the eigenvectors v_i are orthonormal, e.g. for Hermitian A, this means that $x^{(0)}$ is not orthogonal to v_1), this expression will approach the term with the dominant eigenvalue λ_1, i.e.

$$a^{(k)} = A^k x^{(0)} \approx \lambda_1^k \alpha_1 v_1.$$

In order to avoid overflow or underflow in practice, the iteration vector is normalized in each iteration step. Altogether, we obtain the following algorithm for the power method.

Algorithm 6: Power method

Input: $A \in \mathbb{C}^{n\times n}$

Input: $x^{(0)} \in \mathbb{C}^n$ with $\|x^{(0)}\|_2 = 1$ and $\alpha_1 \neq 0$ (coefficient of v_1 in the eigenvector expansion of $x^{(0)}$)

Input: tol > 0

Output: An approximation of the eigenvalue λ_1 of largest modulus and of the associated eigenvector

1 **for** $k = 0, 1, 2, \ldots$ **do**

2 $\quad a^{(k)} \leftarrow Ax^{(k)}$

3 $\quad \rho^{(k)} \leftarrow \left(x^{(k)}\right)^H a^{(k)}$

4 $\quad x^{(k+1)} \leftarrow \frac{a^{(k)}}{\|a^{(k)}\|_2}$

Algorithm 6 generates the sequence $\left(\rho^{(k)}\right)_k$ with

$$\rho^{(k)} = \left(x^{(k)}\right)^H \cdot A \cdot x^{(k)}$$

and the sequences $\left(a^{(k)}\right)_k$, $\left(x^{(k)}\right)_k$ with

$$\begin{aligned}
a^{(0)} &= A \cdot x^{(0)} \\
x^{(1)} &= \frac{a^{(0)}}{\|a^{(0)}\|_2} = \frac{A \cdot x^{(0)}}{\|A \cdot x^{(0)}\|_2} \\
a^{(1)} &= A \cdot x^{(1)} = \frac{A^2 \cdot x^{(0)}}{\|A \cdot x^{(0)}\|_2} \\
x^{(2)} &= \frac{a^{(1)}}{\|a^{(1)}\|_2} = \frac{A^2 \cdot x^{(0)}}{\|A \cdot x^{(0)}\|_2} \cdot \frac{\|A \cdot x^{(0)}\|_2}{\|A^2 \cdot x^{(0)}\|} = \frac{A^2 \cdot x^{(0)}}{\|A^2 \cdot x^{(0)}\|} \\
&\vdots \\
a^{(k)} &= A \cdot x^{(k)} = \frac{A^{k+1} \cdot x^{(0)}}{\|A^k \cdot x^{(0)}\|_2} \\
x^{(k+1)} &= \frac{A^{k+1} \cdot x^{(0)}}{\|A^k \cdot x^{(0)}\|_2} \cdot \frac{\|A^k \cdot x^{(0)}\|_2}{\|A^{k+1} \cdot x^{(0)}\|_2} = \frac{A^{k+1} \cdot x^{(0)}}{\|A^{k+1} \cdot x^{(0)}\|_2}
\end{aligned}$$

Theorem 5.2.1

Let $A \in \mathbb{C}^{n\times n}$ be a diagonalizable matrix with eigenvalues of the form

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$$

Let $\{v_1, \ldots, v_n\}$ be a basis of $\mathbb{C}^n$ consisting of eigenvectors with $Av_i = \lambda_i v_i$ and $\|v_i\|_2 = 1$, and let $x^{(0)} = \sum_{i=1}^n \alpha_i v_i \in \mathbb{C}^n$ be a vector with $\alpha_1 \neq 0$ and $\|x^{(0)}\|_2 = 1$. (If A is normal, the v_i can be chosen orthonormal; then $\alpha_1 \neq 0$ means that $x^{(0)}$ is not orthogonal to the eigenspace of λ_1.) If we define

$$x^{(k+1)} := \frac{a^{(k)}}{\|a^{(k)}\|_2}, \quad \text{with} \quad a^{(k)} := Ax^{(k)},$$

then

$$\varrho(A; x^{(k)}) := \frac{\left(x^{(k)}\right)^H \cdot A \cdot x^{(k)}}{\left(x^{(k)}\right)^H \cdot x^{(k)}} = \left(x^{(k)}\right)^H \cdot a^{(k)} = \rho^{(k)}$$

converges to λ_1.

Proof:
Let $\{v_1, \ldots, v_n\}$ and $\alpha_1, \ldots, \alpha_n$ be as in the hypothesis. Then

$$A^k x^{(0)} = \sum_{i=1}^{n} \alpha_i \lambda_i^k v_i,$$

with $\alpha_1 \neq 0$. This expression can be rewritten as:

$$A^k x^{(0)} = \alpha_1 \lambda_1^k \underbrace{\left(v_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k v_i \right)}_{=:z^{(k)}}.$$

Since $|\lambda_i/\lambda_1| < 1$ for all $i \geq 2$, the expression in parentheses converges to v_1 as $k \to \infty$. This means:

$$\lim_{k\to\infty} z^{(k)} = v_1 \quad \Longrightarrow \quad \lim_{k\to\infty} \left\| z^{(k)} \right\|_2 = \|v_1\|_2 = 1 \tag{5.1}$$

and, since the left factor in the second line below converges to v_1^H and the right factor to $\lambda_1 v_1$,

$$\begin{aligned}
&\left(z^{(k)}\right)^H \cdot A \cdot z^{(k)} \\
&= \left(v_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k v_i \right)^H \cdot A \cdot \left(v_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k v_i \right) \\
&= \left(v_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k v_i \right)^H \cdot \left(\lambda_1 \cdot v_1 + \sum_{i=2}^{n} \frac{\alpha_i}{\alpha_1} \lambda_1 \cdot \left(\frac{\lambda_i}{\lambda_1} \right)^{k+1} v_i \right) \\
&\to v_1^H \cdot (\lambda_1 \cdot v_1) = \lambda_1 \|v_1\|_2^2 = \lambda_1 \quad (k \to \infty)
\end{aligned} \tag{5.2}$$

Moreover,

$$x^{(k)} = \frac{A^k \cdot x^{(0)}}{\|A^k \cdot x^{(0)}\|_2} = \frac{\alpha_1 \lambda_1^k}{|\alpha_1 \lambda_1^k|} \cdot \frac{z^{(k)}}{\|z^{(k)}\|_2} = c_k \cdot \frac{z^{(k)}}{\|z^{(k)}\|_2} \quad \text{with} \quad c_k = \frac{\alpha_1 \lambda_1^k}{|\alpha_1 \lambda_1^k|} \tag{5.3}$$

and

$$\varrho\left(A; x^{(k)}\right) \overset{\|x^{(k)}\|_2=1}{=\!=\!=} \left(x^{(k)}\right)^H \cdot A \cdot x^{(k)} \overset{|c_k|=1}{=\!=\!=} \frac{\left(z^{(k)}\right)^H \cdot A \cdot z^{(k)}}{\|z^{(k)}\|_2^2}$$

$$\xrightarrow{(5.1),(5.2)} \lambda_1 \quad (k \to \infty)$$

□

Remark 5.2.2

(i) Considering (5.3), one can distinguish the following cases:

- In the case $\lambda_1 > 0$ (i.e. real and positive), $x^{(k)}$ converges to the multiple $\frac{\alpha_1}{|\alpha_1|} v_1$ of v_1.
- If λ_1 is negative or complex, $x^{(k)}$ does not converge. For $k \gg 1$, however, $x^{(k)}$ is approximately a multiple of v_1.

(ii) The method converges linearly, and the convergence is the faster, the smaller $\left|\frac{\lambda_2}{\lambda_1}\right|$ is.

(iii) If $A \in \mathbb{C}^{n\times n}$ is a non-diagonalizable matrix with a uniquely determined eigenvalue λ_1 of largest modulus, i.e., $|\lambda_1| = |\lambda_i|$ implies $\lambda_1 = \lambda_i$, then the following holds: If one replaces the representation

$$x^{(0)} = \sum_{i=1}^{n} \alpha_i v_i,$$

of the initial vector $x^{(0)}$ by a linear combination of eigenvectors and generalized eigenvectors of A, one can show in the same way as in the diagonalizable case that, under analogous assumptions on $x^{(0)}$, the Rayleigh quotient $\rho^{(k)}$ in the power method (Algorithm 6) converges to λ_1 and $x^{(k)}$ converges to an eigenvector associated with λ_1.

§3 Wielandt's Inverse Iteration and Shift Strategies

In many (in particular technical) applications, one is looking for the eigenvalue λ_n of smallest modulus of a nonsingular matrix $A \in \mathbb{C}^{n\times n}$, where:

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_{n-1}| > |\lambda_n| > 0.$$

To find it, one can exploit the fact that the eigenvalue of smallest modulus of A is the reciprocal of the eigenvalue of largest modulus of A^{-1}, i.e. for the eigenvalues ν_i of A^{-1} we have:

$$|\nu_n| > |\nu_{n-1}| \geq \cdots \geq |\nu_1|, \quad \text{where} \quad \nu_i = \frac{1}{\lambda_i}, \quad i = 1, \ldots, n.$$

Applying the power method to A^{-1} therefore provides a way to approximate the eigenvalue $\nu_n = \frac{1}{\lambda_n}$ of largest modulus of A^{-1}, and hence the eigenvalue λ_n of smallest modulus of A. In each iteration, the vector $a^{(k)} = A^{-1}x^{(k)}$ then has to be computed; this amounts to solving the linear system $Aa^{(k)} = x^{(k)}$. The method is called *Wielandt's inverse iteration.*

By means of a *shift*, one can also compute other eigenvalues. Here it is assumed that a good approximation μ of an eigenvalue λ_j of A is known, in the sense that

$$|\mu - \lambda_j| < |\mu - \lambda_i| \quad \forall i \neq j$$

holds. Then the matrix $(A - \mu I)^{-1}$ has the eigenvalue $(\lambda_j - \mu)^{-1}$ of largest modulus and the same eigenvectors as A. Thus the inverse iteration applied to $A - \mu I$ yields an approximation of $\frac{1}{\lambda_j - \mu}$, from which λ_j is obtained.

Algorithm 7: Wielandt's inverse iteration with spectral shift

Input: $A \in \mathbb{C}^{n\times n}$, $\mu \approx \lambda_j$, $x^{(0)} \in \mathbb{C}^n$ with $\|x^{(0)}\|_2 = 1$ and $\alpha_j \neq 0$ (coefficient of v_j in the eigenvector expansion of $x^{(0)}$)

Output: An approximation of the eigenvalue λ_j and of the associated eigenvector

for $k = 0, 1, 2, \ldots$ **do**
- Solve $(A - \mu I)a^{(k)} = x^{(k)}$
- $\rho^{(k)} \leftarrow \frac{(x^{(k)})^H a^{(k)}}{\|x^{(k)}\|_2^2}$
- $\lambda_j \approx \mu + 1/\rho^{(k)}$ (since $\rho^{(k)} \approx 1/(\lambda_j - \mu)$)
- $x^{(k+1)} \leftarrow \frac{a^{(k)}}{\|a^{(k)}\|_2}$

§4 The QR Algorithm

The methods presented in the previous sections, the vector iteration and Wielandt's inverse iteration, have the drawback that they can only determine certain individual eigenvalues. In this section we now turn to a more efficient method that allows us to approximate not just one but all eigenvalues of a matrix $A \in \mathbb{R}^{n\times n}$ simultaneously.

The basic algorithm for the method that we want to present and analyze in the following is the QR iteration:

Algorithm 8: QR iteration

Input: $A \in \mathbb{R}^{n\times n}$

$A^{(0)} \leftarrow A$

for $k \leftarrow 1, 2, \ldots$ **do**
- $[Q_k, R_k] \leftarrow \mathtt{qr}(A^{(k-1)})$
- $A^{(k)} \leftarrow R_k Q_k$

Thus, each iteration step incurs the cost of a QR factorization $\mathcal{O}(n^3)$ and of a matrix multiplication $\mathcal{O}(n^3)$. Altogether, this method therefore requires $\mathcal{O}(k_{\max} \cdot n^3)$ operations, where $k_{\max}$ is the number of iterations of the method.

Lemma 5.4.1

Let $A \in \mathbb{R}^{n\times n}$ and let $A^{(k)}$ for $k \in \mathbb{N}_0$ be defined as in Algorithm 8. Then:

(i) The matrices $A^{(k)}$ are orthogonally similar to A.

(ii) If A is symmetric, then so are the matrices $A^{(k)}$.

Proof:

(i) Let $A \in \mathbb{R}^{n\times n}$. With the notation of Algorithm 8 we then have:

$$\begin{aligned} A^{(k)} &= R_k \cdot Q_k = Q_k^T \cdot Q_k \cdot R_k \cdot Q_k = Q_k^T \cdot A^{(k-1)} \cdot Q_k \\ &= \cdots = Q_k^T \cdot Q_{k-1}^T \cdot \cdots \cdot Q_1^T \cdot A^{(0)} \cdot Q_1 \cdot \cdots \cdot Q_k \\ &= (Q_1 \cdot \cdots \cdot Q_k)^T \cdot A \cdot (Q_1 \cdot \cdots \cdot Q_k). \end{aligned}$$

Hence $A^{(k)}$ is orthogonally similar to A.

(ii) Let $A \in \mathbb{R}^{n\times n}$ additionally be symmetric. By (i), for all $k \in \mathbb{N}_0$ there is an orthogonal matrix $P_k \in \mathbb{R}^{n\times n}$ such that $A^{(k)} = P_k^T A P_k$. It follows that:

$$\left(A^{(k)}\right)^T = \left(P_k^T A P_k\right)^T = P_k^T A^T P_k = P_k^T A P_k = A^{(k)}. \qquad \square$$

For the QR iteration, the following convergence result holds, which goes back to Wilkinson:

Theorem 5.4.2 (Wilkinson)

Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix that satisfies the following properties:

(i) The eigenvalues are all distinct and nonzero:

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > 0.$$

(ii) The orthogonal matrix $\widetilde{Q}$ of the principal axis transformation

$$A = \widetilde{Q} \cdot D \cdot \widetilde{Q}^T,$$

where $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$, has an LU factorization of the form

$$\widetilde{Q}^T = L \cdot U.$$

Then for $A^{(k)}, Q_k$ and R_k as defined in Algorithm 8, up to signs, i.e. for suitably sign-normalized QR factorizations (signs of the diagonal entries of R_k):

$$\lim_{k\to\infty} Q_k = I, \quad \lim_{k\to\infty} R_k = \operatorname{diag}(\lambda_1, \ldots, \lambda_n).$$

Regardless of the sign normalization, $\lim_{k\to\infty} A^{(k)} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ holds, as well as

$$a_{ij}^{(k)} = \mathcal{O}\left(\left|\frac{\lambda_i}{\lambda_j}\right|^k\right) \quad \text{for } i > j \text{ and } k \to \infty, \quad \text{where } A^{(k)} = \left(a_{ij}^{(k)}\right)_{i,j=1,\ldots,n}.$$

Proof:

1. We first show by induction that for all $k \in \mathbb{N}$:

$$A^k = \underbrace{Q_1 \cdot \cdots \cdot Q_k}_{=:P_k} \cdot \underbrace{R_k \cdot \cdots \cdot R_1}_{=:U_k} \tag{5.4}$$

For $k = 1$ the claim is clear. From the construction of the $A^{(k)}$ it follows as in the proof of Lemma 5.4.1 that

$$Q_{k+1} \cdot R_{k+1} = A^{(k)} = Q_k^T \cdot \cdots \cdot Q_1^T \cdot A \cdot Q_1 \cdot \cdots \cdot Q_k = P_k^T \cdot A \cdot P_k$$

holds. With (5.4) as the induction hypothesis, it follows that

$$A^{k+1} = A \cdot A^k \overset{(5.4)}{=\!=} A \cdot P_k \cdot U_k = P_k \cdot \left(P_k^T \cdot A \cdot P_k\right) \cdot U_k = P_k \cdot Q_{k+1} \cdot R_{k+1} \cdot U_k = P_{k+1} \cdot U_{k+1}$$

2. With $Q := \widetilde{Q}^T$ we have

$$A^k = Q^T \cdot D^k \cdot Q \quad \text{with} \quad D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$$

and with $Q = L \cdot U$ we get

$$A^k = Q^T \cdot D^k \cdot Q = Q^T \cdot D^k \cdot L \cdot U = Q^T \cdot \left(D^k \cdot L \cdot D^{-k}\right) \cdot \left(D^k \cdot U\right).$$

For the entries of the lower triangular matrix $D^k \cdot L \cdot D^{-k}$ we have, with $L = (\ell_{i,j})_{i,j=1,\ldots,n}$,

$$\left(D^k \cdot L \cdot D^{-k}\right)_{i,j} = \ell_{i,j} \cdot \left(\frac{\lambda_i}{\lambda_j}\right)^k$$

and hence

$$D^k \cdot L \cdot D^{-k} = I + E_k \quad \text{with} \quad \lim_{k\to\infty} E_k = \mathbf{0}$$

With a QR factorization

$$I + E_k = \widetilde{Q}_k \cdot \widetilde{R}_k \tag{5.5}$$

it follows that

$$A^k = Q^T \cdot (I + E_k) \cdot D^k \cdot U = \left(Q^T \cdot \widetilde{Q}_k\right) \cdot \left(\widetilde{R}_k \cdot D^k \cdot U\right) \tag{5.6}$$

Moreover, $\lim\limits_{k\to\infty} E_k = \mathbf{0}$ implies

$$\lim_{k\to\infty} \widetilde{Q}_k = \lim_{k\to\infty} \widetilde{R}_k = I$$

3. In (5.6) we have found another QR factorization of A^k. Since a QR factorization is unique up to the signs of the diagonal entries of R, the QR factorization in (5.5) can be chosen such that

$$P_k = Q^T \cdot \widetilde{Q}_k \quad \text{and} \quad U_k = \widetilde{R}_k \cdot D^k \cdot U$$

holds. This is where the sign normalization enters: P_k and U_k are fixed by the algorithm, while $\lim_{k\to\infty} \widetilde{Q}_k = \lim_{k\to\infty} \widetilde{R}_k = I$ requires $\widetilde{R}_k$ with positive diagonal entries; in general one only has $P_k = Q^T \cdot \widetilde{Q}_k \cdot S_k$ with a diagonal sign matrix S_k, so the following limits of Q_k and R_k hold up to these signs, whereas $A^{(k)} = P_k^T A P_k = S_k \widetilde{Q}_k^T D \widetilde{Q}_k S_k$ and hence $A^{(k)} - D = S_k\big(\widetilde{Q}_k^T D \widetilde{Q}_k - D\big)S_k \to \mathbf{0}$ holds in any case.

For the limit $k \to \infty$ it then follows that

$$\begin{aligned} Q_k &= (Q_1 \cdot \ldots \cdot Q_{k-1})^T \cdot (Q_1 \cdot \ldots \cdot Q_{k-1} \cdot Q_k) \\ &= P_{k-1}^T \cdot P_k = \widetilde{Q}_{k-1}^T \cdot Q \cdot Q^T \cdot \widetilde{Q}_k \\ &= \widetilde{Q}_{k-1}^T \cdot \widetilde{Q}_k \\ &\to I \end{aligned}$$

and

$$\begin{aligned} R_k &= (R_k \cdot R_{k-1} \cdot \ldots \cdot R_1) \cdot (R_{k-1} \cdot \ldots \cdot R_1)^{-1} \\ &= U_k \cdot U_{k-1}^{-1} = \widetilde{R}_k \cdot D^k \cdot U \cdot U^{-1} \cdot D^{-(k-1)} \cdot \widetilde{R}_{k-1}^{-1} \\ &= \widetilde{R}_k \cdot D \cdot \widetilde{R}_{k-1}^{-1} \\ &\to D \end{aligned}$$

Finally, it follows that

$$\lim_{k\to\infty} A^{(k)} = \lim_{k\to\infty} R_k \cdot Q_k = D \cdot I = D. \qquad \nabla\!\!\!/$$

Chapter 6 Iterative Methods for Solving Linear Systems

The methods developed in Chapter 3, which are based on the LU factorization or the Cholesky factorization, belong to the direct methods. In exact arithmetic these methods yield a solution of the linear system $A \cdot x = b$ after finitely many steps.

In this chapter, iterative methods (also called indirect methods) are presented. Starting from an initial value $x^{(0)}$, they generate a sequence $\left(x^{(k)}\right)_{k\in\mathbb{N}}$ that converges to a solution.

Using an iterative method can be advantageous if

(a) each iteration can be carried out efficiently, and

(b) the sequence $\left(x^{(k)}\right)_{k\in\mathbb{N}}$ converges quickly to the solution.

Sources and further reading. The conjugate gradient method of §3 is due to Hestenes and Stiefel [9]. A comprehensive treatment of iterative methods for linear systems can be found in [17]; see also [4].

§1 Classical Iterative Methods – Splitting Methods

The methods presented in this section are all based on the following basic idea: if Q is invertible, then

$$\begin{aligned} A \cdot x = b &\Leftrightarrow \mathbf{0} = b - A \cdot x \\ &\Leftrightarrow \mathbf{0} = Q^{-1} \cdot (b - A \cdot x) \\ &\Leftrightarrow x = x + Q^{-1} \cdot (b - A \cdot x) \\ &\Leftrightarrow x = x - Q^{-1} \cdot A \cdot x + Q^{-1} \cdot b \\ &\Leftrightarrow x = \underbrace{\left(I - Q^{-1} \cdot A\right)}_{=:G} \cdot x + \underbrace{Q^{-1} \cdot b}_{=:c} =: \Phi(x). \end{aligned}$$

This means that the solution of $A \cdot x = b$ is a fixed point of the map Φ, where

$$\Phi(x) = G \cdot x + c.$$

The basic idea of splitting methods is to choose the fixed-point map $\Phi(x)$ such that the method can be carried out efficiently and also converges quickly.

Whether these methods can be applied efficiently depends on Q^{-1}, and hence also G, being easy to compute. The speed of convergence, in turn, is essentially determined by how well Q^{-1} approximates A^{-1}.

In the following, this idea is analyzed mathematically in a formal way, and the conditions for convergence and efficiency of the methods are derived precisely.

Theorem 6.1.1 (Banach fixed-point theorem)

Let $G \in \mathbb{R}^{n\times n}$, $c \in \mathbb{R}^n$, and suppose that for a compatible matrix norm

$$\|G\| < 1.$$

Then the sequence defined by the iteration

$$x^{(k+1)} = \Phi\left(x^{(k)}\right) := G \cdot x^{(k)} + c$$

converges for every initial value $x^{(0)} \in \mathbb{R}^n$ to a fixed point $\widehat{x}$ of the map Φ, i.e., the limit satisfies

$$\widehat{x} = \Phi(\widehat{x}).$$

Moreover, $\widehat{x}$ is the only fixed point of Φ.

Proof: We first show that the sequence $\left(x^{(k)}\right)_k$ is a Cauchy sequence. For $k > 0$ we have:

$$\begin{aligned}
\left\|x^{(k+1)} - x^{(k)}\right\| &= \left\|\Phi\left(x^{(k)}\right) - \Phi\left(x^{(k-1)}\right)\right\| \\
&= \left\|G \cdot x^{(k)} - G \cdot x^{(k-1)}\right\| \\
&= \left\|G \cdot \left(x^{(k)} - x^{(k-1)}\right)\right\| \\
&\leq \|G\| \cdot \left\|x^{(k)} - x^{(k-1)}\right\| \\
&\leq \|G\|^k \cdot \left\|x^{(1)} - x^{(0)}\right\|.
\end{aligned}$$

For $p \geq m$ it follows that:

$$\begin{aligned}
\|x^{(p)} - x^{(m)}\| &= \left\|\sum_{k=m}^{p-1} \left(x^{(k+1)} - x^{(k)}\right)\right\| \\
&\leq \sum_{k=m}^{p-1} \left\|x^{(k+1)} - x^{(k)}\right\| \\
&\leq \sum_{k=m}^{p-1} \|G\|^k \cdot \left\|x^{(1)} - x^{(0)}\right\| \\
&= \|G\|^m \cdot \left\|x^{(1)} - x^{(0)}\right\| \cdot \sum_{k=0}^{p-m-1} \|G\|^k \\
&= \|G\|^m \cdot \left\|x^{(1)} - x^{(0)}\right\| \cdot \frac{1 - \|G\|^{p-m}}{1 - \|G\|} \\
&\leq \underbrace{\frac{1}{1 - \|G\|} \cdot \left\|x^{(1)} - x^{(0)}\right\|}_{=:\alpha} \cdot \|G\|^m \\
&< \varepsilon \quad \text{for } m > \frac{\log \frac{\varepsilon}{\alpha}}{\log \|G\|}.
\end{aligned}$$

Since $\mathbb{R}^n$ is complete with respect to every norm, the sequence $\left(x^{(k)}\right)_k$ converges to a limit $\widehat{x}$. Since Φ is continuous, we have:

$$\widehat{x} = \lim_{k\to\infty} \Phi\left(x^{(k)}\right) \overset{\Phi \text{ continuous}}{=\!=\!=} \Phi\left(\lim_{k\to\infty} x^{(k)}\right) = \Phi\left(\widehat{x}\right).$$

Hence $\widehat{x}$ is a fixed point of Φ. If $\widetilde{x}$ were another fixed point, then we would have:

$$\|\widehat{x} - \widetilde{x}\| = \|\Phi\left(\widehat{x}\right) - \Phi\left(\widetilde{x}\right)\| = \|G \cdot \left(\widehat{x} - \widetilde{x}\right)\| \leq \|G\| \cdot \|\widehat{x} - \widetilde{x}\|.$$

Because of $\|G\| < 1$, this is only possible if $\|\widehat{x} - \widetilde{x}\| = 0$. This proves the uniqueness of the fixed point. ☑

Remark 6.1.2

(a) In Theorem 6.1.1, $\|\cdot\|$ can be an arbitrary compatible matrix norm. This is remarkable, because it means that the sequence $\left(x^{(k)}\right)_k$ converges to a unique fixed point as soon as there exists a compatible matrix norm with $\|G\| < 1$. Since all norms on $\mathbb{R}^n$ are equivalent, the sequence $\left(x^{(k)}\right)_k$ also converges with respect to every other vector norm. This need not necessarily be a vector norm that is compatible with the matrix norm.

(b) The speed of convergence depends on $\|G\|$, because

$$\left\|x^{(k)} - \widehat{x}\right\| = \left\|\Phi\left(x^{(k-1)}\right) - \Phi\left(\widehat{x}\right)\right\| \leq \|G\| \cdot \left\|x^{(k-1)} - \widehat{x}\right\| \leq \cdots \leq \|G\|^k \cdot \left\|x^{(0)} - \widehat{x}\right\|$$

Whether there can be a compatible matrix norm with $\|G\| < 1$ depends on the spectral radius of G, which we define first.

Definition 6.1.3 (Spectral radius)

The spectral radius $\varrho(A)$ of a matrix $A \in \mathbb{R}^{n\times n}$ is defined as

$$\varrho(A) := \max\{|\lambda| : \lambda \text{ is an eigenvalue of } A\}$$

That $\|G\| < 1$ holds for a compatible matrix norm if $\varrho(G) < 1$ is satisfied follows from the following theorem.

Theorem 6.1.4

For $A \in \mathbb{R}^{n\times n}$ we have:

(a) For every compatible matrix norm, $\varrho(A) \leq \|A\|$.

(b) $\varrho(A) = \|A\|_2$ if $A = A^T$.

(c) For every $\varepsilon > 0$ there exists an induced matrix norm (in particular, a compatible matrix norm) with $\|A\| \leq \varrho(A) + \varepsilon$.

Proof:

(a) (Since eigenvectors and Jordan bases of a real matrix may be complex, the vector norms here and in (c) are understood as extended to $\mathbb{C}^n$, with the matrix norm compatible on $\mathbb{C}^n$.) Let x be an eigenvector for an eigenvalue λ with $|\lambda| = \varrho(A)$; then

$$|\lambda| \cdot \|x\| = \|\lambda \cdot x\| = \|A \cdot x\| \leq \|A\| \cdot \|x\| \quad \overset{x\neq \mathbf{0}}{\Longrightarrow} \quad \varrho(A) = |\lambda| \leq \|A\|$$

(b) For a symmetric matrix A, it follows from the principal axis transformation that $\varrho(A)^2 = \varrho(A^T \cdot A)$ and hence

$$\|A\|_2 = \sqrt{\varrho(A^T \cdot A)} = \varrho(A)$$

(c) For A there exists the Jordan normal form $W^{-1} \cdot A \cdot W = J = \operatorname{diag}(J_1, \ldots, J_t)$ with Jordan blocks of the form

$$J_i = \begin{pmatrix} \lambda_i & 1 & & & \\ & \lambda_i & \ddots & & \\ & & \ddots & \ddots & \\ & & & \lambda_i & 1 \\ & & & & \lambda_i \end{pmatrix} \quad (1 \leq i \leq t)$$

For $\varepsilon > 0$ we define $D = \operatorname{diag}(1, \varepsilon, \ldots, \varepsilon^{n-1})$ and form $\widehat{J} = D^{-1} \cdot J \cdot D$. The matrix $\widehat{J}$ then has the form $\widehat{J} = \operatorname{diag}(\widehat{J}_1, \ldots, \widehat{J}_t)$ with

$$\widehat{J}_i = \begin{pmatrix} \lambda_i & \varepsilon & & & \\ & \lambda_i & \ddots & & \\ & & \ddots & \ddots & \\ & & & \lambda_i & \varepsilon \\ & & & & \lambda_i \end{pmatrix} \quad (1 \leq i \leq t)$$

Now we define $T := (W \cdot D)^{-1}$ and, with it, the vector norm defined by

$$\|x\|_T := \|T \cdot x\|_\infty$$

For the induced matrix norm we then have

$$\begin{aligned} \|A\|_T &= \max_{\|x\|_T = 1} \|A \cdot x\|_T = \max_{\|T \cdot x\|_\infty = 1} \|T \cdot A \cdot x\|_\infty = \max_{\|T \cdot x\|_\infty = 1} \|T \cdot A \cdot T^{-1} \cdot T \cdot x\|_\infty \\ &\leq \max_{\|y\|_\infty = 1} \|T \cdot A \cdot T^{-1} \cdot y\|_\infty = \|T \cdot A \cdot T^{-1}\|_\infty \end{aligned}$$

Then

$$\|A\|_T \leq \|T \cdot A \cdot T^{-1}\|_\infty = \|D^{-1} \cdot W^{-1} \cdot A \cdot W \cdot D\|_\infty = \|\widehat{J}\|_\infty \leq \varrho(A) + \varepsilon$$

□

Derivation 6.1.5 (Jacobi method)

We use the additive decomposition

$$A = L + D + R,$$

where

- $D = \operatorname{diag}(A) = \operatorname{diag}(a_{11}, \ldots, a_{nn})$ is the diagonal matrix of A,
- $L = \operatorname{tril}(A, -1)$ denotes the strictly lower triangular part of A, and

- $R = \mathrm{triu}(A, 1)$ is the strictly upper triangular part of A.

Setting $Q = D$ means approximating A by its diagonal matrix. For the iterative method we then obtain:

$$\begin{aligned} x^{(k+1)} &= \left(I - D^{-1} \cdot A\right) \cdot x^{(k)} + D^{-1} \cdot b \\ &= \left(I - D^{-1} \cdot (L + D + R)\right) \cdot x^{(k)} + D^{-1} \cdot b \\ &= \left(I - D^{-1} \cdot L - D^{-1} \cdot D - D^{-1} \cdot R\right) \cdot x^{(k)} + D^{-1} \cdot b \\ &= -D^{-1} \cdot (L + R) \cdot x^{(k)} + D^{-1} \cdot b. \end{aligned} \tag{6.1}$$

Theorem 6.1.6

The Jacobi method defined by (6.1) converges for every initial value $x^{(0)}$ to $\widehat{x} = A^{-1} \cdot b$ if A is strictly diagonally dominant, i.e.,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i$$

Proof: For $G = -D^{-1} \cdot (L + R)$ we have

$$\|G\|_\infty = \max_i \sum_j |G_{ij}| = \max_i \underbrace{\sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|}}_{<1 \ \forall i} < 1$$

and hence convergence follows from Theorem 6.1.1. ☐

Remark 6.1.7

(a) The Jacobi method converges for every initial value $x^{(0)}$ to $\widehat{x} = A^{-1} \cdot b$ if A^T is strictly diagonally dominant, i.e., if

$$|a_{jj}| > \sum_{i \neq j} |a_{ij}| \quad \forall j,$$

because then $\|D \cdot G \cdot D^{-1}\|_1 < 1$, i.e., $\|G\| < 1$ for the matrix norm induced by the vector norm $\|x\| := \|D \cdot x\|_1$.

(b) For an irreducible matrix A, diagonal dominance (instead of strict diagonal dominance) suffices to ensure convergence.

If A is irreducible and

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}| \quad \forall i$$

as well as

$$\exists k \in \{1, \ldots, n\} : \ |a_{kk}| > \sum_{j \neq k} |a_{kj}|,$$

then the Jacobi method converges.

For

$$A = \begin{pmatrix} 2 & -1 & \\ -1 & 2 & -1 \\ & -1 & 2 \end{pmatrix}$$

these assumptions are satisfied.

(c) In (b), too, A can be replaced by A^T.

Derivation 6.1.8 (Gauss–Seidel method)

We again use the additive decomposition

$$A = L + D + R,$$

where

- $D = \operatorname{diag}(A) = \operatorname{diag}(a_{11}, \ldots, a_{nn})$ is the diagonal matrix of A,
- $L = \operatorname{tril}(A, -1)$ denotes the strictly lower triangular part of A, and
- $R = \operatorname{triu}(A, 1)$ is the strictly upper triangular part of A.

Approximating A by $Q = D + L$, we obtain the Gauss–Seidel method with:

$$\begin{aligned} x^{(k+1)} &= \big(I - (L+D)^{-1} \cdot A\big) \cdot x^{(k)} + (L+D)^{-1} \cdot b \\ &= \big(I - \underbrace{(L+D)^{-1} \cdot (L+D+R)}_{=I+(L+D)^{-1} \cdot R}\big) \cdot x^{(k)} + (L+D)^{-1} \cdot b \\ &= -(L+D)^{-1} \cdot R \cdot x^{(k)} + (L+D)^{-1} \cdot b. \end{aligned} \tag{6.2}$$

Remark 6.1.9

(a) If A is strictly diagonally dominant, the Gauss–Seidel method converges.

(b) Since the Gauss–Seidel method approximates the matrix A better, one might expect the Gauss–Seidel method to converge faster. For a certain class of matrices this can indeed be shown. If, for example,

$$-D^{-1} \cdot (L + R) \geq 0,$$

then the Gauss–Seidel method converges faster than the Jacobi method, provided that one of the two methods converges at all.

(c) An important application of the classical iterative methods is the acceleration of other methods. For example, they are used as preconditioners in the CG method or as smoothers in multigrid methods.

§2 The Gradient Method

If $A \in \mathbb{R}^{n\times n}$ is symmetric and positive definite, one can solve $A \cdot x = b$ by solving an associated minimization problem.

Lemma 6.2.1

Let $A \in \mathbb{R}^{n\times n}$ be symmetric and positive definite; then:

$$A \cdot x = b \quad \Leftrightarrow \quad f(x) = \min_y f(y) \quad \textit{with} \quad f(x) = \frac{1}{2} \cdot x^T \cdot A \cdot x - b^T \cdot x.$$

Proof: We have $\nabla f(x) = A \cdot x - b$, and the Hessian $H_f(x) = A$ is symmetric and positive definite. There exists a unique minimizer x, which is characterized by the equation

$$\mathbf{0} = \nabla f(x) = A \cdot x - b$$

□

The basic idea of the gradient method is to search iteratively for the global minimizer by moving in the direction of steepest descent (i.e., in the direction $-\nabla f$):

Choose an initial value $x^{(0)} \in \mathbb{R}^n$.

For $k = 0, 1, \ldots$:

(1) Compute $r^{(k)} = -\nabla f\left(x^{(k)}\right)$.

(2) Determine $x^{(k+1)}$ such that

$$f\left(x^{(k+1)}\right) = \min_{t\geq 0} f\left(x^{(k)} + t \cdot r^{(k)}\right)$$

holds.

For a quadratic function f, (1) and (2) are easy to compute:

(1)

$$f(x) = \frac{1}{2} \cdot x^T \cdot A \cdot x - b^T \cdot x \Rightarrow \nabla f(x) = A \cdot x - b$$
$$\Rightarrow r^{(k)} = b - A \cdot x^{(k)}$$

(2) Define $h : \mathbb{R} \to \mathbb{R}$ by

$$h(t) = f\left(x^{(k)} + t \cdot r^{(k)}\right),$$

then

$$\begin{aligned} h'(t) &= \frac{\mathrm{d}}{\mathrm{d}t} f\left(x^{(k)} + t \cdot r^{(k)}\right) = \nabla f\left(x^{(k)} + t \cdot r^{(k)}\right)^T \cdot r^{(k)} \\ &= \left[A \cdot \left(x^{(k)} + t \cdot r^{(k)}\right) - b\right]^T \cdot r^{(k)} \\ &= \left(x^{(k)}\right)^T \cdot A \cdot r^{(k)} + t \cdot \left(r^{(k)}\right)^T \cdot A \cdot r^{(k)} - b^T \cdot r^{(k)} \\ &= t \cdot \left(r^{(k)}\right)^T \cdot A \cdot r^{(k)} + \underbrace{\left(A \cdot x^{(k)} - b\right)}_{=-r^{(k)}}{}^T \cdot r^{(k)} \\ &= t \cdot \left(r^{(k)}\right)^T \cdot A \cdot r^{(k)} - \left(r^{(k)}\right)^T \cdot r^{(k)} \end{aligned}$$

Therefore

$$h'(t) = 0 \quad \Leftrightarrow \quad t = \underbrace{\frac{\left(r^{(k)}\right)^T \cdot r^{(k)}}{\left(r^{(k)}\right)^T \cdot A \cdot r^{(k)}}}_{\geq 0} =: \alpha_k \tag{6.3}$$

Since A is positive definite, $h''(t) > 0$ for all t, and hence this is the global minimizer. We thus obtain

$$x^{(k+1)} = x^{(k)} + \alpha_k \cdot r^{(k)}.$$

Algorithm 9 implements this idea. Only one matrix-vector product is computed per iteration.

Algorithm 9: Gradient method

Input: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite, right-hand side $b \in \mathbb{R}^n$, initial vector $x^{(0)} \in \mathbb{R}^n$, tolerance tol
Output: approximation $x^{(k)}$ to the solution of the linear system $Ax = b$
$r^{(0)} \leftarrow b - Ax^{(0)}$
$a^{(0)} \leftarrow Ar^{(0)}$
$k \leftarrow 0$
while $\left\|r^{(k)}\right\|_A = \sqrt{\left(r^{(k)}\right)^T a^{(k)}} > tol$ **do**
    $\alpha_k \leftarrow \frac{\left(r^{(k)}\right)^T r^{(k)}}{\left(r^{(k)}\right)^T a^{(k)}}$
    $x^{(k+1)} \leftarrow x^{(k)} + \alpha_k r^{(k)}$
    $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k a^{(k)}$
    $a^{(k+1)} \leftarrow Ar^{(k+1)}$
    $k \leftarrow k + 1$

Theorem 6.2.2 *For the gradient method in Algorithm 9, the following error estimate holds for a symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$:*

$$\|x^{(k)} - \widehat{x}\|_A \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \cdot \|x^{(0)} - \widehat{x}\|_A.$$

Here:

(i) $\widehat{x}$ *denotes the exact solution of* $Ax = b$,

(ii) $\|\cdot\|_A$ *denotes the so-called energy norm, defined by*

$$\|x\|_A := \sqrt{x^T A x},$$

(iii) $\kappa = \kappa_2(A)$ *denotes the condition number of* A *with respect to the Euclidean norm.*

Remark 6.2.3 For large condition numbers $\kappa \gg 1$ we have approximately:

$$\frac{\kappa - 1}{\kappa + 1} = \underbrace{\frac{\kappa}{\kappa + 1}}_{\approx 1} - \underbrace{\frac{1}{\kappa + 1}}_{\approx 0} \approx 1.$$

Since this term describes the error reduction, a large condition number means that the method converges very slowly. This can already be observed in simple examples (see the exercises), for instance with

$$A = \begin{pmatrix} 1 & 0 \\ 0 & a \end{pmatrix}, \quad a \gg 1,$$

and the right-hand side

$$b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

and the initial vector

$$x^{(0)} = \begin{pmatrix} a \\ 1 \end{pmatrix}.$$

§3 The Conjugate Gradient Method

To avoid the "zigzag course" of the gradient method, one chooses descent directions that are A-orthogonal. This means that they are orthogonal with respect to the inner product defined by

$$\langle x, y \rangle_A := x^T \cdot A \cdot y$$

Remark 6.3.1

(a) Two vectors $x, y \in \mathbb{R}^n$ are called conjugate or A-orthogonal if

$$\langle x, y \rangle_A = x^T \cdot A \cdot y = 0$$

holds.

(b) If $d_1, \ldots, d_k \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ are pairwise conjugate, i.e.,

$$\langle d_i, d_j \rangle_A = \|d_i\|_A^2 \cdot \delta_{i,j}, \quad \text{where} \quad \delta_{i,j} := \begin{cases} 1, & i = j, \\ 0, & \text{otherwise}, \end{cases} \tag{6.4}$$

then $d_1, \ldots, d_k$ are linearly independent. Indeed, if

$$\sum_{i=1}^{k} \alpha_i \cdot d_i = \mathbf{0} \quad \text{for } \alpha_1, \ldots, \alpha_k \in \mathbb{R},$$

then it follows for an arbitrary $j \in \{1, \ldots, k\}$ that

$$\begin{aligned} 0 = \langle \mathbf{0}, d_j \rangle_A = \left\langle \sum_{i=1}^{k} \alpha_i \cdot d_i, d_j \right\rangle_A &= \sum_{i=1}^{k} \alpha_i \cdot \langle d_i, d_j \rangle_A \\ = \alpha_j \cdot \langle d_j, d_j \rangle_A &= \alpha_j \cdot \|d_j\|_A^2. \end{aligned}$$

Since $d_j \neq \mathbf{0}$, we have $\|d_j\|_A^2 \neq 0$, and hence $\alpha_j = 0$.

(c) If $d_1, \ldots, d_n \in \mathbb{R}^n$ are conjugate, they form a basis of $\mathbb{R}^n$. Every vector $x \in \mathbb{R}^n$ therefore has a unique representation

$$x = \sum_{k=1}^{n} \alpha_k \cdot d_k.$$

From (6.4) it follows that

$$\langle d_i, x \rangle_A = \langle d_i, \sum_{k=1}^{n} \alpha_k \cdot d_k \rangle_A = \sum_{k=1}^{n} \alpha_k \cdot \langle d_i, d_k \rangle_A = \alpha_i \cdot \langle d_i, d_i \rangle_A,$$

and hence

$$\alpha_i = \frac{\langle d_i, x \rangle_A}{\langle d_i, d_i \rangle_A} = \frac{d_i^T \cdot A \cdot x}{d_i^T \cdot A \cdot d_i}.$$

If x is the solution of $Ax = b$, then

$$\alpha_i = \frac{d_i^T \cdot A \cdot x}{d_i^T \cdot A \cdot d_i} = \frac{d_i^T \cdot b}{d_i^T \cdot A \cdot d_i}. \tag{6.5}$$

Moreover,

$$\begin{aligned} x^T \cdot A \cdot x &= \left(\sum_{k=1}^{n} \alpha_k \cdot d_k \right)^T A \left(\sum_{k=1}^{n} \alpha_k \cdot d_k \right) \\ &= \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \cdot \alpha_j \cdot \underbrace{d_i^T \cdot A \cdot d_j}_{=0 \text{ for } i \neq j} \\ &= \sum_{k=1}^{n} \alpha_k^2 \cdot d_k^T \cdot A \cdot d_k \end{aligned}$$

Motivation 6.3.2 (Conjugate gradient method)

(a) Let $x \in \mathbb{R}^n$ and define the function

$$f(x) = \frac{1}{2} \cdot x^T \cdot A \cdot x - b^T \cdot x.$$

Furthermore, let $d_1, \ldots, d_n \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ be pairwise conjugate directions. For the representation

$$x = \sum_{k=1}^{n} \alpha_k \cdot d_k$$

it then follows that

$$\begin{aligned} f(x) &= f\left(\sum_{k=1}^{n} \alpha_k \cdot d_k \right) \\ &= \frac{1}{2} \cdot \left(\sum_{k=1}^{n} \alpha_k \cdot d_k \right)^T \cdot A \cdot \left(\sum_{k=1}^{n} \alpha_k \cdot d_k \right) - b^T \sum_{k=1}^{n} \alpha_k \cdot d_k \\ &= \sum_{k=1}^{n} \left[\frac{1}{2} \cdot \alpha_k^2 \cdot d_k^T \cdot A \cdot d_k - \alpha_k \cdot d_k^T \cdot b \right] \\ &= \sum_{k=1}^{n} f\left(\alpha_k \cdot d_k \right). \end{aligned}$$

(b) The minimization problem thus reduces to

$$\min_{x\in\mathbb{R}^n} f(x) = \sum_{k=1}^{n} \min_{\alpha_k\in\mathbb{R}} f\left(\alpha_k \cdot d_k\right).$$

Analogously to (6.3), it follows for the optimal scalars $\widehat{\alpha}_k$ that

$$f\left(\widehat{\alpha}_k \cdot d_k\right) = \min_{\alpha_k\in\mathbb{R}} f\left(\alpha_k d_k\right) \quad \Leftrightarrow \quad \widehat{\alpha}_k = \frac{d_k^T b}{d_k^T A d_k}.$$

Altogether, we therefore have

$$f(\widehat{x}) = \min_{x\in\mathbb{R}^n} f(x) \quad \Leftrightarrow \quad \widehat{x} = \sum_{k=1}^{n} \widehat{\alpha}_k \cdot d_k \quad \text{with} \quad \widehat{\alpha}_k = \frac{d_k^T \cdot b}{d_k^T \cdot A \cdot d_k}.$$

So if a basis of conjugate directions is known, then the n-dimensional minimization problem can be reduced to n one-dimensional minimization problems, each of which is easy to solve.

(c) The open question is how to obtain a basis of conjugate directions in practice. In the conjugate gradient method these are constructed successively. In the k-th step, a problem of the form

$$f(x_{k-1} + \widehat{\alpha}_k \cdot d_k) = \min_{\alpha_k\in\mathbb{R}} f(x_{k-1} + \alpha_k \cdot d_k)$$

is solved. Here the solution is obtained as

$$\widehat{\alpha}_k = \frac{d_k^T \cdot (b - A \cdot x_{k-1})}{d_k^T \cdot A \cdot d_k} = \frac{d_k^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k} \quad \text{with} \quad r_{k-1} = b - A \cdot x_{k-1}$$

(d) This yields the following method:

(0) Choose an arbitrary initial value $x_0 \in \mathbb{R}^n$ and compute the residual

$$r_0 = b - A \cdot x_0.$$

Then set the first conjugate direction to

$$d_1 = r_0.$$

(1) If $r_0 = \mathbf{0}$, then x_0 is the solution and the method terminates. Otherwise compute

$$\widehat{\alpha}_1 = \frac{d_1^T \cdot r_0}{d_1^T \cdot A \cdot d_1}, \quad x_1 = x_0 + \widehat{\alpha}_1 \cdot d_1,$$

as well as the new residual

$$r_1 = r_0 - \widehat{\alpha}_1 \cdot A \cdot d_1.$$

Then we orthogonalize r_1 against d_1 with respect to the inner product $\langle \cdot, \cdot \rangle_A$. This gives us

$$d_2 = r_1 - \frac{d_1^T \cdot A \cdot r_1}{d_1^T \cdot A \cdot d_1} d_1.$$

(2) If $r_1 = \mathbf{0}$, then x_1 is the solution and the method terminates. Otherwise compute

$$\widehat{\alpha}_2 = \frac{d_2^T \cdot r_1}{d_2^T \cdot A \cdot d_2}, \quad x_2 = x_1 + \widehat{\alpha}_2 \cdot d_2,$$

as well as the new residual

$$r_2 = r_1 - \widehat{\alpha}_2 \cdot A \cdot d_2.$$

Then determine d_3 by orthogonalizing r_2 against d_1 and d_2 by means of Gram–Schmidt orthogonalization:

$$d_3 = r_2 - \frac{d_1^T \cdot A \cdot r_2}{d_1^T \cdot A \cdot d_1} d_1 - \frac{d_2^T \cdot A \cdot r_2}{d_2^T \cdot A \cdot d_2} \cdot d_2.$$

$\vdots$

(k) If $r_{k-1} = \mathbf{0}$, then x_{k-1} is the solution and the method terminates. Otherwise compute

$$\widehat{\alpha}_k = \frac{d_k^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k}, \quad x_k = x_{k-1} + \widehat{\alpha}_k \cdot d_k,$$

as well as the new residual

$$r_k = r_{k-1} - \widehat{\alpha}_k \cdot A \cdot d_k.$$

Then determine d_{k+1} by orthogonalizing r_k against $d_1, \ldots, d_k$ by means of Gram–Schmidt orthogonalization:

$$d_{k+1} = r_k - \sum_{\ell=1}^{k} \frac{d_\ell^T \cdot A \cdot r_k}{d_\ell^T \cdot A \cdot d_\ell} \cdot d_\ell.$$

In exact arithmetic, the method therefore terminates after at most n steps. This procedure is implemented in Algorithm 10.

Algorithm 10: Conjugate gradient method (first version)

Input: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite, right-hand side $b \in \mathbb{R}^n$, initial vector $x_0 \in \mathbb{R}^n$, tolerance tol
Output: approximation x_k to the solution of the linear system $Ax = b$
$r_0 \leftarrow b - A \cdot x_0$
$d_1 \leftarrow r_0$
$k \leftarrow 1$
while $\|r_{k-1}\|_2 > tol$ **do**
    $\widehat{\alpha}_k \leftarrow \frac{d_k^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k}$
    $x_k \leftarrow x_{k-1} + \widehat{\alpha}_k \cdot d_k$
    $r_k \leftarrow r_{k-1} - \widehat{\alpha}_k \cdot A \cdot d_k$
    $d_{k+1} \leftarrow r_k$
    for $\ell = 1$ ***to*** k **do**
        $d_{k+1} \leftarrow d_{k+1} - \frac{d_\ell^T \cdot A \cdot r_k}{d_\ell^T \cdot A \cdot d_\ell} \cdot d_\ell$
    $k \leftarrow k + 1$

Algorithm 10 has the drawback that all conjugate directions must be stored. Moreover, the computational cost of the orthogonalization (lines 11 and 12) grows with every iteration.

Lemma 6.3.3

In Algorithm 10 we have

$$d_i^T \cdot r_j = 0 \quad \forall i \leq j \tag{6.6}$$

and

$$r_i^T \cdot r_j = 0 \quad \forall i < j \tag{6.7}$$

Proof: In the representation

$$x_j = x_0 + \sum_{\ell=1}^{j} \widehat{\alpha}_\ell \cdot d_\ell$$

the coefficients $\widehat{\alpha}_1, \ldots, \widehat{\alpha}_j$ are chosen such that

$$f\left(x_0 + \sum_{\ell=1}^{j} \widehat{\alpha}_\ell \cdot d_\ell\right) = \min_{\alpha_1,\ldots,\alpha_j \in \mathbb{R}} f\left(x_0 + \sum_{\ell=1}^{j} \alpha_\ell \cdot d_\ell\right)$$

holds. At the minimizer, the necessary condition

$$\frac{\mathrm{d}}{\mathrm{d}\,\alpha_i} f\left(x_0 + \sum_{\ell=1}^{j} \alpha_\ell \cdot d_\ell\right)\Bigg|_{\alpha_1=\widehat{\alpha}_1,\ldots,\alpha_j=\widehat{\alpha}_j} = 0 \quad \text{for} \quad 1 \leq i \leq j$$

must therefore be satisfied. A direct computation thus yields, for the derivative with respect to α_i and $i \in \{1, \ldots, j\}$,

$$\begin{aligned}
0 =& \frac{\mathrm{d}}{\mathrm{d}\,\alpha_i} f\left(x_0 + \sum_{\ell=1}^{j} \alpha_\ell \cdot d_\ell\right)\Bigg|_{\alpha_1=\widehat{\alpha}_1,\ldots,\alpha_j=\widehat{\alpha}_j} \\
=& \nabla f\left(x_0 + \sum_{\ell=1}^{j} \widehat{\alpha}_\ell \cdot d_\ell\right)^T \cdot d_i \\
=& \nabla f\left(x_j\right)^T \cdot d_i \\
=& \left(A \cdot x_j - b\right)^T \cdot d_i \\
=& - r_j^T \cdot d_i = -d_i^T \cdot r_j
\end{aligned}$$

This proves (6.6).

For every $k \in \{1, \ldots, n\}$, d_k is constructed from $r_0, \ldots, r_{k-1}$ by means of Gram–Schmidt orthogonalization. Therefore

$$\begin{aligned}
\operatorname{span}\{r_0\} &= \operatorname{span}\{d_1\} \\
\operatorname{span}\{r_0, r_1\} &= \operatorname{span}\{d_1, d_2\} \\
&\vdots \\
\operatorname{span}\{r_0, \ldots, r_{k-1}\} &= \operatorname{span}\{d_1, \ldots, d_k\}
\end{aligned}$$

Hence r_{k-1} can be written as a linear combination of $d_1, \ldots, d_k$, i.e., there exist coefficients $\beta_1, \ldots, \beta_k$ with

$$r_{k-1} = \sum_{\ell=1}^{k} \beta_\ell \cdot d_\ell.$$

Thus, for $i = k-1 < j$ (and hence $k \leq j$),

$$r_i^T \cdot r_j = r_{k-1}^T \cdot r_j = \sum_{\ell=1}^{k} \beta_\ell \cdot \underbrace{d_\ell^T \cdot r_j}_{=0 \text{ (6.6)}} = 0$$

□

Corollary 6.3.4

In Algorithm 10 we have

$$d_k^T \cdot r_{k-1} = r_{k-1}^T \cdot r_{k-1}$$

Proof:

$$\begin{aligned} d_k^T \cdot r_{k-1} &= \left(r_{k-1} - \sum_{\ell=1}^{k-1} \frac{d_\ell^T \cdot A \cdot r_{k-1}}{d_\ell^T \cdot A \cdot d_\ell} \cdot d_\ell \right)^T \cdot r_{k-1} \\ &= r_{k-1}^T \cdot r_{k-1} - \sum_{\ell=1}^{k-1} \frac{d_\ell^T \cdot A \cdot r_{k-1}}{d_\ell^T \cdot A \cdot d_\ell} \cdot d_\ell^T \cdot r_{k-1} \\ &\overset{\text{Lemma 6.3.3}}{=\!=\!=} r_{k-1}^T \cdot r_{k-1} \end{aligned}$$

□

Corollary 6.3.5

In Algorithm 10 we have

$$d_\ell^T \cdot A \cdot r_k = \begin{cases} 0 & \ell < k \\ -\frac{r_k^T \cdot r_k}{\widehat{\alpha}_k} & \ell = k \end{cases}$$

Proof: We have

$$r_\ell = r_{\ell-1} - \widehat{\alpha}_\ell \cdot A \cdot d_\ell$$

and $\widehat{\alpha}_\ell \neq 0$: by Corollary 6.3.4, $\widehat{\alpha}_\ell = r_{\ell-1}^T r_{\ell-1} / (d_\ell^T A d_\ell)$, which vanishes only if $r_{\ell-1} = \mathbf{0}$, in which case the method would already have terminated before computing $\widehat{\alpha}_\ell$. It follows that

$$A \cdot d_\ell = \frac{r_{\ell-1} - r_\ell}{\widehat{\alpha}_\ell}.$$

Hence

$$\begin{aligned} d_\ell^T \cdot A \cdot r_k = r_k^T \cdot A \cdot d_\ell = r_k^T \cdot \left(\frac{r_{\ell-1} - r_\ell}{\widehat{\alpha}_\ell} \right) \\ \overset{\text{Lemma 6.3.3}}{=\!=} \underbrace{\frac{r_k^T \cdot r_{\ell-1}}{\widehat{\alpha}_\ell}}_{=0} - \frac{r_k^T \cdot r_\ell}{\widehat{\alpha}_\ell} \\ \overset{\text{Lemma 6.3.3}}{=\!=} \begin{cases} 0 & \ell < k \\ -\frac{r_k^T \cdot r_k}{\widehat{\alpha}_k} & \ell = k \end{cases} \end{aligned}$$

□

From Corollary 6.3.4 and Corollary 6.3.5 it now follows that

$$\begin{aligned} d_{k+1} &= r_k - \sum_{\ell=1}^{k} \frac{d_\ell^T A r_k}{d_\ell^T A d_\ell} d_\ell \\ &\overset{\text{Corollary 6.3.5}}{=\!=} r_k - \frac{d_k^T A r_k}{d_k^T A d_k} d_k = r_k + \frac{r_k^T \cdot r_k}{\widehat{\alpha}_k} \cdot \frac{1}{d_k^T A d_k} d_k \\ &= r_k + r_k^T \cdot r_k \cdot \frac{d_k^T A d_k}{d_k^T r_{k-1}} \cdot \frac{1}{d_k^T A d_k} d_k = r_k + \frac{r_k^T \cdot r_k}{d_k^T r_{k-1}} d_k \\ &\overset{\text{Corollary 6.3.4}}{=\!=} r_k + \frac{r_k^T \cdot r_k}{r_{k-1}^T r_{k-1}} d_k \end{aligned}$$

and

$$\frac{d_k^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k} \overset{\text{Corollary 6.3.4}}{=\!=} \frac{r_{k-1}^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k}$$

Hence Algorithm 10 can be simplified to Algorithm 11.

Algorithm 11: Conjugate gradient method (final version)

Input: $A \in \mathbb{R}^{n \times n}$ symmetric positive definite, right-hand side $b \in \mathbb{R}^n$, initial vector $x_0 \in \mathbb{R}^n$, tolerance tol
Output: approximation x_k to the solution of the linear system $Ax = b$
$r_0 \leftarrow b - A \cdot x_0$
$d_1 \leftarrow r_0$
$k \leftarrow 1$
while $\|r_{k-1}\|_2 = \sqrt{r_{k-1}^T \cdot r_{k-1}} > tol$ **do**
    $\alpha_k \leftarrow \frac{r_{k-1}^T \cdot r_{k-1}}{d_k^T \cdot A \cdot d_k}$
    $x_k \leftarrow x_{k-1} + \alpha_k \cdot d_k$
    $r_k \leftarrow r_{k-1} - \alpha_k \cdot A \cdot d_k$
    $d_{k+1} \leftarrow r_k + \frac{r_k^T \cdot r_k}{r_{k-1}^T \cdot r_{k-1}} \cdot d_k$
    $k \leftarrow k + 1$

Theorem 6.3.6 *Let $A \in \mathbb{R}^{n\times n}$ be symmetric and positive definite, $b \in \mathbb{R}^n$, and $\widehat{x} = A^{-1}\cdot b$. For the k-th iterate of the conjugate gradient method with initial value $x_0 \in \mathbb{R}^n$ we have*

$$\|\widehat{x} - x_k\|_A \leq 2 \cdot \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \cdot \|\widehat{x} - x_0\|_A \quad \textit{with} \quad \kappa = \kappa_2(A)$$

Proof: $\rightsquigarrow$Numerical Analysis II (Chebyshev polynomials). $\square$

Appendix A Supplements to the Chapters

§1 Supplement to Chapter 3

§1.1 Supplement to §1

A bit of fun with block matrices.

Example A.1.1 (Product of upper triangular matrices)

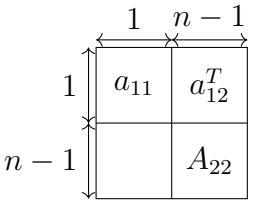
If A and B are upper triangular $n \times n$ matrices, then $A \cdot B$ is also an upper triangular matrix.

Proof: We prove this statement by induction on n.

Base case: $n = 1$
Obvious.

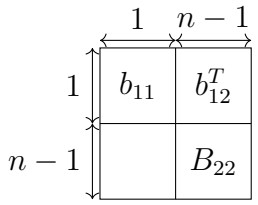
Induction step: $n - 1 \to n$
Consider

$$A = \begin{pmatrix} a_{11} & a_{12}^T \\ & A_{22} \end{pmatrix}$$



and

$$B = \begin{pmatrix} b_{11} & b_{12}^T \\ & B_{22} \end{pmatrix}.$$



Here A_{22} and B_{22} are upper triangular $(n-1) \times (n-1)$ matrices, and by the induction hypothesis $A_{22} \cdot B_{22}$ is also an upper triangular matrix. From

$$A \cdot B = \begin{pmatrix} a_{11} & a_{12}^T \\ & A_{22} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12}^T \\ & B_{22} \end{pmatrix} = \begin{pmatrix} a_{11} \cdot b_{11} & a_{11} \cdot b_{12}^T + a_{12}^T B_{22} \\ & A_{22} \cdot B_{22} \end{pmatrix},$$

it therefore follows that $A \cdot B$ is also an upper triangular matrix. ☐

Example A.1.2 (Inverse of an upper triangular matrix)

If A is an upper triangular $n \times n$ matrix and A^{-1} exists, then A^{-1} is also an upper triangular matrix.

Proof: We prove this statement by induction on n.

Base case: $n = 1$
Obvious.

Induction step: $n - 1 \to n$
Consider

$$A = \begin{pmatrix} a_{11} & a_{12}^T \\ & A_{22} \end{pmatrix}$$

and

$$A^{-1} = \begin{pmatrix} b_{11} & b_{12}^T \\ b_{21} & B_{22} \end{pmatrix}.$$

By the induction hypothesis, A_{22}^{-1} is an upper triangular $(n-1) \times (n-1)$ matrix. From

$$\begin{pmatrix} 1 & \\ & I_{n-1} \end{pmatrix} = I_n = A \cdot A^{-1} = \begin{pmatrix} a_{11} & a_{12}^T \\ & A_{22} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12}^T \\ b_{21} & B_{22} \end{pmatrix}$$
$$= \begin{pmatrix} a_{11} \cdot b_{11} + a_{12}^T \cdot b_{21} & a_{11} \cdot b_{12}^T + a_{12}^T B_{22} \\ A_{22} \cdot b_{21} & A_{22} \cdot B_{22} \end{pmatrix}$$

it follows that

$$A_{22} \cdot b_{21} = \mathbf{0} \quad \text{and} \quad A_{22} \cdot B_{22} = I_{n-1}.$$

Due to the existence and uniqueness of A_{22}^{-1} we obtain

$$b_{21} = \mathbf{0} \quad \text{and} \quad B_{22} = A_{22}^{-1}.$$

□

The proof can easily be extended to show that the diagonal entries of A^{-1} are the reciprocals of the diagonal entries of A.

Example A.1.3 (Forward substitution (column-wise algorithm))

Let A be an invertible lower triangular $n \times n$ matrix. To solve $Ax = b$, one can first consider

$$A = \begin{pmatrix} a_{11} & \\ \mathbf{a}_{21} & A_{22} \end{pmatrix} \qquad \begin{array}{c} \\ 1 \\ n-1 \end{array} \begin{array}{c} \begin{array}{cc} 1 & n-1 \end{array} \\ \begin{array}{|c|c|} \hline a_{11} & \\ \hline \mathbf{a}_{21} & A_{22} \\ \hline \end{array} \end{array} \qquad \text{with } \mathbf{a}_{21} = \begin{pmatrix} a_{2,1} \\ \vdots \\ a_{n,1} \end{pmatrix}$$

$$x = \begin{pmatrix} x_1 \\ \mathbf{x}_2 \end{pmatrix} \qquad \begin{array}{c} \\ 1 \\ n-1 \end{array} \begin{array}{c} \begin{array}{c} 1 \end{array} \\ \begin{array}{|c|} \hline x_1 \\ \hline \mathbf{x}_2 \\ \hline \end{array} \end{array} \qquad \text{with } \mathbf{x}_2 = \begin{pmatrix} x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$b = \begin{pmatrix} b_1 \\ \mathbf{b}_2 \end{pmatrix} \qquad \begin{array}{c} \\ 1 \\ n-1 \end{array} \begin{array}{c} \begin{array}{c} 1 \end{array} \\ \begin{array}{|c|} \hline b_1 \\ \hline \mathbf{b}_2 \\ \hline \end{array} \end{array} \qquad \text{with } \mathbf{b}_2 = \begin{pmatrix} b_2 \\ \vdots \\ b_n \end{pmatrix}$$

To simplify the notation and to avoid name clashes, boldface symbols have been used for vectors here. From

$$\begin{pmatrix} b_1 \\ \mathbf{b}_2 \end{pmatrix} = b = A \cdot x = \begin{pmatrix} a_{11} & \\ \mathbf{a}_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ \mathbf{x}_2 \end{pmatrix} = \begin{pmatrix} a_{11} \cdot x_1 \\ \mathbf{a}_{21} \cdot x_1 + A_{22} \cdot \mathbf{x}_2 \end{pmatrix}$$

it follows that

$$x_1 = \frac{b_1}{a_{11}}, \; A_{22} \cdot \mathbf{x}_2 = \mathbf{b}_2 - \mathbf{a}_{21} \cdot x_1.$$

From this one can develop an algorithm that performs the following operations in the first step:

$$x_1 \leftarrow \tfrac{b_1}{a_{11}},$$

$$\mathbf{b}_2 \leftarrow \mathbf{b}_2 - \mathbf{a}_{21} \cdot x_1, \qquad \text{i.e.} \begin{pmatrix} b_2 \\ \vdots \\ b_n \end{pmatrix} \leftarrow \begin{pmatrix} b_2 - a_{21} \cdot x_1 \\ \vdots \\ b_n - a_{n1} \cdot x_1 \end{pmatrix}.$$

In the subsequent steps, $A_{22}\mathbf{x}_2 = \mathbf{b}_2$ is then solved by proceeding analogously.
In MATLAB this can be implemented directly as follows:

```
1 function x = forward_solve_colwise(A, b)
2     n = size(A, 2);
3     x = zeros(n, 1);
4     for j = 1 : n
5         x(j) = b(j) / A(j,j);
6         b(j+1:n) = b(j+1:n) - A(j+1:n, j) * x(j);
7     end
8 end
```

Moreover, one can rewrite the algorithm and the MATLAB code such that A, x and b are accessed componentwise:

```
1 for j = 1 to n do
2 |   x_j ← b_j / a_jj;
3 |   for i = j + 1 to n do
4 |   |   b_i ← b_i − a_ij · x_j;
```

Example A.1.4 (Forward substitution (row-wise algorithm))

Again let A be an invertible lower triangular $n \times n$ matrix. To solve $Ax = b$, we consider, for some i with $1 \leq i \leq n$, the leading $i \times i$ principal submatrix

$$\begin{pmatrix} A_{11} & \\ \mathbf{a}_{i1}^T & a_{ii} \end{pmatrix} \qquad \begin{array}{c} \\ i-1 \\ 1 \end{array} \overset{\begin{array}{cc} i-1 & 1 \end{array}}{\begin{array}{|c|c|} \hline A_{11} & \\ \hline \mathbf{a}_{i1}^T & a_{ii} \\ \hline \end{array}} \quad \text{with } \mathbf{a}_{i1}^T = \begin{pmatrix} a_{i,1} & \dots & a_{i,i-1} \end{pmatrix}$$

as well as

$$\begin{pmatrix} \mathbf{x}_1 \\ x_i \end{pmatrix} \qquad \begin{array}{c} \\ i-1 \\ 1 \end{array} \overset{\begin{array}{c} 1 \end{array}}{\begin{array}{|c|} \hline \mathbf{x}_1 \\ \hline x_i \\ \hline \end{array}} \quad \text{with } \mathbf{x}_1 = \begin{pmatrix} x_1 \\ \vdots \\ x_{i-1} \end{pmatrix}$$

$$\begin{pmatrix} \mathbf{b}_1 \\ b_i \end{pmatrix} \qquad \begin{array}{c} \\ i-1 \\ 1 \end{array} \overset{\begin{array}{c} 1 \end{array}}{\begin{array}{|c|} \hline \mathbf{b}_1 \\ \hline b_i \\ \hline \end{array}} \quad \text{with } \mathbf{b}_1 = \begin{pmatrix} b_1 \\ \vdots \\ b_{i-1} \end{pmatrix}$$

Then

$$\begin{pmatrix} \mathbf{b}_1 \\ b_i \end{pmatrix} = \begin{pmatrix} A_{11} & \\ \mathbf{a}_{i1}^T & a_{ii} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x}_1 \\ x_i \end{pmatrix} = \begin{pmatrix} A_{11} \cdot \mathbf{x}_1 \\ \mathbf{a}_{i1}^T \cdot \mathbf{x}_1 + a_{ii} \cdot x_i \end{pmatrix}.$$

Hence, if the solution of $A_{11}\mathbf{x}_1 = \mathbf{b}_1$ is known, one can compute

$$x_i = \tfrac{1}{a_{ii}} \left(b_i - \mathbf{a}_{i1}^T \cdot \mathbf{x}_1 \right)$$

In this way one can successively compute $x_1, \dots, x_n$. In MATLAB this can be implemented directly as follows:

```
1 function x = forward_solve_rowwise(A, b)
2     n = size(A, 2);
3     x = zeros(n, 1);
4     for i = 1 : n
5         x(i) = (b(i) - A(i, 1:i-1) * x(1:i-1)) / A(i,i);
6     end
7 end
```

Of course, here too one can derive a componentwise algorithm that, for each i, performs the computation

$$x_i = \tfrac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j \right)$$

by working through the individual steps

$$\begin{aligned} x_i &\leftarrow b_i, \\ x_i &\leftarrow x_i - a_{i1} \cdot x_1, \\ &\vdots \\ x_i &\leftarrow x_i - a_{i,i-1} \cdot x_{i-1}, \\ x_i &\leftarrow x_i / a_{ii} \end{aligned}$$

This yields the following algorithm:

```
1 for i = 1 to n do
2 |   x_i ← b_i;
3 |   for j = 1 to i − 1 do
4 |   |   x_i ← x_i − a_ij · x_j;
5 |   x_i ← x_i / a_ii;
```

Appendix B Linear Algebra Miscellany

Sources and further reading. This appendix, in particular the sections on norms, inner products and completeness, makes frequent use of Kreyszig [14], where these topics are treated in the general setting of normed and inner product spaces.

§1 Principal Axis Transformation

Theorem B.1.1 (Principal axis transformation (PAT))

Let $A \in \mathbb{R}^{n\times n}$. Then the following statements are equivalent:

(1) $A = A^T$, *i.e., A is symmetric*

(2) $\exists\, Q, D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{R}^{n\times n} :\ A = Q \cdot D \cdot Q^T \quad$ *and* $\quad Q^T \cdot Q = I$, *i.e., A is orthogonally diagonalizable.*

Remark B.1.2

(a) In Theorem B.1.1 the columns of Q are orthogonal to each other and normalized $\big(|x| := \sqrt{x \bullet x}\big)$ with respect to the Euclidean inner product $\big(x \bullet y := x^T \cdot y\big)$, because, considering Q column by column, we have

$$Q^T \cdot Q = \begin{pmatrix} q_1^T \\ \vdots \\ q_n^T \end{pmatrix} \cdot (q_1, \ldots, q_n) = \begin{pmatrix} q_1^T \cdot q_1 & \ldots & q_1^T \cdot q_n \\ \vdots & & \vdots \\ q_n^T \cdot q_1 & \ldots & q_n^T \cdot q_n \end{pmatrix}$$

$$= \begin{pmatrix} 1 & & \mathbf{0} \\ & \ddots & \\ \mathbf{0} & & 1 \end{pmatrix} = \operatorname{diag}(1, \ldots, 1) = I$$

Thus

$$q_i \bullet q_i = q_i^T \cdot q_i = 1 \ \forall i \quad \text{and} \quad q_i \bullet q_j = q_i^T \cdot q_j = 0 \ \forall i \neq j$$

and

$$Q^T \cdot q_i = e_i \ \forall i$$

(b) From (a) it follows that $Q^{-1} = Q^T$ and $q_i \neq \mathbf{0}$ for $1 \leq i \leq n$.

(c) The columns of Q are eigenvectors of A, and the diagonal of D contains the corresponding eigenvalues. Because of

$$A = Q \cdot D \cdot Q^T \quad \overset{(b)}{\Leftrightarrow} \quad A \cdot Q = Q \cdot D \cdot Q^T \cdot Q = Q \cdot D$$

we have

$$\begin{aligned} A \cdot Q &= A \cdot (q_1, \ldots, q_n) = (A \cdot q_1, \ldots, A \cdot q_n) \\ &\quad || \\ Q \cdot D &= (q_1, \ldots, q_n) \cdot \mathrm{diag}(\lambda_1, \ldots, \lambda_n) = (\lambda_1 \cdot q_1, \ldots, \lambda_n \cdot q_n) \end{aligned}$$

For $i \in \{1, \ldots, n\}$ we therefore have $A \cdot q_i = \lambda_i \cdot q_i$ and, as noted in (b), $q_i \neq \mathbf{0}$. Hence q_i is an eigenvector of A for the eigenvalue λ_i.

(d) In general, a real matrix can have complex eigenvalues. The characteristic polynomial can have complex roots even though all its coefficients are real.

A special feature that follows from the principal axis transformation is that this cannot happen for a real symmetric matrix. The theorem states that

$$\lambda_i \in \mathbb{R} \quad \text{for} \quad 1 \leq i \leq n.$$

The fact that all eigenvalues of a real symmetric matrix are real can be proven directly and serves as a tool for the proof of the principal axis transformation.

Lemma B.1.3 (Eigenvalues of a real symmetric matrix)

Let $A \in \mathbb{R}^{n \times n}$ with $A = A^T$. Then all eigenvalues of A are real.

Proof: Let λ be an eigenvalue of A with corresponding eigenvector v, i.e.,

$$\lambda \in \mathbb{C},\ v \in \mathbb{C}^n \setminus \{\mathbf{0}\} \quad \text{with} \quad A \cdot v = \lambda \cdot v.$$

Since $A = A^T$, it follows that

$$\begin{aligned} \overline{v}^T \cdot A \cdot v &= \overline{v}^T \cdot (\lambda \cdot v) = \lambda \cdot \overline{v}^T \cdot v, \\ &\quad || \\ \overline{v}^T \cdot A^T \cdot v &= (\overline{A \cdot v})^T \cdot v = (\overline{\lambda} \cdot \overline{v})^T \cdot v = \overline{\lambda} \cdot \overline{v}^T \cdot v. \end{aligned}$$

This implies

$$\lambda \cdot \overline{v}^T \cdot v = \overline{\lambda} \cdot \overline{v}^T \cdot v. \tag{B.1}$$

Because of

$$\overline{v}^T \cdot v = \sum_{i=1}^{n} \overline{v}_i \cdot v_i = \sum_{i=1}^{n} |v_i|^2 > 0 \quad (\text{since } v \neq \mathbf{0}),$$

we can divide equation (B.1) by $\overline{v}^T \cdot v$ and obtain

$$\lambda = \overline{\lambda} \quad \Longleftrightarrow \quad \lambda \in \mathbb{R}.$$

$\square$

Remark B.1.4 (Further tools for the proof of the PAT)

(a) The characteristic polynomial

$$p_A(\lambda) := \det(A - \lambda \cdot I)$$

of an $n \times n$ matrix has degree n. By the fundamental theorem of algebra, p_A has at least one root in $\mathbb{C}$.

(b) If A is a real matrix and λ a real eigenvalue, then there exists a corresponding real eigenvector, i.e.,

$$A \cdot v = \lambda \cdot v, \quad \lambda \in \mathbb{R},\ v \in \mathbb{R}^n \setminus \{\mathbf{0}\}.$$

(c) By the basis extension theorem, a vector $v \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ can be extended to a basis of $\mathbb{R}^n$. If one uses the basis vectors as the columns of a matrix W, then this matrix is invertible. Thus

$$v \in \mathbb{R}^n \setminus \{\mathbf{0}\} \implies \exists w_2, \ldots, w_n \in \mathbb{R}^n : \text{ for } W := (v, w_2, \ldots, w_n) \text{ there exists } W^{-1}.$$

(d) With Gram–Schmidt orthogonalization one can extend a vector $v \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ to an orthogonal basis of $\mathbb{R}^n$. Moreover, this basis can be normalized. If, for the vectors from (c), one defines

$$v_1 := \frac{1}{\sqrt{v^T \cdot v}} \cdot v$$

and, for $k = 2, \ldots, n$,

$$\widetilde{w}_k := w_k - \sum_{\ell=1}^{k-1} \left(v_\ell^T \cdot w_k\right) \cdot v_\ell, \quad v_k := \frac{1}{\sqrt{\widetilde{w}_k^T \cdot \widetilde{w}_k}} \cdot \widetilde{w}_k,$$

then

$$V := (v_1, \ldots, v_n)$$

is an orthogonal matrix, i.e.,

$$V^{-1} = V^T.$$

Proof of Theorem B.1.1 (principal axis transformation):

(1) ⇒ (2)

Base case: $n = 1$
Choose $Q = (1)$ and $D = A = (a_{11})$.

Induction step: $n-1 \to n$

By Remark B.1.4 (a), A has an eigenvalue $\lambda \in \mathbb{C}$; by the preceding lemma on the eigenvalues of real symmetric matrices, $\lambda \in \mathbb{R}$, and by Remark B.1.4 (b) there exists a corresponding real eigenvector. Hence there exists an eigenvalue $\lambda \in \mathbb{R}$ with corresponding eigenvector $v \in \mathbb{R}^n \backslash \{\mathbf{0}_n\}$. By Remark B.1.4 (d), the normalized eigenvector

$$v_1 := \frac{1}{\sqrt{v^T \cdot v}} \cdot v$$

can be extended to an orthogonal matrix $V = (v_1, v_2, \ldots, v_n)$. Then

$$\begin{aligned} V^T \cdot A \cdot V &= V^T \cdot A \cdot (v_1,\ v_2, \ldots,\ v_n) \\ &= \left(\lambda \cdot V^T \cdot v_1,\ V^T \cdot A \cdot v_2,\ \ldots,\ V^T \cdot A \cdot v_n\right) \\ &= \begin{pmatrix} \lambda & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix} \end{aligned} \tag{B.2}$$

Because of

$$(V^T \cdot A \cdot V)^T = V^T \cdot A^T \cdot V = V^T \cdot A \cdot V$$

we have

$$\begin{pmatrix} \lambda & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix} = \begin{pmatrix} \lambda & \widetilde{a}_{12}^T \\ & \widetilde{A}_{22} \end{pmatrix}^T = \begin{pmatrix} \lambda & \\ \widetilde{a}_{12} & \widetilde{A}_{22}^T \end{pmatrix}$$

Comparing the blocks yields

$$\widetilde{a}_{12} = \mathbf{0}_{n-1} \tag{B.3}$$

$$\widetilde{A}_{22}^T = \widetilde{A}_{22} \tag{B.4}$$

From (B.4) it follows that the $(n-1) \times (n-1)$ matrix $\widetilde{A}_{22}$ is symmetric, and hence by the induction hypothesis there exist real $(n-1) \times (n-1)$ matrices $\widetilde{Q}$ and $\widetilde{D}$ such that

$$\widetilde{A}_{22} = \widetilde{Q} \cdot \widetilde{D} \cdot \widetilde{Q}^T, \quad \widetilde{Q}^T \cdot \widetilde{Q} = I_{n-1}, \quad \widetilde{D} = \operatorname{diag}\left(\widetilde{\lambda}_1, \ldots, \widetilde{\lambda}_{n-1}\right) \tag{B.5}$$

holds. Substituting (B.3) and (B.5) into (B.2), we first obtain

$$V^T \cdot A \cdot V = \begin{pmatrix} \lambda & \\ & \widetilde{Q} \cdot \widetilde{D} \cdot \widetilde{Q}^T \end{pmatrix} = \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix} \cdot \begin{pmatrix} \lambda & \\ & \widetilde{D} \end{pmatrix} \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}^T$$

and, after a further rearrangement,

$$A = \underbrace{\left[V \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}\right]}_{=:Q} \cdot \underbrace{\begin{pmatrix} \lambda & \\ & \widetilde{D} \end{pmatrix}}_{=:D} \cdot \left[V \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}\right]^T = Q \cdot D \cdot Q^T.$$

That Q, as a product of orthogonal matrices, is also orthogonal can be confirmed by the following calculation:

$$\begin{aligned} Q^T \cdot Q &= \left[V \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}\right]^T \cdot \left[V \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix}\right] = \begin{pmatrix} 1 & \\ & \widetilde{Q}^T \end{pmatrix} \underbrace{V^T \cdot V}_{=I_n} \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix} \\ &= \begin{pmatrix} 1 & \\ & \widetilde{Q}^T \end{pmatrix} \cdot \begin{pmatrix} 1 & \\ & \widetilde{Q} \end{pmatrix} = \begin{pmatrix} 1 & \\ & \widetilde{Q}^T \cdot \widetilde{Q} \end{pmatrix} = I \end{aligned}$$

(2) ⇒ (1)

$$A = Q \cdot D \cdot Q^T \quad \Longrightarrow \quad A^T = \left(Q \cdot D \cdot Q^T\right)^T = Q \cdot D^T \cdot Q^T \overset{D^T = D}{=\!=\!=} A$$

□

§2 The Spectral Theorem

Remark B.2.1

(a) In the following, the notation $A^H := \overline{A}^T$ is used.

(b) For vectors $x, y \in \mathbb{C}^n$, $\langle x, y\rangle := x^H \cdot y$ can be regarded as an inner product if the axioms are stated as follows:

(S1) For all $x \in \mathbb{C}^n$: $\langle x, x\rangle \geq 0$ and

$$\langle x, x\rangle = 0 \;\Leftrightarrow\; x = \mathbf{0}.$$

(S2) For all $x, y, z \in \mathbb{C}^n$ and all $\alpha, \beta \in \mathbb{C}$:

$$\langle x, \alpha \cdot y + \beta \cdot z\rangle = \alpha \cdot \langle x, y\rangle + \beta \cdot \langle x, z\rangle.$$

(S3) For all $x, y \in \mathbb{C}^n$:

$$\langle x, y\rangle = \overline{\langle y, x\rangle}.$$

By writing $\langle x, x\rangle \geq 0$, (S1) implicitly requires that $\langle x, x\rangle \in \mathbb{R}$ always holds. Because of

$$\langle x, x\rangle = x^H \cdot x = \sum_{\ell=1}^{n} \overline{x}_\ell \cdot x_\ell = \sum_{\ell=1}^{n} |x_\ell|^2$$

this is satisfied.

(c) The inner product defined in (b) induces a norm via

$$\|x\| := \sqrt{\langle x, x\rangle} := \sqrt{x^H \cdot x}.$$

(d) At some points in the proofs, the transformation

$$x^T \cdot \overline{x} = \overline{x^H \cdot x} \overset{x^H \cdot x \in \mathbb{R}}{=\!=\!=} x^H \cdot x = \|x\|^2$$

is used.

(e) In Remark B.1.4 (c) the basis extension theorem was used for $\mathbb{R}^n$. Of course, the theorem also holds for $\mathbb{C}^n$. Hence:

$$v \in \mathbb{C}^n \setminus \{\mathbf{0}\} \;\Longrightarrow\; \exists w_2, \ldots, w_n \in \mathbb{C}^n : \; W := (v, w_2, \ldots, w_n) \text{ is invertible.}$$

(f) In Remark B.1.4 (d) Gram–Schmidt orthogonalization was used to orthogonalize vectors in $\mathbb{R}^n$. The procedure works analogously in every vector space on which an inner product is defined.

If, for the vectors from (e), one defines

$$v_1 := \frac{1}{\|v\|} \cdot v$$

and, for $k = 2, \ldots, n$,

$$\widetilde{w}_k := w_k - \sum_{\ell=1}^{k-1} \langle v_\ell, w_k \rangle \cdot v_\ell, \quad v_k := \frac{1}{\|\widetilde{w}_k\|} \cdot \widetilde{w}_k,$$

then

$$V := (v_1, \ldots, v_n)$$

is a unitary matrix, i.e.:

$$V^{-1} = V^H.$$

Lemma B.2.2 (Schur decomposition)

Let $A \in \mathbb{C}^{n \times n}$. Then there exist an upper triangular matrix $T \in \mathbb{C}^{n \times n}$ and a unitary matrix $U \in \mathbb{C}^{n \times n}$ with

$$A = U \cdot T \cdot U^H \quad \text{and} \quad U^H \cdot U = I.$$

Proof:

Base case: $n = 1$
Choose $U = (1)$ and $T = A$.

Induction step: $n - 1 \to n$
Let λ be an eigenvalue of A with corresponding eigenvector v. Then let the normalized eigenvector

$$v_1 := \frac{1}{\|v\|} \cdot v$$

be extended to a unitary matrix $V := (v_1, v_2, \ldots, v_n)$ such that $V^H \cdot V = I$. Then:

$$\begin{aligned}
V^H \cdot A \cdot V &= V^H \cdot A \cdot (v_1, v_2, \ldots, v_n) \\
&= \big(V^H \cdot \underbrace{A \cdot v_1}_{=\lambda \cdot v_1}, \; V^H \cdot A \cdot v_2, \; \ldots, \; V^H \cdot A \cdot v_n\big) \\
&= \big(\lambda \cdot V^H \cdot v_1, \; V^H \cdot A \cdot v_2, \; \ldots, \; V^H \cdot A \cdot v_n\big) \\
&= \big(\lambda \cdot e_1, \; V^H \cdot A \cdot v_2, \; \ldots, \; V^H \cdot A \cdot v_n\big) \\
&= \begin{pmatrix} \lambda & \widetilde{a}^T \\ & \widetilde{A} \end{pmatrix},
\end{aligned}$$

where $\widetilde{A} \in \mathbb{C}^{(n-1)\times(n-1)}$ and $\widetilde{a} \in \mathbb{C}^{(n-1)}$. By the induction hypothesis (IH),

$$\widetilde{A} = \widetilde{U} \cdot \widetilde{T} \cdot \widetilde{U}^H,$$

where $\widetilde{T}$ is an upper triangular matrix and $\widetilde{U}^H \cdot \widetilde{U} = I_{n-1}$. It follows that:

$$\begin{pmatrix} 1 & \\ & \widetilde{U}^H \end{pmatrix} \cdot V^H \cdot A \cdot V \cdot \begin{pmatrix} 1 & \\ & \widetilde{U} \end{pmatrix} = \begin{pmatrix} \lambda & \widetilde{a}^T \cdot \widetilde{U} \\ & \widetilde{T} \end{pmatrix}.$$

Define

$$U := V \cdot \begin{pmatrix} 1 & \\ & \widetilde{U} \end{pmatrix}.$$

Then

$$U^H \cdot A \cdot U = \begin{pmatrix} \lambda & \widetilde{a}^T \cdot \widetilde{U} \\ & \widetilde{T} \end{pmatrix} = T,$$

where T is an upper triangular matrix.

Finally, one shows that U is unitary:

$$\begin{aligned} U^H \cdot U &= \begin{pmatrix} 1 & \\ & \widetilde{U}^H \end{pmatrix} \cdot V^H \cdot V \cdot \begin{pmatrix} 1 & \\ & \widetilde{U} \end{pmatrix} \\ &= \begin{pmatrix} 1 & \\ & \widetilde{U}^H \cdot \widetilde{U} \end{pmatrix} = I. \end{aligned}$$

This proves the claim. ⊿

Definition B.2.3 (Normal matrices)

A matrix $A \in \mathbb{C}^{n \times n}$ with the property $A \cdot A^H = A^H \cdot A$ is called a normal matrix.

Lemma B.2.4 (Normal triangular matrices are diagonal matrices)

If $T \in \mathbb{C}^{n \times n}$ is an upper triangular matrix and $T^H \cdot T = T \cdot T^H$ holds, then T is a diagonal matrix.

Proof:

Base case: $n = 1$
Every 1×1 matrix is obviously a diagonal matrix.

Induction step: $n - 1 \to n$
Consider the block structure of T:

$$T = \begin{pmatrix} t_{11} & t_{12}^T \\ & T_{22} \end{pmatrix}$$

with $t_{11} \in \mathbb{C}$, $t_{12} \in \mathbb{C}^{n-1}$, and $T_{22} \in \mathbb{C}^{(n-1)\times(n-1)}$, where T_{22} is an upper triangular matrix.
Compute:

$$\begin{aligned} T \cdot T^H &= \begin{pmatrix} t_{11} & t_{12}^T \\ & T_{22} \end{pmatrix} \cdot \begin{pmatrix} \overline{t_{11}} & \\ \overline{t_{12}} & T_{22}^H \end{pmatrix} = \begin{pmatrix} t_{11} \cdot \overline{t_{11}} + t_{12}^T \cdot \overline{t_{12}} & t_{12}^T \cdot T_{22}^H \\ T_{22} \cdot \overline{t_{12}} & T_{22} \cdot T_{22}^H \end{pmatrix}, \\ || & \\ T^H \cdot T &= \begin{pmatrix} \overline{t_{11}} & \\ \overline{t_{12}} & T_{22}^H \end{pmatrix} \cdot \begin{pmatrix} t_{11} & t_{12}^T \\ & T_{22} \end{pmatrix} = \begin{pmatrix} \overline{t_{11}} \cdot t_{11} & \overline{t_{11}} \cdot t_{12}^T \\ \overline{t_{12}} \cdot t_{11} & \overline{t_{12}} \cdot t_{12}^T + T_{22}^H \cdot T_{22} \end{pmatrix}. \end{aligned}$$

Compare the diagonal blocks:

$$\underbrace{\overline{t_{11}} \cdot t_{11}}_{=|t_{11}|^2} = \underbrace{t_{11} \cdot \overline{t_{11}}}_{=|t_{11}|^2} + \underbrace{t_{12}^T \cdot \overline{t_{12}}}_{=\|t_{12}\|^2}, \tag{B.6}$$

$$T_{22} \cdot T_{22}^H = \overline{t_{12}} \cdot t_{12}^T + T_{22}^H \cdot T_{22}. \tag{B.7}$$

From (B.6) it follows that $\|t_{12}\|^2 = 0$, and hence $t_{12} = \mathbf{0}$. Substituting this into (B.7) also yields:

$$T_{22} \cdot T_{22}^H = T_{22}^H \cdot T_{22}.$$

By the induction hypothesis, the upper triangular $(n-1)\times(n-1)$ matrix T_{22} is a diagonal matrix, and hence so is T. ⍁

Theorem B.2.5 (Spectral theorem)

For a matrix $A \in \mathbb{C}^{n\times n}$ the following statements are equivalent:

(1) $A^H \cdot A = A \cdot A^H$, *i.e., A is normal.*

(2) $\exists U \in \mathbb{C}^{n\times n},\ D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{C}^{n\times n}: \quad A = U \cdot D \cdot U^H \quad$ *and* $\quad U^H \cdot U = I.$

Proof:

(1) ⇒ (2)
From Lemma B.2.2 it follows that

$$\exists U,\ T \in \mathbb{C}^{n\times n}: \quad A = U \cdot T \cdot U^H,\ U^H \cdot U = I \text{ and } T \text{ is an upper triangular matrix.}$$

Since $A^H \cdot A = A \cdot A^H$ holds, it follows that

$$A^H \cdot A = \left(U \cdot T \cdot U^H\right)^H \cdot U \cdot T \cdot U^H = U \cdot T^H \cdot U^H \cdot U \cdot T \cdot U^H = U \cdot T^H \cdot T \cdot U^H,$$

$$\| \qquad\qquad\qquad\

This shows that $A^H \cdot A = A \cdot A^H$ holds. ∎

Corollary B.2.6 (Special spectral theorem for Hermitian matrices)

For a matrix $A \in \mathbb{C}^{n\times n}$ the following statements are equivalent:

(1) $A^H = A$, i.e., A is Hermitian.

(2) $\exists U \in \mathbb{C}^{n\times n},\ D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{R}^{n\times n}: \quad A = U \cdot D \cdot U^H \quad$ and $\quad U^H \cdot U = I.$

Proof: **(1) ⇒ (2)**

Obviously, a Hermitian matrix is normal, since $A^H \cdot A = A \cdot A = A \cdot A^H$ holds. The spectral theorem therefore yields

$$\exists U,\ D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{C}^{n\times n}: \quad A = U \cdot D \cdot U^H \quad \text{and} \quad U^H \cdot U = I.$$

But then $A = A^H$ also implies

$$\begin{aligned} A &= U \cdot D \cdot U^H \\ &\| \\ A^H &= U \cdot D^H \cdot U^H \end{aligned}$$

and therefore

$$D = D^H \iff \operatorname{diag}(\lambda_1, \ldots, \lambda_n) = \operatorname{diag}(\overline{\lambda}_1, \ldots, \overline{\lambda}_n) \iff \lambda_1, \ldots, \lambda_n \in \mathbb{R}$$

(2) ⇒ (1)

$$A = U \cdot D \cdot U^H \quad \Longrightarrow \quad A^H = \left(U \cdot D \cdot U^H\right)^H = U \cdot D^H \cdot U^H \overset{D^H = D}{=\!=} A$$

∎

§3 Norms in Finite-Dimensional Vector Spaces

This section primarily summarizes the most important properties of finite-dimensional normed vector spaces that are particularly relevant for numerical linear algebra. Although a special case is the focus here, basic definitions, such as that of a norm, are kept as general as possible. The reason is that in numerical analysis infinite-dimensional vector spaces are particularly interesting, but they are more difficult to handle. To understand what is meant by "more difficult", it is worthwhile to first consider the supposedly simpler case (which is done in this section). Afterwards, one can investigate how the concepts summarized here carry over to infinite-dimensional vector spaces.

§3.1 Basic Definitions

Definition B.3.1 (Finite- and infinite-dimensional vector spaces)

A vector space V over $\mathbb{K}$ is finite-dimensional if there is an $n \in \mathbb{N}$ such that a subset of n linearly independent vectors exists and any $n+1$ vectors are always linearly dependent. The number n is called the dimension of V, and one writes $\dim V = n$. In addition, one defines that the trivial vector space $V = \{\mathbf{0}\}$ is finite-dimensional with $\dim V = 0$.

If a vector space is not finite-dimensional, one says that V is infinite-dimensional and writes $\dim V = \infty$.

Definition B.3.2 (Norm)

Let V be a vector space over $\mathbb{K} = \mathbb{R}$ (real vector space) or $\mathbb{K} = \mathbb{C}$ (complex vector space). A norm is a real-valued function on V whose value at $x \in V$ is denoted by $\|x\|$, i.e.

$$\|\cdot\| : V \to \mathbb{R}$$

and which, for arbitrary $x, y \in V$ and $\alpha \in \mathbb{K}$, satisfies the following properties:

(N1) $\|x\| \geq 0$ *(nonnegativity)*

(N2) $\|x\| = 0 \quad \Longleftrightarrow \quad x = \mathbf{0}$ *(definiteness)*

(N3) $\|\alpha x\| = |\alpha| \cdot \|x\|$ *(homogeneity)*

(N4) $\|x + y\| \leq \|x\| + \|y\|$ *(triangle inequality)*

V is then called a normed vector space.

Example B.3.3 (Norms on $\mathbb{R}^n$ and $\mathbb{C}^n$)

First, a few examples of different norms on $V = \mathbb{R}^n$ with $\mathbb{K} = \mathbb{R}$ or $V = \mathbb{C}^n$ with $\mathbb{K} = \mathbb{C}$. The verification that the norm axioms are satisfied is omitted here. The only goal is to illustrate that there is more than one way to define a norm on a vector space.

(a) The norms defined by

$$\|x\|_1 := \sum_{i=1}^{n} |x_i|, \quad \|x\|_2 := \sqrt{\sum_{i=1}^{n} |x_i|^2} \quad \text{and} \quad \|x\|_\infty := \max_{1 \leq i \leq n} |x_i|$$

are particularly important in numerical linear algebra. More generally, for $1 \leq p \leq \infty$ these norms can be defined by

$$\|x\|_p := \begin{cases} \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}}, & \text{for } 1 \leq p < \infty, \\ \max_{1 \leq i \leq n} |x_i|, & \text{for } p = \infty \end{cases}$$

These are called p-norms.

(b) If $\omega_1, \ldots, \omega_n > 0$, then

$$\|x\|_\omega := \sum_{i=1}^{n} \omega_i \cdot |x_i|$$

defines a weighted norm. An important application of these norms lies in improving the condition of linear systems, e.g., by row and column scaling, in order to reduce numerical instabilities. Moreover, they are used in optimization theory, e.g., in *weighted least squares*.

(c) In optimization theory in particular, norms defined as combinations of p-norms are also of interest. For example, the norm defined by

$$\|x\| := \|x\|_1 + \|x\|_\infty,$$

which is used in mixed error criteria.

§3.2 Equivalence of Norms: Motivation and Definition

Before the notion of equivalence of norms is defined in general in Definition B.3.4 and subsequently treated in general, its practical significance will be motivated and verified for the p-norms. We first show that

$$\|x\|_\infty \leq \|x\|_1 \leq n \cdot \|x\|_\infty \tag{B.8}$$

holds. In numerical linear algebra, the practical benefit could be that a method produces a sequence $\left(x^{(k)}\right)_{k\in\mathbb{N}}$ for which one can easily show that it converges to some $\widehat{x}$ with respect to the 1-norm. In fact, however, one is interested in the sequence converging with respect to the ∞-norm because, for example, one wants to estimate when the maximum deviation of the components from the limit falls below a given tolerance. A direct proof of convergence with respect to the ∞-norm might be more difficult, but it is not necessary, since by (B.8) it follows directly from convergence with respect to the 1-norm. We have:

$$\begin{aligned}
x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_1} \widehat{x} &\Leftrightarrow \lim_{k\to\infty} \|x^{(k)} - \widehat{x}\|_1 = 0 \\
&\Leftrightarrow \forall \varepsilon > 0\ \exists k(\varepsilon): \quad \underbrace{\|x^{(k)} - \widehat{x}\|_1}_{\geq \|x^{(k)} - \widehat{x}\|_\infty} < \varepsilon \quad \forall k \geq k(\varepsilon) \\
&\Rightarrow \lim_{k\to\infty} \|x^{(k)} - \widehat{x}\|_\infty = 0 \qquad (*) \\
&\Leftrightarrow x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_\infty} \widehat{x}.
\end{aligned}$$

For the application described, the implication

$$x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_1} \widehat{x} \ \Rightarrow\ x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_\infty} \widehat{x}$$

is sufficient. However, it is easy to show that the converse also holds, and hence the equivalence of convergence with respect to the 1-norm and the ∞-norm, since

$$\begin{aligned} x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_\infty} \widehat{x} &\Leftrightarrow \lim_{k\to\infty} \|x^{(k)} - \widehat{x}\|_\infty = 0 \\ &\Leftrightarrow \forall \varepsilon > 0 \; \exists k(\varepsilon): \quad \underbrace{\|x^{(k)} - \widehat{x}\|_\infty}_{\geq \frac{1}{n}\cdot\|x^{(k)}-\widehat{x}\|_1} < \varepsilon \quad \forall k \geq k(\varepsilon) \\ &\Rightarrow \text{For } \varepsilon > 0\text{: } \|x^{(k)} - \widehat{x}\|_1 < \varepsilon \quad \forall k \geq k\left(\frac{\varepsilon}{n}\right) \\ &\Leftrightarrow x^{(k)} \xrightarrow[k\to\infty]{\|\cdot\|_1} \widehat{x}. \end{aligned}$$

In general, two norms are equivalent if each can be bounded by the other up to constant factors. This is made precise in the following definition.

Definition B.3.4 (Equivalence of norms) *Two norms $\|\cdot\|$ and $|||\cdot|||$ on a vector space V are called equivalent if there are constants $C_1, C_2 > 0$ such that*

$$C_1 \cdot \|x\| \leq |||x||| \leq C_2 \cdot \|x\| \quad \forall x \in V. \tag{B.9}$$

Remark B.3.5 Since the constants C_1 and C_2 in (B.9) are positive, this is equivalent to:

$$\frac{1}{C_2} \cdot |||x||| \leq \|x\| \leq \frac{1}{C_1} \cdot |||x|||.$$

It therefore does not matter which of the two norms in (B.9) is on the left or on the right side. If two norms are equivalent, each can be bounded by the other both from above and from below.

Equivalent norms define the same topology on a vector space. An important consequence is that, for the convergence of a sequence, it does not matter which of the equivalent norms is used to prove the convergence.

Lemma B.3.6

If $\|\cdot\|$ and $|||\cdot|||$ are two equivalent norms on a vector space V, then for a sequence $\left(x^{(k)}\right)_{k\in\mathbb{N}}$:

$$\lim_{k\to\infty} \|x^{(k)} - \widehat{x}\| = 0 \quad \Longleftrightarrow \quad \lim_{k\to\infty} |||x^{(k)} - \widehat{x}||| = 0,$$

where $\widehat{x} \in V$ is the limit with respect to the equivalent norms.

Proof:
Since the norms are equivalent,

$$\exists C_1, C_2 > 0: \; C_1 \cdot \|x\| \leq |||x||| \leq C_2 \cdot \|x\| \quad \forall x \in V.$$

(1) $\Rightarrow$ (2):
From the assumption

$$\lim_{k\to\infty} \|x^{(k)} - \widehat{x}\| = 0$$

it follows that:

$$\forall \varepsilon > 0 \; \exists k(\varepsilon) : \; \|x^{(k)} - \widehat{x}\| < \varepsilon \quad \forall k \geq k(\varepsilon).$$

With the upper bound in (B.9) we obtain:

$$|||x^{(k)} - \widehat{x}||| \leq C_2 \cdot \|x^{(k)} - \widehat{x}\| < C_2 \cdot \frac{\varepsilon}{C_2} = \varepsilon \quad \forall k \geq k\left(\frac{\varepsilon}{C_2}\right).$$

Hence

$$\lim_{k\to\infty} |||x^{(k)} - \widehat{x}||| = 0.$$

(2) ⇒ (1):
Analogously. From the assumption

$$\lim_{k\to\infty} |||x^{(k)} - \widehat{x}||| = 0$$

it follows that:

$$\forall \varepsilon > 0 \; \exists k(\varepsilon) : \; |||x^{(k)} - \widehat{x}||| < \varepsilon \quad \forall k \geq k(\varepsilon).$$

With the lower bound in (B.9) we obtain:

$$\|x^{(k)} - \widehat{x}\| \leq \frac{1}{C_1} \cdot |||x^{(k)} - \widehat{x}||| < \frac{1}{C_1} \cdot C_1 \cdot \varepsilon = \varepsilon \quad \forall k \geq k\left(C_1 \cdot \varepsilon\right).$$

Hence

$$\lim_{k\to\infty} \|x^{(k)} - \widehat{x}\| = 0.$$

□

§3.3 Equivalence of the p-Norms in $\mathbb{R}^n$ and $\mathbb{C}^n$

In this section, the equivalence of the p-norms is proved explicitly and the constants C_1 and C_2 are determined directly. These norms play a central role in numerical linear algebra, since concrete estimates between different norms are often needed in practice. For example, such estimates are relevant for assessing the accuracy of direct solution methods or the speed of convergence of iterative methods.

Moreover, it is helpful to deal with special cases first before considering the general case. They provide intuitive insights and lay a foundation on which abstract considerations are easier to follow. The general proof of the equivalence of norms in finite-dimensional normed vector spaces follows later in Section §3.4.

Lemma B.3.7 (Equivalence of $\|\cdot\|_1$, $\|\cdot\|_2$ and $\|\cdot\|_\infty$)
For all $x \in \mathbb{R}^n$ or $x \in \mathbb{C}^n$:

(a) $\|x\|_\infty \leq \|x\|_1 \leq n \cdot \|x\|_\infty$

(b) $\|x\|_\infty \leq \|x\|_2 \leq \sqrt{n} \cdot \|x\|_\infty$

(c) $\|x\|_2 \leq \|x\|_1 \leq \sqrt{n} \cdot \|x\|_2$

Hence (see also Remark B.3.5):

(a)* $\frac{1}{n} \cdot \|x\|_1 \leq \|x\|_\infty \leq \|x\|_1$

(b)* $\frac{1}{\sqrt{n}} \cdot \|x\|_2 \leq \|x\|_\infty \leq \|x\|_2$

(c)* $\frac{1}{\sqrt{n}} \cdot \|x\|_1 \leq \|x\|_2 \leq \|x\|_1$

Proof:
For the proof we need the following tools:

1. **Index k for (a) and (b):** Let $k \in \{1, \ldots, n\}$ be the index for which $|x_k| = \|x\|_\infty$ holds. This gives the largest component (in absolute value) of the vector x, and hence:
$$|x_k| = \max_{1 \le j \le n} |x_j| = \|x\|_\infty.$$

2. **Cauchy–Schwarz inequality for (c):**
$$x^T y \le \|x\|_2 \cdot \|y\|_2 \quad \forall x, y \in \mathbb{R}^n \quad \text{or} \quad |x^H y| \le \|x\|_2 \cdot \|y\|_2 \quad \forall x, y \in \mathbb{C}^n.$$
This is proved in Lemma B.3.8.

Regarding (a): By the definition of the index k, the lower bound follows:
$$\|x\|_1 = \sum_{i=1}^{n} |x_i| \ge |x_k| = \|x\|_\infty.$$
The upper bound follows since $|x_i| \le \|x\|_\infty$ for all i:
$$\|x\|_1 = \sum_{i=1}^{n} |x_i| \le \sum_{i=1}^{n} \|x\|_\infty = n \cdot \|x\|_\infty.$$

Regarding (b): By the definition of the index k, the lower bound follows:
$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} |x_i|^2} \ge \sqrt{|x_k|^2} = |x_k| = \|x\|_\infty.$$
For the upper bound we use that $|x_i|^2 \le \|x\|_\infty^2$ holds for all i:
$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} |x_i|^2} \le \sqrt{\sum_{i=1}^{n} \|x\|_\infty^2} = \sqrt{n} \cdot \|x\|_\infty.$$

Regarding (c): For $x = \mathbf{0}$ there is nothing to show; let $x \neq \mathbf{0}$. For $\alpha \in [0, 1]$ we have $\alpha \ge \alpha^2$. Therefore, for every $i \in \{1, \ldots, n\}$:
$$0 \le |x_i| \le \|x\|_1 \quad \Longrightarrow \quad \frac{|x_i|}{\|x\|_1} \in [0, 1] \quad \Longrightarrow \quad \frac{|x_i|}{\|x\|_1} \ge \left(\frac{|x_i|}{\|x\|_1} \right)^2.$$
The lower bound is obtained by summing, rearranging, and taking the square root:
$$1 = \sum_{i=1}^{n} \frac{|x_i|}{\|x\|_1} \ge \sum_{i=1}^{n} \left(\frac{|x_i|}{\|x\|_1} \right)^2 = \frac{\sum_{i=1}^{n} |x_i|^2}{\|x\|_1^2} = \frac{\|x\|_2^2}{\|x\|_1^2} \quad \Longrightarrow \quad \|x\|_1^2 \ge \|x\|_2^2.$$
For the upper bound we define:

- In the real case ($\mathbb{K} = \mathbb{R}$): the vector $y \in \mathbb{R}^n$ with

$$y_j = \begin{cases} 1, & x_j \geq 0, \\ -1, & x_j < 0. \end{cases} \tag{B.10}$$

  Then:

$$\|x\|_1 = x^T y \quad \text{and} \quad \|y\|_2 = \sqrt{n}.$$

  The Cauchy–Schwarz inequality yields:

$$\|x\|_1 = x^T y \leq \|x\|_2 \cdot \|y\|_2 = \sqrt{n} \cdot \|x\|_2.$$

- In the complex case ($\mathbb{K} = \mathbb{C}$): the vector $y \in \mathbb{C}^n$ with

$$y_j = \begin{cases} \frac{x_j}{|x_j|}, & x_j \neq 0, \\ 0, & x_j = 0. \end{cases} \tag{B.11}$$

  Then:

$$\|x\|_1 = x^H y \quad \text{and} \quad \|y\|_2 \leq \sqrt{n}.$$

  The Cauchy–Schwarz inequality for the Hermitian inner product yields:

$$\|x\|_1 = x^H y \leq \|x\|_2 \cdot \|y\|_2 \leq \sqrt{n} \cdot \|x\|_2.$$

- **Remark:** Although the real case follows directly from the complex case, so that from a purely mathematical point of view it would not be necessary to treat both cases separately, the separate treatment has great didactic value. The definition in (B.10) for the real case is more intuitively accessible and provides a clearer entry into the problem. Once this case is understood, students often find it easier to follow the analogous approach in the complex case, as in (B.11).

  This step-by-step approach fosters an important skill in the study of mathematics: to tackle more complex matters by solving simpler cases step by step and then generalizing. This method teaches one to approach challenges systematically and strategically – an essential skill for one's studies and beyond.



Lemma B.3.8 (Cauchy–Schwarz inequality)

For all $x, y \in \mathbb{C}^n$

$$|x^H \cdot y| \leq \|x\|_2 \cdot \|y\|_2,$$

and equality holds if and only if x *and* y *are linearly dependent. For* $x, y \in \mathbb{R}^n$ *we have* $x^H \cdot y = x^T \cdot y$.

Proof: For $y = \mathbf{0}$ both sides are zero. Let $y \neq \mathbf{0}$. For every $\alpha \in \mathbb{C}$ we have

$$0 \leq \|x - \alpha y\|_2^2 = (x - \alpha y)^H \cdot (x - \alpha y) = x^H x - \alpha \cdot x^H y - \overline{\alpha} \cdot y^H x + |\alpha|^2 \cdot y^H y.$$

The trick is to choose α such that $x - \alpha y$ is orthogonal to y, i.e.,

$$\alpha = \frac{y^H x}{y^H y} \quad \text{and hence} \quad \overline{\alpha} = \frac{x^H y}{y^H y}.$$

With $(y^H x) \cdot (x^H y) = |x^H y|^2$, the last three terms are then all equal to $|x^H y|^2 / \|y\|_2^2$, and we obtain

$$0 \leq \|x\|_2^2 - \frac{|x^H y|^2}{\|y\|_2^2} - \frac{|x^H y|^2}{\|y\|_2^2} + \frac{|x^H y|^2}{\|y\|_2^2} = \|x\|_2^2 - \frac{|x^H y|^2}{\|y\|_2^2}.$$

Multiplying by $\|y\|_2^2$ and taking the square root yields the inequality. Equality holds if and only if $\|x - \alpha y\|_2 = 0$, i.e., $x = \alpha y$. Conversely, if x and y are linearly dependent, then $y = \mathbf{0}$ or $x = \beta y$ with $\beta \in \mathbb{C}$, and in both cases equality holds by direct computation. ☐

§3.4 Equivalence of Norms in Finite-Dimensional Vector Spaces

Fortunately, in finite-dimensional vector spaces all norms are equivalent to each other. This means that every norm can be bounded by every other norm up to constant factors. In the following, this statement is proved.

Before the general proof is given, we need an auxiliary lemma with which the norm of a linear combination of vectors can be bounded from below by the sum of the absolute values of the coefficients. This result provides an important foundation for the proof of the equivalence of norms.

Lemma B.3.9 (Lower bound for the norm of a linear combination) *Let V be a normed vector space and $x_1, \ldots, x_n \in V$ linearly independent vectors. Then there exists a constant $C > 0$ such that*

$$\|\alpha_1 \cdot x_1 + \cdots + \alpha_n \cdot x_n\| \geq C \cdot (|\alpha_1| + \cdots + |\alpha_n|) \quad \forall \alpha_1, \ldots, \alpha_n \in \mathbb{K}. \tag{B.12}$$

Proof: If $\alpha_1 = \cdots = \alpha_n = 0$, the inequality holds trivially, so let $(\alpha_1, \ldots, \alpha_n) \neq (0, \ldots, 0)$. Let

$$S := |\alpha_1| + \cdots + |\alpha_n|, \quad \beta_j := \frac{\alpha_j}{S} \text{ for } j = 1, \ldots, n.$$

Then statement (B.12) is equivalent to:

$$\exists C > 0 : \quad \|\beta_1 x_1 + \cdots + \beta_n x_n\| \geq C \quad \text{for all } \beta_1, \ldots, \beta_n \in \mathbb{K} \text{ with } \sum_{j=1}^{n} |\beta_j| = 1. \tag{B.13}$$

Suppose this is not the case. Then there exists a sequence $(\beta_1^{(k)}, \ldots, \beta_n^{(k)})_{k \in \mathbb{N}}$ such that:

$$\|\beta_1^{(k)} x_1 + \cdots + \beta_n^{(k)} x_n\| < \frac{1}{k} \quad \text{and} \quad \sum_{j=1}^{n} \left|\beta_j^{(k)}\right| = 1 \quad \forall k \in \mathbb{N}. \tag{B.14}$$

If we define

$$y^{(k)} := \beta_1^{(k)} x_1 + \cdots + \beta_n^{(k)} x_n,$$

then $\|y^{(k)}\| \to 0$ for $k \to \infty$, i.e.,

$$\lim_{k \to \infty} \|y^{(k)}\| = 0. \tag{B.15}$$

We show that there exists a subsequence of $(y^{(k)})_{k \in \mathbb{N}}$ that converges to a vector $y \neq 0$. This leads to a contradiction:

- For every $j \in \{1, \dots, n\}$ the sequence $(\beta_j^{(k)})_{k\in\mathbb{N}}$ is bounded, since

$$\left|\beta_j^{(k)}\right| \leq \sum_{j=1}^{n} |\beta_j^{(k)}| = 1.$$

  By the Bolzano–Weierstrass theorem, $(\beta_1^{(k)})_{k\in\mathbb{N}}$ has a convergent subsequence. Denoting the corresponding index function by φ_1, we have:

$$\beta_1^{(\varphi_1(k))} \to \beta_1 \quad \text{for } k \to \infty.$$

- The subsequence $(\beta_2^{(\varphi_1(k))})_{k\in\mathbb{N}}$ is also bounded. Therefore it has a convergent subsequence. Again by the Bolzano–Weierstrass theorem we obtain a strictly increasing $\psi_2 : \mathbb{N} \to \mathbb{N}$ such that, for the index function $\varphi_2 := \varphi_1 \circ \psi_2$,

$$\beta_2^{(\varphi_2(k))} \to \beta_2 \quad \text{for } k \to \infty.$$

  Since $(\beta_1^{(\varphi_2(k))})_{k\in\mathbb{N}}$ is a subsequence of $(\beta_1^{(\varphi_1(k))})_{k\in\mathbb{N}}$, we still have $\beta_1^{(\varphi_2(k))} \to \beta_1$.

- We continue this process inductively. For every $j \in \{2, \dots, n\}$ there exists a strictly increasing $\psi_j : \mathbb{N} \to \mathbb{N}$ such that, for the index function $\varphi_j := \varphi_{j-1} \circ \psi_j$,

$$\beta_j^{(\varphi_j(k))} \to \beta_j \quad \text{for } k \to \infty.$$

  As subsequences of convergent sequences, $(\beta_i^{(\varphi_j(k))})_{k\in\mathbb{N}}$ still converge to β_i for all $i < j$.

- Finally, φ_n is a common index function that works for all $j \in \{1, \dots, n\}$. Thus $(\beta_j^{(\varphi_n(k))})_{k\in\mathbb{N}}$ converges to β_j for every j.

With $y := \sum_{j=1}^{n} \beta_j x_j$ it follows that

$$\lim_{k\to\infty} y^{(\varphi_n(k))} = y,$$

since

$$\|y^{(\varphi_n(k))} - y\| \leq \sum_{j=1}^{n} \left|\beta_j^{(\varphi_n(k))} - \beta_j\right| \cdot \|x_j\| \to 0.$$

Since $\sum_{j=1}^{n} |\beta_j| = 1$, there exists at least one $j \in \{1, \dots, n\}$ with $\beta_j \neq 0$. Because of the linear independence of $x_1, \dots, x_n$, we have $y \neq 0$. This contradicts the assumption that $\lim_{k\to\infty} \|y^{(k)}\| = 0$. Therefore, inequality (B.12) must hold. ☐

With this, the following theorem is easy to prove.

Theorem B.3.10 (Equivalence of norms in finite-dimensional vector spaces)

In a finite-dimensional vector space V, every norm $\|\cdot\|$ is equivalent to any other norm $|||\cdot|||$.

Proof: Let $\dim V = n$ and let $b_1, \ldots, b_n$ be a basis of V. Every $x \in V$ can be written as

$$x = \alpha_1 b_1 + \cdots + \alpha_n b_n,$$

where $\alpha_1, \ldots, \alpha_n \in \mathbb{K}$ are the coordinates of x with respect to the basis. By Lemma B.3.9 we have:

$$\exists \widetilde{C} > 0 : \ \|\alpha_1 b_1 + \cdots + \alpha_n b_n\| \geq \widetilde{C} \cdot \big(|\alpha_1| + \cdots + |\alpha_n|\big).$$

Moreover, the triangle inequality yields:

$$|||x||| = |||\alpha_1 b_1 + \cdots + \alpha_n b_n||| \leq \sum_{j=1}^{n} |\alpha_j| \cdot |||b_j|||.$$

Setting $\max\limits_{1 \leq j \leq n} |||b_j||| = M$, we obtain:

$$|||x||| \leq M \cdot \big(|\alpha_1| + \cdots + |\alpha_n|\big).$$

Combined with the lower bound, this gives:

$$|||x||| \leq M \cdot \frac{1}{\widetilde{C}} \cdot \|x\|.$$

If we define $C_2 := M \cdot \frac{1}{\widetilde{C}}$, then:

$$|||x||| \leq C_2 \cdot \|x\|.$$

Interchanging the roles of $\|\cdot\|$ and $|||\cdot|||$, one shows analogously that there exists a $C_1 > 0$ with:

$$C_1 \cdot \|x\| \leq |||x|||.$$

Since the constants C_1 and C_2 are independent of x, the claim follows. ☐

§3.5 Completeness: Finite-Dimensional Normed Vector Spaces Are Always Banach Spaces

First, we define in general what it means for a sequence in a normed vector space to converge, and what it means for a sequence to satisfy the Cauchy criterion. Then we show that every convergent sequence always satisfies the Cauchy criterion. An example from an infinite-dimensional vector space, however, illustrates that the converse does not hold in general. This means that there exist normed vector spaces in which there are Cauchy sequences that do not converge.

This motivates the definition of a *Banach space*: a normed vector space in which every Cauchy sequence converges is called a Banach space. This property is of great importance, since it is often easier to show that a sequence satisfies the Cauchy criterion than to prove its convergence directly.

Finally, we prove that every finite-dimensional normed vector space is always a Banach space.

Definition B.3.11 (Convergence and Cauchy sequences)

Let V be a normed vector space over $\mathbb{K}$ and $(x_k)_{k\in\mathbb{N}}$ a sequence in V.

(a) (x_k) converges *to $x \in V$ if*

$$\forall \varepsilon > 0 \; \exists N(\varepsilon) : \; \|x_k - x\| < \varepsilon \quad \forall k > N(\varepsilon).$$

One writes $\lim_{k\to\infty} x_k = x$ or $x_k \to x$.

(b) (x_k) *is a* Cauchy sequence *if*

$$\forall \varepsilon > 0 \; \exists N(\varepsilon) : \; \|x_k - x_\ell\| < \varepsilon \quad \forall k, \ell > N(\varepsilon).$$

The difference is that the definition of a Cauchy sequence does not refer to a limit. The limit does not even have to be known, or to exist in V.

Lemma B.3.12

Every convergent sequence in a normed vector space is a Cauchy sequence.

Proof: Let $x_k \to x$ and $\varepsilon > 0$. Choose N such that $\|x_k - x\| < \frac{\varepsilon}{2}$ for all $k > N$. For $k, \ell > N$ the triangle inequality gives

$$\|x_k - x_\ell\| \le \|x_k - x\| + \|x - x_\ell\| < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

□

The converse does not hold in general, as the following example shows.

Example B.3.13 (A Cauchy sequence without a limit)

Let V be the vector space of real sequences $x = (x_1, x_2, x_3, \dots)$ with only finitely many nonzero entries, with the norm

$$\|x\|_\infty := \max_{j\in\mathbb{N}} |x_j|.$$

V is infinite-dimensional, since the unit sequences $e_1, e_2, \dots$ are linearly independent. We consider

$$x_k := \Big(1, \frac{1}{2}, \frac{1}{3}, \dots, \frac{1}{k}, 0, 0, \dots\Big) \in V.$$

For $\ell > k$ the sequences x_k and x_ℓ differ only in the entries $k+1, \dots, \ell$, hence

$$\|x_k - x_\ell\|_\infty = \frac{1}{k+1} \quad \text{for } \ell > k.$$

So (x_k) is a Cauchy sequence. Suppose $x_k \to x \in V$. Since x has only finitely many nonzero entries, there is an m with $x_j = 0$ for all $j > m$. For all $k > m$ this gives

$$\|x_k - x\|_\infty \ge |(x_k)_{m+1} - x_{m+1}| = \frac{1}{m+1},$$

which contradicts $\|x_k - x\|_\infty \to 0$. Thus (x_k) has no limit in V.

The "natural" limit $(1, \frac{1}{2}, \frac{1}{3}, \dots)$ does not lie in V: the space has "holes".

Definition B.3.14 (Banach space)

A normed vector space V in which every Cauchy sequence converges (to a limit in V) is called complete *or a* Banach space.

From analysis we know that $\mathbb{R}$ is complete. So is $\mathbb{C}$, since a sequence of complex numbers is a Cauchy sequence if and only if its real and imaginary parts are. With Lemma B.3.9, this carries over to every finite-dimensional normed vector space (cf. also [14]).

Theorem B.3.15 (Finite-dimensional normed vector spaces are complete)

Every finite-dimensional normed vector space V over $\mathbb{K} = \mathbb{R}$ or $\mathbb{K} = \mathbb{C}$ is a Banach space.

Proof: For $V = \{\mathbf{0}\}$ there is nothing to show. Let $\dim V = n \geq 1$, let $b_1, \ldots, b_n$ be a basis of V, and let (x_k) be a Cauchy sequence in V with

$$x_k = \alpha_1^{(k)} b_1 + \cdots + \alpha_n^{(k)} b_n.$$

By Lemma B.3.9 there is a $C > 0$ with

$$C \cdot \left|\alpha_j^{(k)} - \alpha_j^{(\ell)}\right| \leq C \cdot \sum_{i=1}^{n} \left|\alpha_i^{(k)} - \alpha_i^{(\ell)}\right| \leq \|x_k - x_\ell\| \quad \text{for } j = 1, \ldots, n.$$

Hence each coordinate sequence $(\alpha_j^{(k)})_{k \in \mathbb{N}}$ is a Cauchy sequence in $\mathbb{K}$. Since $\mathbb{K}$ is complete, it converges to some $\alpha_j \in \mathbb{K}$. We set

$$x := \alpha_1 b_1 + \cdots + \alpha_n b_n \in V.$$

By the triangle inequality,

$$\|x_k - x\| = \Big\| \sum_{j=1}^{n} \big(\alpha_j^{(k)} - \alpha_j\big) b_j \Big\| \leq \sum_{j=1}^{n} \left|\alpha_j^{(k)} - \alpha_j\right| \cdot \|b_j\| \;\longrightarrow\; 0 \quad \text{for } k \to \infty,$$

so $x_k \to x \in V$. ∎

In particular, $\mathbb{R}^n$, $\mathbb{C}^n$ and $\mathbb{R}^{n \times n}$ are complete with respect to every norm. This is what was used for the Neumann series in Chapter 3.

Bibliography

[1] Anderson, E., Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney and D. Sorensen: *LAPACK Users' Guide*, 3rd edition, SIAM, Philadelphia, 1999.

[2] Bendixson, Ivar: Sur les racines d'une équation fondamentale, *Acta Mathematica* 25 (1902), 359–365.

[3] Benoît, Commandant: Note sur une méthode de résolution des équations normales provenant de l'application de la méthode des moindres carrés à un système d'équations linéaires en nombre inférieur à celui des inconnues (Procédé du Commandant Cholesky), *Bulletin Géodésique* 2 (1924), 67–77.

[4] Demmel, James W.: *Applied Numerical Linear Algebra*, SIAM, Philadelphia, 1997.

[5] Francis, J. G. F.: The QR transformation: a unitary analogue to the LR transformation, Parts I and II, *The Computer Journal* 4 (1961/62), 265–271 and 332–345.

[6] Gerschgorin, S.: Über die Abgrenzung der Eigenwerte einer Matrix, *Izvestiya Akademii Nauk SSSR*, no. 6 (1931), 749–754.

[7] Givens, Wallace: Computation of plane unitary rotations transforming a general matrix to triangular form, *Journal of the Society for Industrial and Applied Mathematics* 6 (1958), 26–50.

[8] Golub, Gene H. and Charles F. Van Loan: *Matrix Computations*, 4th edition, Johns Hopkins University Press, Baltimore, 2013.

[9] Hestenes, Magnus R. and Eduard Stiefel: Methods of conjugate gradients for solving linear systems, *Journal of Research of the National Bureau of Standards* 49 (1952), 409–436.

[10] Higham, Nicholas J.: *Accuracy and Stability of Numerical Algorithms*, 2nd edition, SIAM, Philadelphia, 2002.

[11] Horn, Roger A. and Charles R. Johnson: *Matrix Analysis*, 2nd edition, Cambridge University Press, 2012.

[12] Householder, Alston S.: Unitary triangularization of a nonsymmetric matrix, *Journal of the ACM* 5 (1958), 339–342.

[13] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, IEEE, 2019.

[14] Kreyszig, Erwin: *Introductory Functional Analysis with Applications*, John Wiley & Sons, New York, 1978.

[15] Kublanovskaya, V. N.: On some algorithms for the solution of the complete eigenvalue problem, *USSR Computational Mathematics and Mathematical Physics* 1 (1962), 637–657 (Russian original 1961).

[16] Overton, Michael L.: *Numerical Computing with IEEE Floating Point Arithmetic*, SIAM, Philadelphia, 2001.

[17] Saad, Yousef: *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, Philadelphia, 2003.

[18] Trefethen, Lloyd N. and David Bau III: *Numerical Linear Algebra*, SIAM, Philadelphia, 1997.

[19] Wilkinson, James H.: *The Algebraic Eigenvalue Problem*, Clarendon Press, Oxford, 1965.